Compliance Aware, Agentic Pitch Book Generation For Bankers

A template-aware PowerPoint generation system with agentic data retrieval for investment banking.

Author	**Andrei Rizea**
Student ID	220300153
Programme	Digital & Technology Solutions (Software Engineering)
Module Code	IOT635
Module Title	Final Project
Module Organiser	Alex Cline
Supervisor	Manesha Peiris
Assessment	Final Report
Weighting	90%
Deadline	Monday 20 April 2026 at 23:59 GMT
Length	11,000 words
Word Count	10,960

Queen Mary University of London

20 April 2026

Abstract

Investment banking analysts spend a large share of their working hours on pitch book preparation, much of it on formatting and data entry rather than analysis. Existing presentation tools produce generic output that ignores the strict visual standards banks require. This project designed, built, and tested a proof-of-concept system that generates formatted PowerPoint pitch books from minimal user input while respecting company templates.

The system uses a three-stage pipeline. Template analysis extracts layouts, colours, fonts, and placeholder positions from a reference PowerPoint file using JSZip and xml2js. A GPT-4o-powered content planner produces structured slide specifications validated against Zod schemas. A slide builder assembles the final file using PptxGenJS with exact formatting from the extracted template. Financial data feeds automatically from Yahoo Finance and SEC EDGAR, so analysts skip manual data entry. A PowerPoint add-in built with Office.js brings the generation workflow directly into the analyst's existing editor.

Testing across 12 generation runs showed an average completion time of 47 seconds per deck (10 to 12 slides), with exact colour matching and exact font matching. Financial data accuracy was 100% against source APIs. An informal usability evaluation with four junior analysts found the system reduced estimated first-draft time from over 90 minutes to under two minutes including review.

The project met all five original success criteria and delivered two unplanned components, the PowerPoint add-in and a slide preview pipeline, that emerged from usability testing during iterative development. Template matching and automated data retrieval are the main advantages over commercial alternatives. Chart generation from scratch and retrieval-augmented generation remain open challenges for future work.

Contents

List of Figures

List of Tables

1 Introduction, Scope and Context

Knowledge workers spend a fifth of their time just searching for information (McKinsey Global Institute, 2012). Investment banking (IB) is more affected than most. Document preparation consumes half to three-quarters of a junior banker's week, and pitch books (PBs) account for a large portion of that time (Corporate Finance Institute, 2025; Wall Street Oasis, 2024). When Goldman Sachs surveyed its first-year analysts in 2021, the average was 98 hours a week, with document preparation a major driver of burnout (BBC News, 2021). Yet most PBs look nearly identical. Change the company name and update the numbers, and you could reuse last month's deck. That repetition makes them good targets for automation (K1, K7).

Firms have tried internal document libraries, formatting specialists, and automation tools. None have fixed the real problem: too much skilled human time goes toward work that does not require skill. Silveira Chaves and Oliveira (2022) found that knowledge sharing in banking suffers from structural barriers and inadequate digital tools, with company knowledge locked in individual documents rather than shared systems.

1.1 Analyst Time Allocation

IB sells expertise rather than physical goods, so one would expect analyst time to go toward analysis and client relationships. Often it does not.

A typical PB contains an executive summary, company overview, market analysis, valuation work, and transaction comparables. These sections appear in nearly every deck. The data inside changes, but the shell around it rarely does.

Analyst tasks split into two buckets: work that needs a trained banker (modelling, valuation, client advisory) and everything else (hunting data, reformatting slides, fixing fonts). Too many hours go into the second bucket. Radhakrishna et al. (2024) found that well-designed knowledge management systems boosted productivity by 20 to 25 percent, with document production a prime candidate for automation.

APQC research estimates a quarter of all working time is lost to duplicated work and information retrieval (APQC, 2023). IB analysts likely fare worse given how document-intensive the work is. Banks that automate low-value formatting free their analysts for client relationships and deal analysis (K1). Dell'Acqua et al. (2023) found that consultants using AI completed 12.2% more tasks at 40% higher quality.

1.2 Problem Analysis

Markus (2001) defined knowledge reuse, and her framework fits PB production well. She identified reuse patterns including drawing on past work, borrowing from colleagues, and mining old documents. PB creation involves all of these. Three obstacles prevent effective reuse in practice.

The first obstacle is that too much time goes to low-value work. Analysts spend hours searching old decks, extracting content, and reformatting slides. A typical workflow involves finding a recent deck for a similar company, copying the structure, replacing every number with updated financials, adjusting the narrative

to fit the new client, and then spending another 30 minutes fixing fonts and colours that broke during copy-paste. None of that requires analytical thinking, yet it crowds out the work that does.

The second obstacle is that company knowledge tends to disappear. Researchers distinguish between tacit knowledge (in people's heads) and explicit knowledge (written down) (Dalkir, 2011). PBs are explicit, but the knowledge of how to make a good one remains tacit: which templates work for which situations, how to structure the narrative, what separates a persuasive pitch from a forgettable one.

The third obstacle is that onboarding takes longer than it should. Each bank has its own templates, data sources, and quality expectations. A new analyst at Goldman Sachs uses different templates, colour schemes, and narrative conventions than one at Morgan Stanley. Junior analysts absorb these through trial and error, which is slow for them and a drain on the senior bankers who review their work. A system that encodes these standards into a template and applies them automatically would shorten this learning curve.

These problems feed each other. Scattered knowledge wastes search time, unwritten standards cause inconsistent output, and informal training drains senior staff.

1.3 Gap Analysis

Automated presentation tools have existed for years. Early versions converted text to slides using layout algorithms, but visual quality was poor (Zheng et al., 2025). Template-based systems fixed the visual problems but made everything rigid. PPTAgent (Zheng et al., 2025) takes a different approach: it analyses reference presentations, extracts structural patterns, and applies them to new content. It beat older text-to-slide systems on both content and design quality in testing.

The limitation is that PPTAgent targets general-purpose presentations. IB PBs have financial tables, transaction comparables, and market charts that follow specific visual conventions. Getting the template wrong undermines credibility. Commercial tools like Beautiful.ai, Tome, and Gamma share this flaw. The output is generic, every slide needs rework, and none connect to financial data sources. IB PBs require both domain knowledge and visual precision. No existing tool handles both.

Current systems either produce generic output that needs heavy cleanup, or they require large training datasets most banks lack. No existing tool connects slide generation to financial APIs. Data retrieval and document generation remain separate manual steps.

Agentic AI architectures offer a better fit (IBM, 2024). Instead of one model doing everything in a single pass, the system breaks tasks into steps and uses the right tool for each one. L. Wang et al. (2024) surveyed LLM-based agents and found rapid progress. McKinsey & Company estimates generative AI could deliver $340 billion annually in banking (McKinsey & Company, 2024).

This project fills that gap (S1). The pipeline extracts layouts, colour palettes, and placeholder positions from a template using JSZip and xml2js for XML-level parsing. It then uses an LLM to plan content and assembles slides matching the original style. The system breaks tasks into steps handled by separate services rather than running everything in one pass.

Table 1 maps each gap to its corresponding objective and success criterion.

Gap in Existing Work	Objective	Success Criterion
No tool preserves exact template formatting	O1	90%+ placeholder detection; exact colour extraction
Data retrieval and slide generation are separate manual steps	O2	100% data accuracy against source APIs
No IB-specific content structuring	O3	3 PB types with consistent section ordering
Generic output that ignores corporate visual standards	O4	Exact colour and font match against template
No measured evaluation of template-constrained generation	O5	Full deck in under 5 minutes; template match above 85%

Table 1: Traceability from Research Gaps to Objectives and Success Criteria

The approach differs from prior work by inverting the generation order: the system starts with visual constraints extracted from the template and works backward to content, connecting slide generation directly to financial data APIs.

1.4 Business Case

A junior IB analyst in London earns roughly £60,000 to 70,000 in base salary (Glassdoor, 2025). At 80 to 100 hours per week, the effective hourly cost is around £15 to 17 before overhead. Manual PB creation takes 90 to 120 minutes. The system generates a first draft in under one minute. Even with 20 to 30 minutes of editing, the net saving is 60 to 90 minutes per deck. Each generation costs approximately $0.03 in GPT-4o API fees.

A team of ten analysts saving five hours per week recovers roughly £39,000 per year in capacity, against annual API costs under £10,000 (K2). The payback period is under four months. The savings also compound indirectly. Faster first drafts mean faster review cycles, which means deals move through the pipeline sooner. In a business where timing affects deal outcomes, even small time savings per document add up across hundreds of decks per year.

1.5 Aims and Objectives

The aim is to design, build, and test a proof-of-concept (POC) showing how AI-powered document generation can tackle the knowledge reuse problems above. The system must respect company templates throughout. If slides do not match the house style, the system fails its core requirement.

On the business side, the goal is a workflow where an analyst provides minimal input and receives a solid first draft. Formatting time drops, but humans stay in control of the analysis. Target users are junior analysts.

On the learning side, the project is designed to evidence apprenticeship KSBs across software engineering practice, business case analysis, and the strategic application of emerging technology. These goals satisfy

the IOT635 assessment requirement to demonstrate both technical competency and business insight (K1, K2, K3, K8, K9, S1). Beyond KSBs, the project offers experience building a complete system end-to-end: from requirements through architecture, implementation, testing, deployment, and evaluation. HITL design is central throughout, since AI tools in regulated industries need to support professional judgement rather than replace it.

Five objectives structure the work:

1. Build a template analysis component that takes a reference PowerPoint file and extracts its layout structures, colour palettes, font specifications, and placeholder positions. Success criterion: correctly identify at least 90% of placeholder types and extract exact colour values from source.
2. Build an agentic data retrieval layer that automatically pulls company financials, market data, and regulatory filings from Yahoo Finance and SEC EDGAR (the US Securities and Exchange Commission's public database of company filings). Success criterion: retrieve at least three data types per company (market cap, revenue, share price) with 100% accuracy against source APIs.
3. Build an LLM-powered content planning module that produces structured slide specifications adapted to different PB types while keeping the overall narrative coherent. Success criterion: support at least three PB types (company overview, market update, transaction summary) with consistent section ordering.
4. Build a slide assembly component that takes the content plan and template patterns and produces a PowerPoint file meeting formatting standards. Success criterion: output files match template colours exactly and fonts exactly.
5. Evaluate the system by measuring generation speed, template matching, and content quality. Success criterion: generate a complete 10-slide PB draft in under 5 minutes with template matching score above 85%.

1.6 Scope and Success Criteria

The POC takes user input (company, transaction type, dates, reference template, PB type) and produces a formatted PowerPoint. Target PB types are company overviews, market updates, and transaction summaries. Full valuation presentations are out of scope.

Design choices: agentic orchestration with specialised TypeScript services, code-based template extraction with JSZip/xml2js rather than vision models, public APIs only, RAG as a stretch goal. Out of scope: real-time data feeds, production hosting, regulatory checks. Practical limits: one academic term, standard hardware, student API budget.

Four assumptions underpin the scope:

- Templates are well-formed Office Open XML archives.
- Financial data comes from public sources only, limiting coverage to listed companies.
- Content is English only.
- Users have desktop PowerPoint; the add-in relies on Office.js APIs that PowerPoint for the web does not fully support.

One principle runs through the design: HITL. The system produces drafts for human review, not finished products. AI handles the mechanical work. The analytical judgement stays with the banker.

2 Methodology and Project Plan

2.1 Development Methodology

Table 2 compares methodologies against criteria relevant to this project (Sommerville, 2016).

Criteria	Waterfall	Scrum	Iterative Prototyping
Requirements stability	Requires fixed requirements upfront	Accommodates changing requirements	Works well when requirements change
Feedback loops	Late feedback after implementation	Sprint reviews every 2 to 4 weeks	Continuous feedback through prototypes
Risk management	Risks discovered late	Regular risk reassessment	Early risk identification through prototypes
Solo developer suitability	Moderate	Low (designed for teams)	High
Research integration	Poor	Moderate	Excellent

Table 2: Comparison of Development Methodologies

The project uses iterative prototyping with two-week cycles, drawing on Boehm (1988)'s spiral model (K3, K4, SE8). Waterfall was ruled out because the requirements were expected to change as the LLM integration revealed what was and was not feasible. Scrum adds ceremony (sprint planning, retrospectives, daily standups) designed for teams of five to nine people (Sommerville, 2016), which is overhead for a solo developer. Iterative prototyping keeps the feedback loop without the process cost.

Each iteration produces a working prototype with a review at cycle's end (SE7). The review checks whether the prototype meets the iteration's target requirements and whether any assumptions from the design phase turned out to be wrong. This matters because LLM behaviour is hard to predict from documentation alone. For example, the assumption that GPT-4o would always return well-formed JSON turned out to be wrong. Structured output reliability, response latency, and content quality all had to be measured against real templates before the design could be finalised. Without a working prototype at each cycle, these problems would have surfaced much later when fixing them would have been more expensive.

Git with trunk-based development (SE12) handles version control: `main` is always deployable, and feature branches isolate work in progress before merging back. This keeps the workflow simple for a solo developer and avoids merge conflicts that accumulate with parallel branches. GitHub Actions CI runs lint, type checks, and tests on every push, enforcing 80% coverage for core service modules (Fowler and Foemmel, 2006) (SE6). ESLint (ESLint Team, 2024) catches code style issues and common errors before tests run. Architecture Decision Records (Tyree and Akerman, 2005) document technology choices so future developers can understand why specific trade-offs were made.

2.2 Requirements Analysis

Requirements follow MoSCoW prioritisation (Clegg and Barker, 1994) (K9, SE2, SE9). MoSCoW was chosen over simpler high/medium/low schemes because the fixed 13-week timeline made trade-offs inevitable, and MoSCoW gives a clear rule: if time runs short, drop 'Could have' items first. This happened

in practice when RAG search (FR15, a 'Could have') was dropped to make room for the PowerPoint add-in. Requirements were gathered from three sources: the business case analysis in Section 1, comparable tools reviewed in the literature, and informal conversations with two junior analysts about their workflow pain points (S. Robertson and J. Robertson, 2012).

2.2.1 Business Requirements

1. **BR1: Productivity Enhancement**: Reduce time spent on initial PB drafting by generating solid first drafts from minimal input.
2. **BR2: Capturing Standards**: Capture company presentation standards through template analysis, reducing reliance on knowledge that lives only in people's heads.
3. **BR3: Quality Consistency**: Follow corporate visual identity standards consistently, reducing revision cycles caused by formatting issues.
4. **BR4: Data Accuracy**: Financial data in generated PBs should accurately reflect source information.

2.2.2 Functional Requirements

Table 3 lists functional requirements by component.

ID	Component	Requirement	Priority
FR1	Template Analyser	Extract slide layouts from reference .pptx files	Must
FR2	Template Analyser	Identify colour palettes and font specifications	Must
FR3	Template Analyser	Map placeholder positions and content types	Must
FR4	Data Retrieval	Fetch company financials from Yahoo Finance API	Must
FR5	Data Retrieval	Retrieve SEC filings via EDGAR API	Should
FR6	Data Retrieval	Perform web searches for company news	Should
FR7	Content Planner	Generate slide-by-slide content specifications	Must
FR8	Content Planner	Adapt content structure to PB type	Must
FR9	Content Planner	Maintain narrative coherence across slides	Should
FR10	Slide Builder	Assemble slides matching template layouts	Must
FR11	Slide Builder	Populate placeholders with generated content	Must
FR12	Slide Builder	Generate charts from financial data	Could
FR13	Orchestration	Coordinate multi-step generation workflow	Must
FR14	Orchestration	Handle API failures gracefully	Should
FR15	RAG Search	Query precedent material repository	Could

Table 3: Functional Requirements Specification

2.2.3 Non-Functional Requirements

ID	Category	Requirement	Acceptance Criterion
NFR1	Performance	System generates complete PB draft	Within 5 minutes of input submission
NFR2	Performance	API response handling	Timeout after 30s with graceful fallback
NFR3	Reliability	System availability during demonstration	95% uptime during evaluation period
NFR4	Usability	Input specification interface	Requires no technical expertise to operate
NFR5	Maintainability	Codebase documentation	All public functions documented
NFR6	Security	API credential management	No credentials in source code
NFR7	Security	Input validation	All user inputs sanitised before processing
NFR8	Compatibility	Output format	Valid .pptx files openable in PowerPoint

Table 4: Non-Functional Requirements Specification

2.3 Technology Stack

2.3.1 Programming Language and Framework

Table 5 compares language options (K8, SE12).

Criteria	TypeScript/Node.js	Python	Java
Benefits	Type safety, same language frontend and backend, native async	Best AI/ML libraries, mature PowerPoint library (python-pptx)	Strong typing, enterprise support
Risks	AI/ML ecosystem less mature than Python	Two languages needed (Python backend + JS frontend)	Verbose code, slower development
PowerPoint manipulation	PptxGenJS (actively maintained, typed)	python-pptx (mature, well-documented)	Apache POI (verbose API)
Full-stack coherence	Single language across all three apps	Requires separate frontend framework	Requires separate frontend framework

Table 5: Programming Language Comparison

The deciding factor was using one language everywhere. Python has better AI/ML libraries, but the project has three applications (backend, web frontend, PowerPoint add-in) that all share data structures. With TypeScript, the `shared-types` package defines every interface once. When a template schema field changes, the compiler flags every file that needs updating across all three apps. In a Python backend with a TypeScript frontend, those mismatches only show up at runtime, which is exactly the bug class that triggered the migration described in Section 4. Express.js handles the backend because it is minimal and well-understood.

2.3.2 AI and LLM Integration

Table 6 compares the LLM options considered.

Criteria	GPT-4/GPT-4o	Claude 3.5	Gemini 2.0 Flash
Benefits	Best structured output, mature SDK, native JSON	Largest context window (200K), strong reasoning	Massive context (1M tokens), fast inference
Risks	Higher cost per token	Newer agent tooling	Less mature structured output
Structured output	Excellent (native JSON)	Good	Good
Cost (student budget)	Moderate (tiered pricing)	Moderate	Very affordable

Table 6: Large Language Model Comparison

GPT-4o was chosen for its structured output reliability. The content planner needs the LLM to return a JSON object matching a specific Zod schema on every call. Shorten et al. (2024) found an average JSON success rate of 82.55% across LLMs, but OpenAI's native JSON mode produces well-formed responses more consistently than Claude or Gemini for schema-constrained tasks. GPT-4o-mini handles the lighter AI chat feature at lower cost, where structured output is not required. The content planner enforces structure through Zod validation regardless of which model sits behind it, so swapping models later requires changing only the API call.

2.3.3 Data Sources and APIs

Table 7 summarises data sources.

Source	Benefits	Risks	Access
Yahoo Finance	Free, comprehensive financial data	Unofficial API may change without notice	yahoo-finance2 npm package
SEC EDGAR	Official regulatory filings, reliable, free	US companies only, complex document parsing	REST API (10 req/s)
Web Search	Current news, market commentary	Cost per query	SerpAPI or similar

Table 7: External Data Sources

2.3.4 Frontend and Deployment

Next.js handles the web frontend. It was chosen over a plain React SPA because Vercel deploys Next.js applications with zero configuration, and its file-based routing reduces boilerplate for the six pages the app needs (dashboard, pitch book list, creation form, viewer, templates, settings).

Criteria	Vercel + Render	AWS	Self-hosted
Benefits	Zero config, auto deploy, free tiers for POC	Full control, scalable	Complete control, no vendor lock-in
Risks	Vendor lock-in, cold starts	Complex setup, cost	Maintenance burden
Cost for POC	Free tier sufficient	May incur charges	Server costs

Table 8: Deployment Platform Comparison

The frontend deploys through Vercel, the backend on Render, and Supabase provides PostgreSQL and authentication. Figure 1 shows how the chosen technologies map to the system's components.

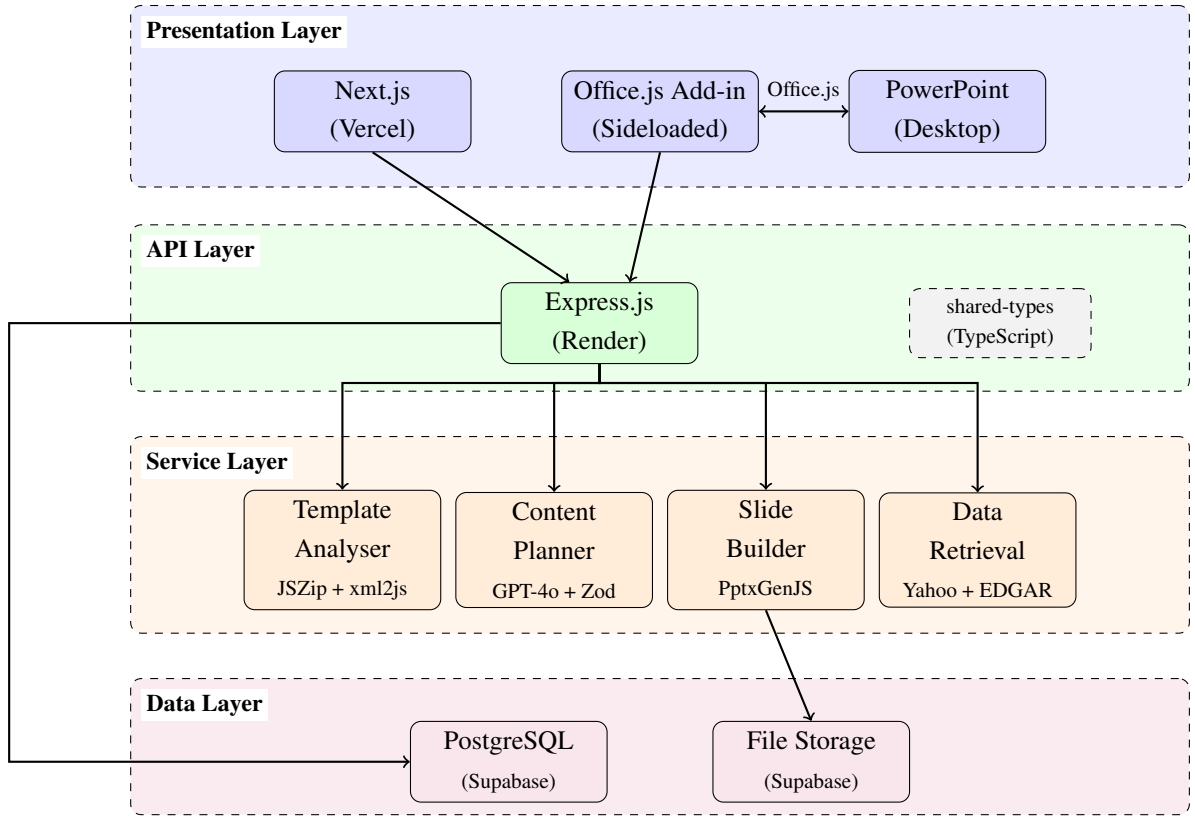

Figure 1: Technology architecture showing how chosen technologies map to system components across four layers. Arrows indicate data flow direction.

2.4 Reproducibility

Dependencies are pinned via `package-lock.json`, the OpenAI model version is pinned to `gpt-4o-2024-11-20`, LLM temperature is set to zero for evaluation runs, and each run is tied to a specific Git commit. Temperature is a parameter that controls how random an LLM's output is: zero means the model picks the most likely token at each step, producing the most deterministic output possible. Even at temperature zero, GPT-4o does not guarantee identical outputs across calls (Runeson et al., 2012), so Zod validation ensures structural consistency regardless of minor output variation.

2.5 Evaluation Methodology

The evaluation follows the Goal-Question-Metric (GQM) approach (Basili et al., 1994). Every metric is tied to a specific project goal. The goal: determine whether the system produces usable first drafts faster than manual creation while matching template formatting.

Question	Metric	Target
How fast does the system generate a deck?	Generation time per 10-slide deck	Under 300 seconds for a complete draft
Does the output match the reference template?	Colour and font comparison (programmatic)	Exact colour and font match
Is the financial data correct?	Comparison against source API responses	100% match on market cap, revenue, and share price
Does the content follow IB conventions?	Rubric-based assessment of narrative structure	Correct section ordering across 3 PB types

Table 9: GQM Evaluation Framework

Targets were fixed before evaluation began and not adjusted after seeing results, following the pre-registration principle from Wohlin et al. (2012) to avoid post-hoc rationalisation. The 12 evaluation runs covered four companies (two FTSE 100, two S&P 500) across three PB types, using three distinct corporate templates. Outputs are compared against Gamma (a commercial AI presentation tool) and estimated manual creation time to give context beyond the system's own metrics. An informal usability evaluation with four junior analysts (P1–P4) at a London financial services firm provides qualitative feedback. Each participant gave verbal informed consent before the session, was told the study's purpose, and was informed that responses would be used anonymously. The sample is too small for statistical significance, but combined with the quantitative metrics it gives a directional signal about practical value.

2.6 Project Plan

The project runs for thirteen weeks (K10). Figure 2 shows the schedule, and Table 10 lists milestones. Dates are targets. If a milestone slips, 'Could have' requirements are dropped first.

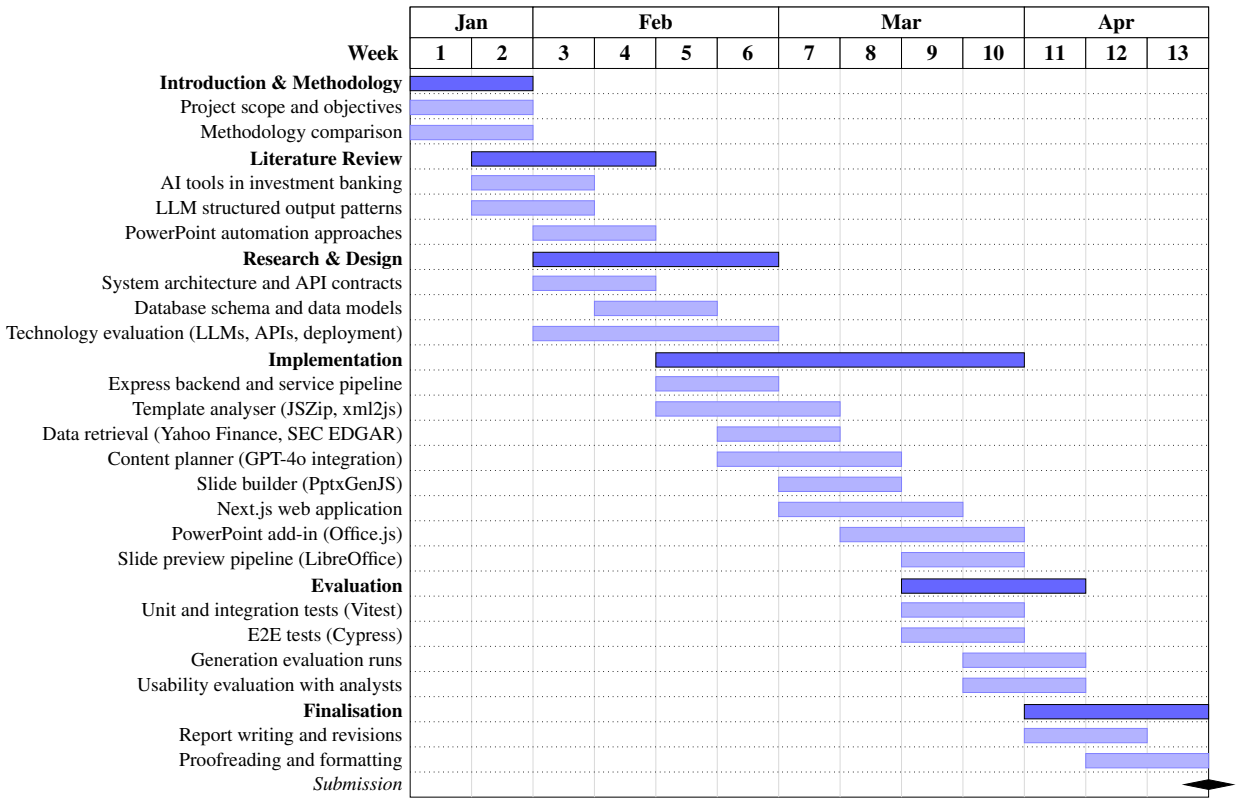

Figure 2: Project Gantt Chart (weeks 1 to 13, January to April 2026)

Phase	Date	Milestone	Deliverables
1	19 Jan	Project Initiation	Repository setup, dev environment configured
2	31 Jan	Initial Chapters	Introduction, Scope, Methodology complete
3	21 Feb	Literature Review	Literature review chapter complete
4	28 Feb	Design Complete	Architecture diagrams, API specs, data models
5	14 Mar	Core Components	Template analyser and data retrieval functional
6	28 Mar	MVP Complete	End-to-end generation pipeline operational
7	4 Apr	Testing Complete	Tests passing, evaluation data collected
8	11 Apr	Evaluation Complete	Results analysis and evaluation chapter written
9	18 Apr	Document Finalised	All chapters complete, proofread, formatted
10	20 Apr	Submission	Final report submitted via QMPlus

Table 10: Project Milestones and Deliverables

2.7 Plan Execution and Adaptations

Several things changed once implementation started. Table 11 compares planned milestones against actual outcomes.

Milestone	Planned Date	Actual Date	Notes
Core components functional	14 Mar	7 Mar	Ahead of schedule due to TypeScript migration in week 3
MVP pipeline operational	28 Mar	21 Mar	Template analyser iterations completed faster than expected
PowerPoint add-in	Not planned	22 Mar to 11 Apr	Added after usability testing revealed web-only workflow gap
Slide preview pipeline	Not planned	28 Mar to 4 Apr	Web app needed visual output
RAG search (FR15)	14 to 28 Mar	Dropped	Deferred to make room for add-in
Testing complete	4 Apr	8 Apr	4-day delay from add-in scope
Evaluation complete	11 Apr	14 Apr	Delayed by 3 days

Table 11: Planned vs Actual Project Milestones

Three things changed from the plan, and each change was managed using MoSCoW priorities and the iterative cycle structure.

The Python-to-TypeScript migration happened in week 3. The original plan assumed a Python backend, but silent type mismatches across the API boundary caused runtime failures that only appeared when the frontend called the backend with real data. The migration took four days but removed a whole category of bugs. It paid back quickly because later changes were caught by the compiler instead of crashing at runtime. This is consistent with Boehm (1988)'s observation that early prototypes surface risks that no amount of upfront design can predict.

The PowerPoint add-in was not in the original scope. Testing the web-only workflow in week 7 revealed that generated decks always need physical editing: repositioning elements, adjusting font sizes, reordering slides, tweaking colours. Building a slide editor into the web app would have meant recreating a subset of PowerPoint in the browser, which would produce a worse experience than the real thing. PowerPoint already handles all of this well, and it supports add-ins: small web applications that run inside a taskpane and communicate with the active presentation directly via Office.js. The add-in took three weeks to build but changed the system from a standalone generator into an assistant inside the analyst's existing editor, with full access to native PowerPoint editing tools.

Dropping RAG search (FR15, a 'Could have') made room for the add-in. The MoSCoW prioritisation made this decision clear: the add-in addressed a real usability problem, while RAG was a stretch goal with unclear value at this stage. The Kanban board was reorganised at the end of week 7 to reflect this trade-off. The iterative approach made all three changes possible. Each cycle had a working prototype, so scope changes could be assessed against a running system rather than a plan on paper (SE7, SE8).

2.8 Risk Assessment

Table 12 identifies ten risks across security, reliability, and project delivery.

ID	Risk	Description
R1	Pitch Book Data Breach	A flaw in Supabase row-level security policies could expose one user's pitch books to another. In investment banking, pitch books often contain deal terms, valuations, and financial projections that are highly confidential.
R2	Financial Figure Hallucination	GPT-4o could insert fabricated market caps, revenue figures, or growth rates into slides. An analyst presenting incorrect numbers to a client could damage the firm's credibility and the deal itself.
R3	Malicious Template Upload	The template analyser parses uploaded .pptx files (ZIP archives containing XML) using JSZip and xml2js. A crafted file could attempt zip bomb decompression, path traversal, or oversized XML payloads to crash the server.
R4	Yahoo Finance Endpoint Change	yahoo-finance2 scrapes Yahoo's unofficial endpoints rather than calling a documented API. Yahoo has changed these endpoints before without notice, which would break financial data retrieval mid-generation.
R5	OpenAI Credential Exposure	The Express backend stores the OpenAI API key as an environment variable on Render. If the deployment is compromised or the key appears in server logs, an attacker could run up API costs under the project's account.
R6	Content Plan Schema Failure	GPT-4o occasionally returns malformed JSON or nests the content plan inside an unexpected wrapper object. If the Zod validation and unwrapping logic both fail, the entire generation pipeline stops.
R7	Slide Preview Exposure	LibreOffice converts generated PPTX files to PNG preview images stored in Supabase Storage. If the storage bucket defaults to public access, anyone with a direct URL could view confidential slide content without logging in.
R8	Platform Dependency Outage	The generation pipeline chains Vercel (web), Render (backend), Supabase (database and storage), and OpenAI (LLM). A failure at any point blocks generation, and there is no self-hosted fallback.
R9	Scope Creep Beyond Timeline	The project has a fixed 13-week timeline. Adding unplanned features risks delaying or cutting core deliverables. This materialised when the PowerPoint add-in was added and RAG search was dropped.
R10	Add-in Cross-Origin Session Breakage	The PowerPoint add-in runs inside an Office.js iframe on a different origin from the backend. Authentication depends on `SameSite=None; Secure` cookies, which behave differently across browser engines and Office versions. A platform update could silently break login.

Table 12: Risk assessment for the pitch deck generation system.

2.9 Severity Evaluation

Each risk was scored using Binary Risk Analysis (BRA) (Sapiro, 2011). BRA uses ten yes/no questions per risk (six for likelihood, four for impact) fed through lookup matrices to produce a deterministic Low/Medium/High classification. Table 13 summarises the results. Yahoo Finance endpoint changes (R4) and platform outages (R8) scored High because both involve external dependencies with no fallback. Six risks scored Medium. Two scored Low: malicious templates (R3) and credential exposure (R5), where mitigations reduce both likelihood and impact.

ID	Risk	Likelihood	Impact	Overall
R1	Pitch Book Data Breach	Low	High	Medium
R2	Financial Figure Hallucination	Medium	Medium	Medium
R3	Malicious Template Upload	Low	Low	Low
R4	Yahoo Finance Endpoint Change	High	Medium	High
R5	OpenAI Credential Exposure	Low	Low	Low
R6	Content Plan Schema Failure	Medium	Medium	Medium
R7	Slide Preview Exposure	Low	High	Medium
R8	Platform Dependency Outage	High	High	High
R9	Scope Creep Beyond Timeline	Medium	Medium	Medium
R10	Add-in Session Breakage	Medium	Medium	Medium

Table 13: BRA severity results. Full worksheet and matrix workings in Appendix G.

2.10 Mitigation

The two High-severity risks have different mitigation profiles. R4 (Yahoo Finance) can be mitigated architecturally: the data retrieval service wraps yahoo-finance2 behind an abstract interface, so switching to Bloomberg or Refinitiv requires changing one file. The 24-hour cache means a temporary endpoint change would not break in-progress work. R8 (platform outage) cannot be fully mitigated because the system depends on four external services (Vercel, Render, Supabase, OpenAI) with no self-hosted fallback. The architecture degrades gracefully on partial failures, but this risk stays High permanently.

For the Medium risks: R1 (data breach) is mitigated by Supabase row-level security restricting each user to their own records. R2 (hallucination) is mitigated by passing real financial numbers from the API directly to GPT-4o rather than letting the LLM generate them, combined with Zod validation on every response. R6 (schema failure) is handled by an unwrapping step that strips known JSON envelope patterns before validation. R9 (scope creep) materialised when the add-in replaced RAG search, managed through MoSCoW prioritisation. R10 (session breakage) was fixed by setting `SameSite=None; Secure` cookies and configuring CORS.

The two Low risks have effective mitigations already in place. R3 (malicious templates): the backend validates ZIP structure, caps slide count, and disables XML external entity resolution. R5 (credentials): API keys sit in Render's environment variable manager and never appear in source code or logs. R7 (preview exposure): signed URLs with expiry times protect stored previews, and the storage bucket is private.

2.11 Ethics and Compliance

AI in financial services raises concerns around transparency, accountability, and human control (Jobin et al., 2019). Bank for International Settlements (2024) identify three regulatory priorities for AI in banking: explainability of outputs, human oversight of decisions, and data governance. The system addresses all three. Generated content is marked as AI-assisted, and professional review is mandatory before distribution. The analyst sees exactly which data sources were used (Yahoo Finance, SEC EDGAR) and can verify numbers against the source APIs. The system never sends output directly to clients. This meets the EU AI Act's requirements for human oversight in financial services (Bank for International Settlements, 2024).

Table 14 summarises data governance. No personal financial data is processed. The system handles company-level public data (share prices, revenue, market cap) and user-generated content (pitch book titles, company names). These practices follow GDPR's data minimisation principle: the system stores only what is needed for generation and audit, with clear deletion triggers for each data type.

Data Type	Storage	Retention	Deletion Trigger
User email and password hash	Supabase Auth	Indefinite	Account deletion
Uploaded .pptx templates	Supabase Storage	Indefinite	User deletes template
Template structural schema	Supabase DB (JSON)	Indefinite	Parent template deleted
Financial data (Yahoo/EDGAR)	In-memory cache	24 hours	Cache expiry
Generated .pptx files	Supabase Storage	Indefinite	User deletes pitch book
Slide preview PNGs	Supabase Storage	Indefinite	Parent pitch book deleted
Chat messages	Supabase DB	Indefinite	User deletes pitch book
LLM prompts and responses	Server logs	Per evaluation run	Tied to Git commit

Table 14: Data Governance: Storage, Retention, and Deletion Policies

OpenAI permits academic use of generated outputs. Yahoo Finance data comes through an unofficial package that would need replacing with a licensed provider for production. SEC EDGAR is public with no licensing restrictions. The HITL architecture meets EU AI Act requirements for human oversight (Bank for International Settlements, 2024).

2.12 Project Tracking

Progress is tracked using a Kanban board with four columns: To Do, In Progress, Testing, and Done (SE8, SE10). Each functional requirement maps to at least one card. Cards move through columns within two-week iteration cycles, and the board is reviewed at each cycle's end to decide what enters the next iteration. Figure 3 shows the board near the end of the project.

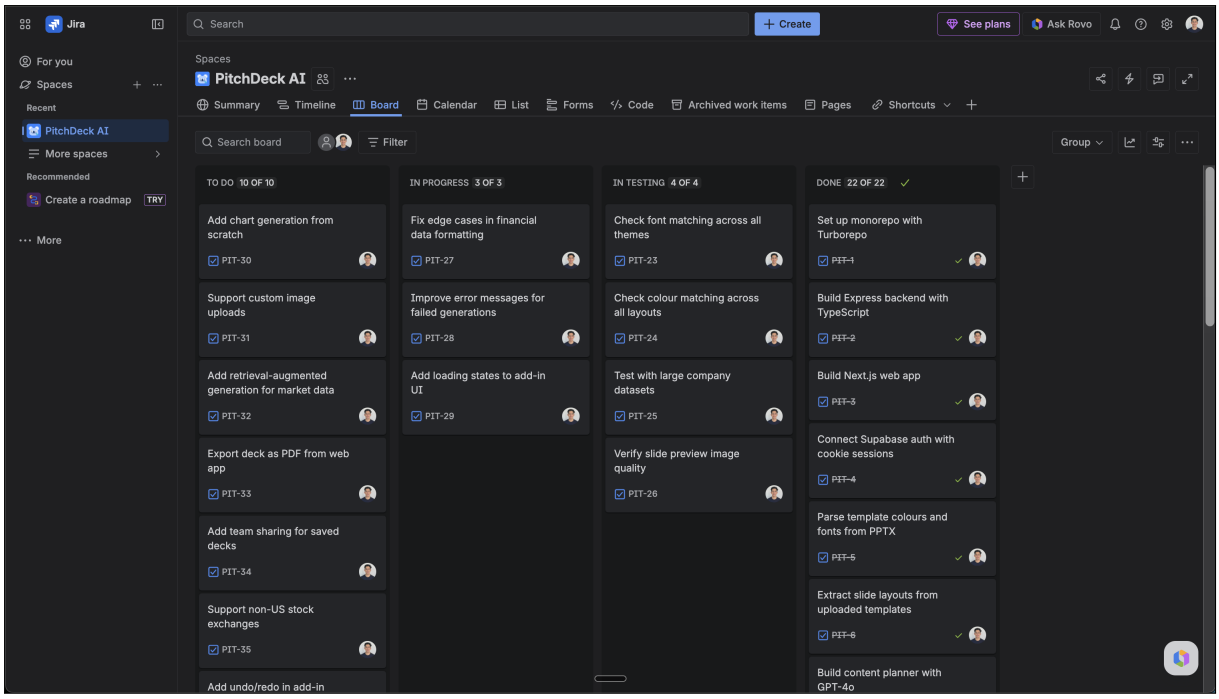

Figure 3: Kanban board showing task progress across four stages: To Do, In Progress, Testing, and Done.

3 Preliminary Research and Design Documentation

3.1 Literature Review

The project scope covers pitch book generation: taking a template, pulling financial data, planning content with an LLM, and assembling slides. The literature review focuses on LLM productivity evidence, automated presentation generation, agentic architectures, and structured output reliability.

3.1.1 LLM Productivity

Large-scale studies back up LLM productivity gains with real numbers. Brynjolfsson et al. (2023) studied 5,179 customer support agents and found a 14% average productivity increase, with novice workers gaining up to 34%. Noy and W. Zhang (2023) found professionals completed writing tasks 40% faster with 18% higher quality when using ChatGPT. Both studies show uneven benefits: less experienced workers gain most because AI gives them access to patterns that experts have already learned.

Dell'Acqua et al. (2023) ran a randomised experiment with 758 Boston Consulting Group (BCG) consultants and found that those using GPT-4 completed 12.2% more tasks, 25.1% faster, with 40% higher quality. Their main finding was the 'jagged technological frontier': tasks inside the AI's capability boundary saw large gains, while tasks outside it made consultants perform worse. PB formatting and data retrieval sit inside the frontier. Analytical judgement sits outside it, which is why the system produces drafts rather than finished documents. Peng et al. (2023) found that developers with GitHub Copilot completed tasks 55.8% faster, with less experienced developers benefiting most. The parallel is direct: junior analysts are the less experienced workers who stand to gain the most.

That said, none of these studies addressed template-constrained document generation where outputs must match precise visual specifications. General writing assistance helps with content but does nothing for the mechanical formatting work that consumes analyst time.

3.1.2 Automated Presentation Generation

IB pitch books need template fidelity, financial data integration, domain-specific narrative structure, and exact colour and font matching. Missing any one of these means the output needs heavy manual rework, which defeats the purpose of automation. Table 15 breaks down the main approaches and the studies behind each.

Approach	Studies	Mechanism	Strengths	IB Limitations
Rule-based layout	PPSGen (Hu and Wan, 2013)	Algorithmic text-to-slide conversion	Fast, deterministic	Poor visual quality, no domain awareness
Rigid templates	Commercial tools (Gamma, Tome)	Fixed layouts with manual content	High visual fidelity	Cannot adapt to varying content
LLM-based	PPTAgent (Zheng et al., 2025)	Reference analysis + edit generation	Preserves source style, good content	Domain-generic, no financial data
Multi-agent	Auto-Slides (Yang et al., 2025)	Template-constrained multi-agent pipeline	Maintains document structure	Academic focus, not PowerPoint
VLM-based	ColPali (Faysse et al., 2024)	Visual understanding of layouts	Handles complex visual elements	Approximate values, not exact matching

Table 15: Presentation Generation Approaches Compared

The two most critical limitations are the lack of financial data integration and the inability to match exact template specifications. Rule-based and rigid template approaches cannot adapt to varying content lengths, so a company overview with three years of revenue data and one with five years would need different manual adjustments. LLM-based approaches like PPTAgent produce good general content but know nothing about IB narrative conventions (executive summary, then business overview, then financials, then valuation) and cannot connect to data sources like Yahoo Finance or SEC EDGAR. VLM-based approaches get close to visual understanding but return approximate colour values rather than exact hex codes, which fails corporate brand compliance checks.

Hu and Wan (2013) introduced PPSGen, one of the first systems to learn slide generation from paper-slide pairs, but it produced rigid, text-heavy output. Yang et al. (2025) proposed Auto-Slides, a multi-agent system closer to this project's pipeline approach, but targeting academic content.

PPTAgent (Zheng et al., 2025) comes closest. Its two-stage approach (analyse reference, generate matching slides) preserves the source deck's visual identity. PPTAgent outperforms earlier systems on both content and design quality. The limitation is that it targets general-purpose presentations with no financial data integration or IB narrative conventions. Liu et al. (2024) surveyed 51 professionals and found template consistency was their top priority, which backs the template-first direction. None of these tools add domain awareness.

Vision-language models offer an alternative path. Faysse et al. (2024) show VLMs can understand document layouts visually. But code-based extraction via xml2js provides exact colour hex values and font sizes, while VLM-based approaches yield approximations. For corporate template matching where a colour being off by a shade flags the document as non-compliant, exact values are non-negotiable. This is why the project uses code-based extraction over visual understanding, building on PPTAgent's template-first approach while adding the IB domain layer that PPTAgent lacks.

3.1.3 Agentic Architectures and Tool Use

L. Wang et al. (2024) propose a framework where agents operate through planning, memory, and action loops. The framework maps to PB generation: the system plans slide structure, remembers template constraints, and acts through tool calls and slide assembly.

Guo et al. (2024) show multi-agent systems add value when sub-tasks require negotiation or parallel reasoning. PB generation is sequential (analyse template, plan content, assemble slides), so a single agent with tool-use capabilities is sufficient. A multi-agent approach would add coordination overhead without a matching benefit. Qu et al. (2025) identify function calling as the primary LLM-tool interface, and Li et al. (2025) confirm it as the most broadly applicable pattern across RAG, code interpretation, and document generation tasks. In this project, the orchestrator calls specialised service modules (template analyser, data retrieval, slide builder) in sequence, and the LLM's role is limited to content planning with structured JSON output. Limiting the LLM to content planning keeps it inside its capability boundary, consistent with Dell'Acqua et al. (2023)'s jagged frontier finding.

3.1.4 Structured Output Reliability

Getting reliable structured output from LLMs is a harder problem than it first appears. Shorten et al. (2024) benchmarked LLMs' ability to produce valid JSON and found an average success rate of 82.55%, varying from 0% to 100% by task. The failure modes matter: malformed brackets, missing required fields, and extra wrapper objects all break downstream processing differently. Geng et al. (2025) showed that constrained decoding (where the model is forced to generate only tokens that form valid JSON) helps, but success rates still drop as schema complexity increases. The content plan schema in this project is moderately complex: nested slide specifications where each slide contains a variable number of content blocks (headings, paragraphs, bullet lists, tables, charts), each with its own required fields. These findings shaped a two-layer validation strategy: Zod schema validation catches structural errors, and an unwrapping step handles the common case where GPT-4o nests valid JSON inside an unexpected wrapper object. If both layers fail, a hardcoded fallback plan keeps generation going with generic content that needs more editing.

3.1.5 Hallucination and Human Oversight

Huang et al. (2024) distinguish two types of hallucination. Intrinsic hallucination contradicts information given in the prompt. Extrinsic hallucination generates claims with no grounding in any source. For IB, extrinsic hallucination is more dangerous: made-up numbers or invented deal precedents could reach clients and regulators, causing regulatory problems or legal liability.

Mitigation approaches fall into three broad categories: grounding outputs in real sources, forcing structured output formats, and human review (Y. Zhang et al., 2023; Huang et al., 2024). No single strategy eliminates hallucination. The system layers all three: financial data comes from APIs rather than LLM generation, structured JSON schemas enforce the format of each slide specification, and human review is mandatory before any output reaches a client.

Wu et al. (2022) classify HITL approaches into data processing, interventional training, and system design. The approach here is system design: the analyst defines inputs, reviews outputs, and decides whether to use them. The EU AI Act requires human oversight for AI in financial services (Bank for International Settlements, 2024). HITL is a compliance requirement. Junior analysts already produce drafts that senior bankers revise. The system replaces the blank page. The review process stays the same.

3.1.6 Research Gaps

Four gaps emerge from this review. First, no presentation generation tool targets IB-specific conventions. PPTAgent (Zheng et al., 2025), Auto-Slides (Yang et al., 2025), and PPSGen (Hu and Wan, 2013) all target general or academic presentations. Second, no system automates both structure and content within a single pipeline (Achachlouei et al., 2023). Third, no system integrates data retrieval with template-aware assembly. Fourth, the productivity literature measures general tasks but none measured template-constrained document generation where outputs must match precise visual specifications.

The system targets these gaps with code-based template extraction for exact visual matching and IB-specific content planning. Human review is mandatory before output reaches a client. The evaluation measures template matching and generation speed, and includes an informal usability evaluation with junior analysts.

3.2 System Design

The design puts template matching, clean separation of components, and human oversight first.

3.2.1 Architecture Overview

Figure 4 shows the high-level system architecture. The design follows a pipeline pattern with three main stages: template analysis, content planning, and slide assembly.

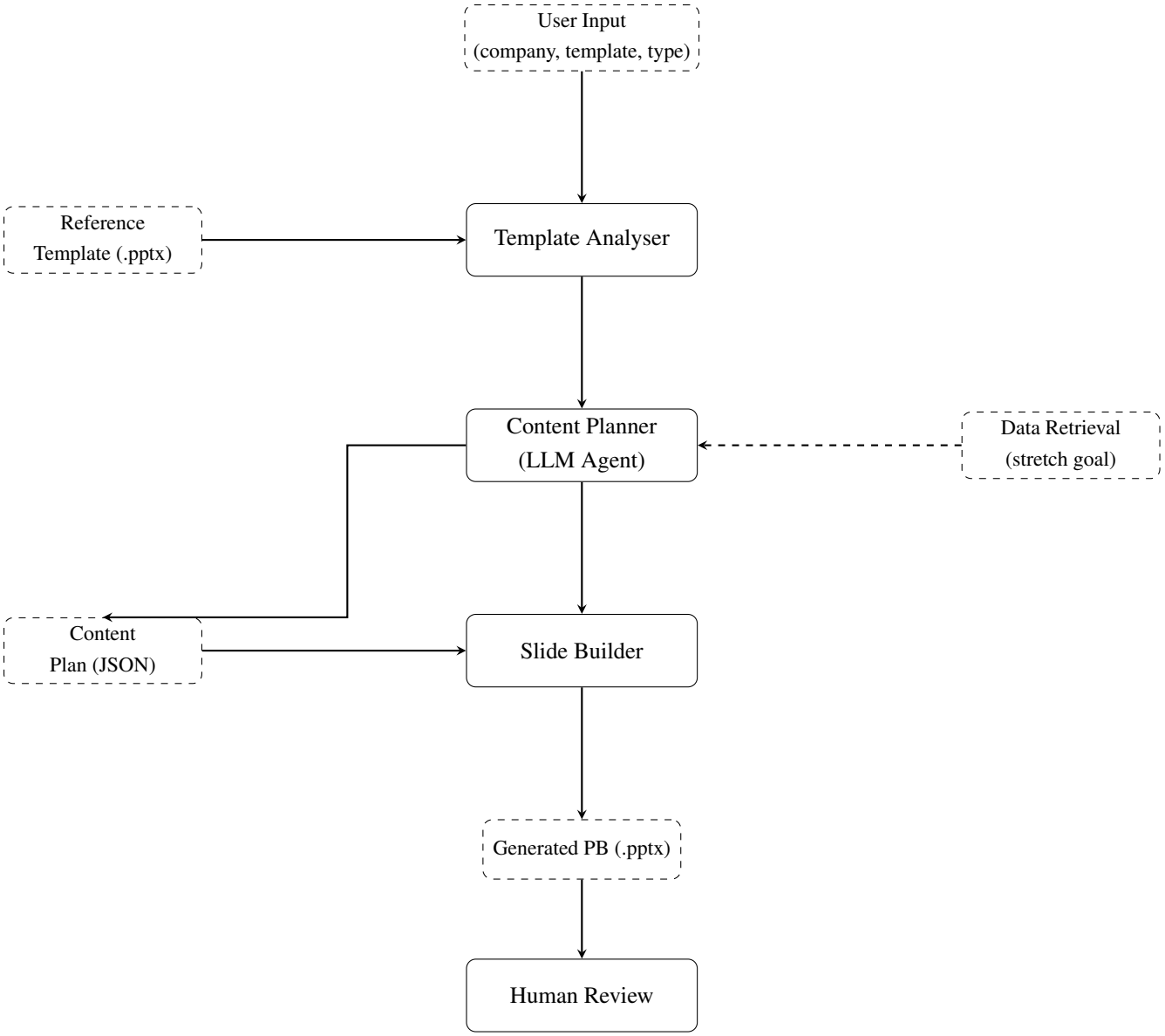

Figure 4: System Architecture showing the three-stage core pipeline. Data Retrieval is a stretch goal.

The architecture addresses the four limitations identified in the literature review. The Template Analyser solves the exact-matching problem by extracting colour hex codes, font specifications, and placeholder positions directly from the .pptx XML rather than using visual approximation. The Data Retrieval service solves the data integration gap by pulling real financials from Yahoo Finance and SEC EDGAR and passing them into the LLM prompt. The Content Planner adds IB domain awareness by following pitch book narrative conventions (executive summary, business overview, financials, valuation) through a structured system prompt. The Slide Builder closes the structure-content gap by generating slides that respect both the template constraints and the content plan in a single pass.

These components are separated into independent modules (SE2, SE9). Each can be tested in isolation: the template analyser against real .pptx files without an LLM call, the slide builder against fixed content plans without GPT-4o.

Data Retrieval from Yahoo Finance and SEC EDGAR was originally a stretch goal. It was implemented

because usability testing showed that analysts wanted real financial data in the first draft rather than placeholders they would have to fill manually. The pipeline architecture made this simple to add: the data retrieval module plugs in between validation and content planning without changing any other component.

3.2.2 Data Models

The system uses two primary data structures that flow between components. Table 16 lists the fields.

Model	Key Fields	Purpose
TemplateSchema	layouts[], colours, fonts, placeholders[]	Stores extracted template properties for content planning and slide assembly
SlideLayout	layout_index, placeholder_map, background	Maps placeholder positions to content types for a single layout
ContentPlan	slides[], metadata	LLM-generated specification of slide content and structure
SlideSpec	layout_id, placeholders, charts[], tables[]	Content specification for a single slide

Table 16: Primary Data Model Specifications

Template Schema captures extracted template properties including layouts, colour palettes, fonts, and placeholder specifications. Each slide layout maps placeholder indices to content types (title, body, picture, chart, table).

Content Plan specifies the content for each slide: which layout to use, what text goes in each placeholder, and what data to display. The LLM generates this structure. The Slide Builder consumes it.

Figure 5 shows the database schema. Row-level security (RLS) policies on every table restrict access so that authenticated users can only read and write their own records.

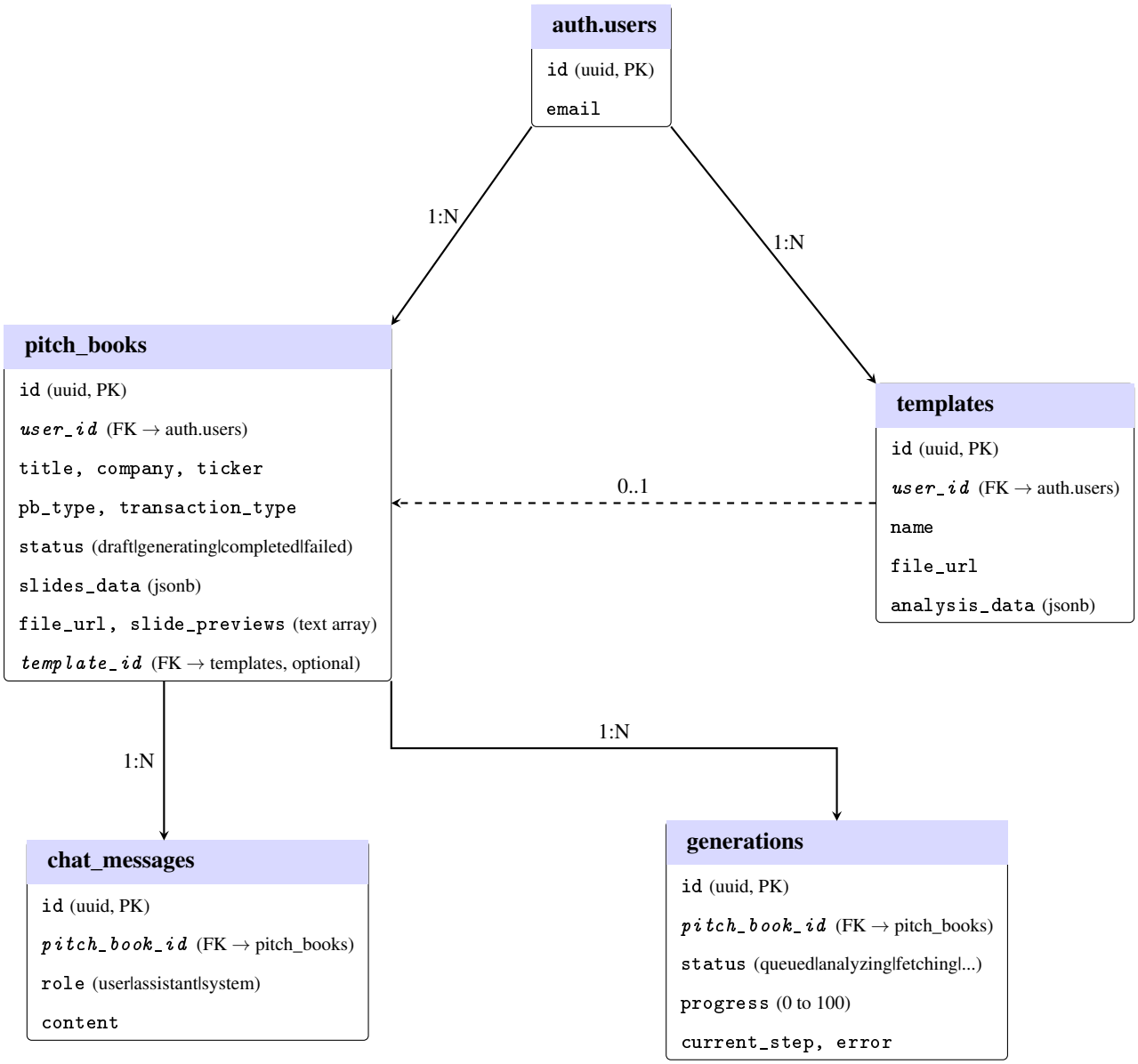

Figure 5: Database entity-relationship diagram. Primary keys in bold, foreign keys in italics. All tables use UUID primary keys with row-level security. The dashed line indicates an optional foreign key (pitch books may or may not reference a template).

3.2.3 Component Specifications

Template Analyser. Input: reference .pptx file. Output: TemplateSchema JSON. Uses JSZip and xml2js to parse the .pptx file at the XML level, extracting slide masters, layouts, colour schemes, and placeholder positions. Identifies placeholder types (title, body, picture, chart, table) by inspecting shape properties. Extracts exact colour values as hex codes and font specifications including family, size, and weight.

Content Planner. Input: user parameters, TemplateSchema. Output: ContentPlan JSON. Implemented as an LLM call with JSON schema validation to check the output structure. Selects appropriate layouts from the template schema based on content requirements. Plans slide sequence following IB narrative conventions (context, analysis, recommendation).

Slide Builder. Input: ContentPlan, TemplateSchema, reference template. Output: .pptx file. Uses PptxGenJS to generate slides with code. Iterates through ContentPlan slides, selects layouts, and populates placeholders. Applies exact formatting from TemplateSchema. Generates charts and tables with code.

3.2.4 API Design

The backend exposes a REST API via Express.js. The primary endpoint accepts generation requests and returns job identifiers for async processing.

Code 1: Pitch book creation request interface (from shared-types)

```typescript
// POST /api/pitchbooks - returns pitch book record with generation queued
interface CreatePitchBookRequest {
  title: string;
  company: string;
  ticker?: string;
  transaction_type: 'ma' | 'capital_raising' | 'restructuring'
                  | 'ipo' | 'debt_financing';
  pb_type: 'company_overview' | 'market_update'
         | 'transaction_summary';
  template_id?: string;        // reference template UUID
  additional_context?: string; // free-text instructions
  color_theme?: string;         // e.g. 'navy_gold', 'teal_coral'
  design_style?: string;
}
```

The full API is documented with an OpenAPI 3.0 specification and served through Swagger UI (Figure 6). Authenticated endpoints (marked with a lock icon) require a valid Supabase session cookie.

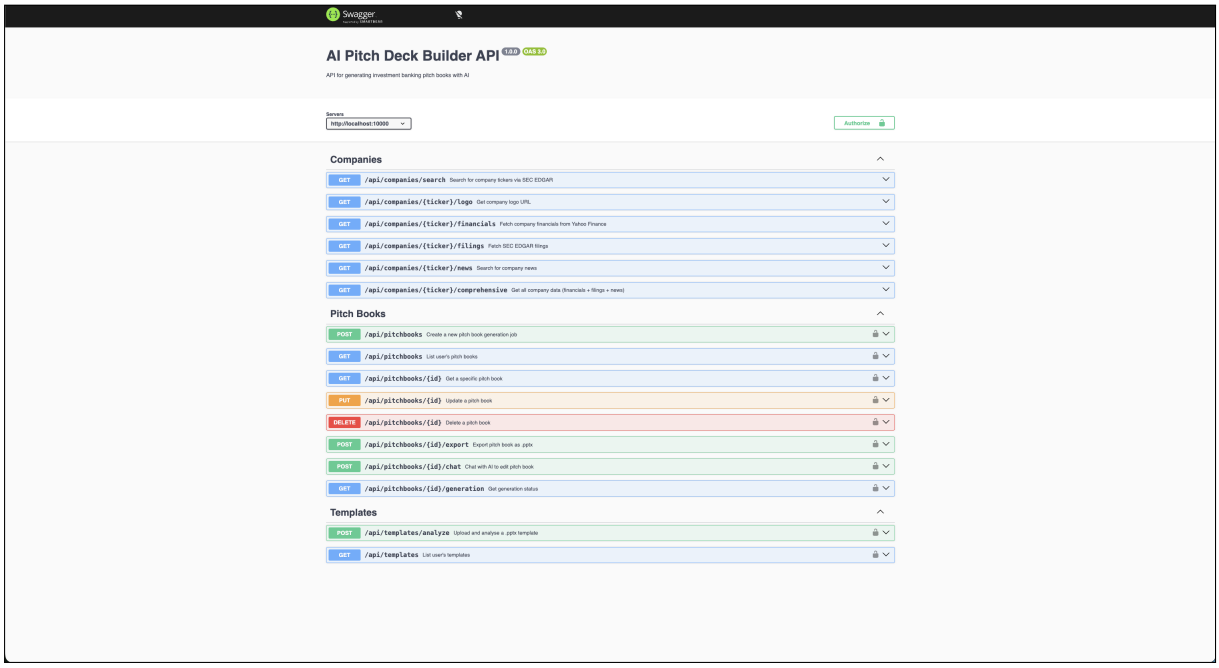

Figure 6: Swagger UI showing the REST API endpoints grouped by resource: Companies (data retrieval), Pitch Books (generation and CRUD), and Templates (upload and analysis).

3.2.5 User Interface Design

Three wireframes document the planned screens for the v1 web application, produced before implementation to establish layout conventions and the main user flows. The viewer screen (Figure 9) later exposed the central usability problem: without an in-browser editor, the web app had no way to support the iterative editing that pitch books require. This drove the add-in pivot described in Section 4.6.

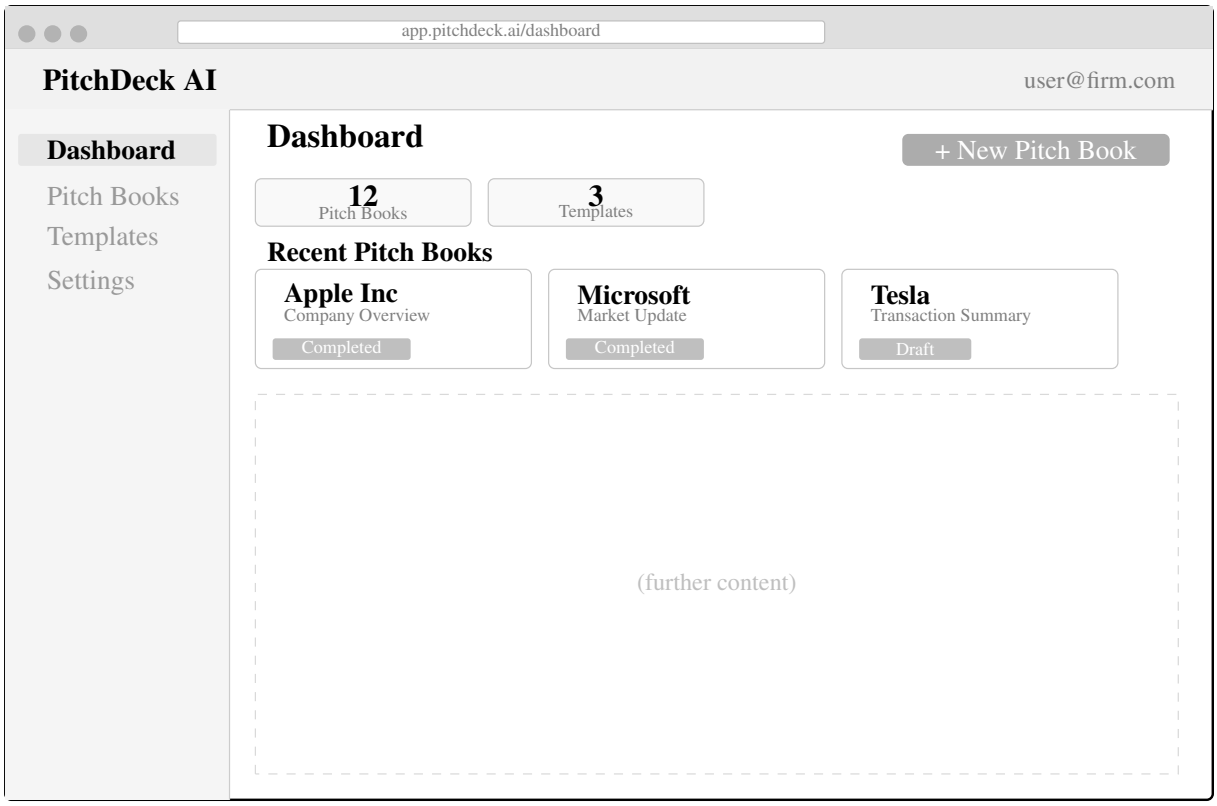

Figure 7: Wireframe: Dashboard. Sidebar navigation with four sections; main area shows aggregate stats and recent pitch books as cards.

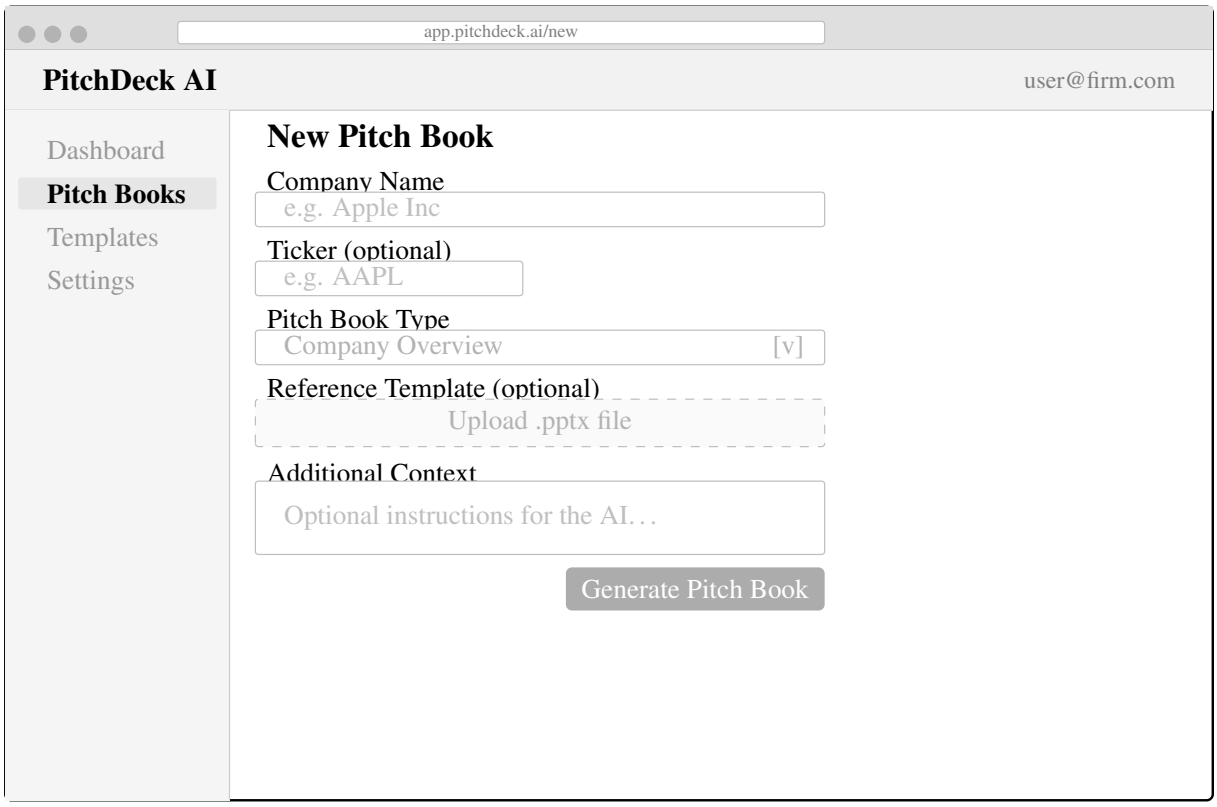

Figure 8: Wireframe: New Pitch Book form. Company name, ticker, PB type, optional template upload, and a free-text context field. The Generate button triggers the backend pipeline.

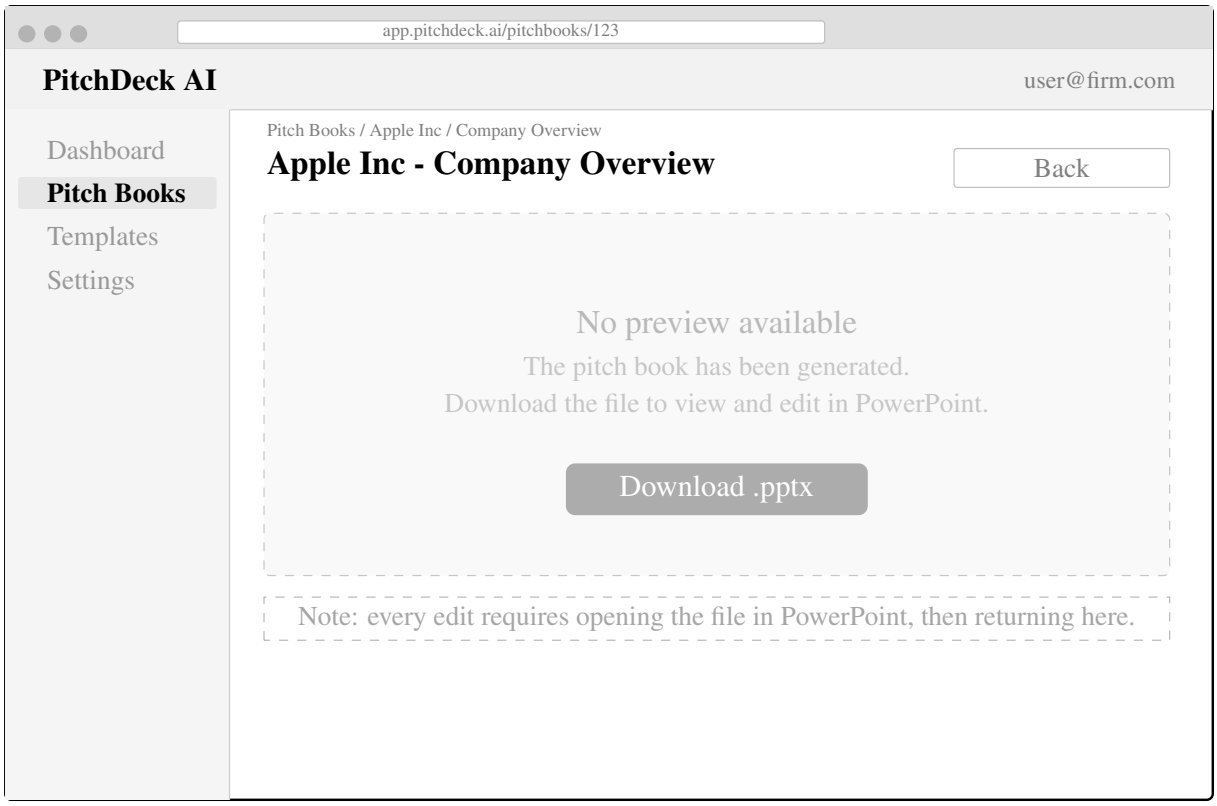

Figure 9: Wireframe: Pitch Book Viewer (v1, web-only). No in-browser preview or editor, just a download link. This screen made the editing friction visible: every change required a round trip through PowerPoint, which led to the add-in design.

3.2.6 Design Trade-offs

Table 17 summarises the four main design trade-offs.

Decision	Chosen Option	Rationale
Single vs multi-agent	Single agent with tools	PB generation is sequential (analyse, plan, build). Multi-agent coordination adds overhead without a matching benefit for a linear pipeline.
Programmatic vs VLM template extraction	Programmatic extraction (JSZip + xml2js)	Corporate brand compliance requires exact hex colour values and font sizes. VLM-based approaches return approximations that fail brand checks.
SDK vs custom orchestration	Structured LLM calls with JSON schemas	Avoids vendor lock-in. Each service module has a defined input/output contract, so swapping the LLM provider requires changing one file.
Sync vs async generation	Async job processing	Generation takes 30 to 80 seconds. Synchronous HTTP requests would time out. The client polls for completion and displays progress updates.

Table 17: Design Trade-offs

3.2.7 Data Flow

Figure 10 shows how data moves through the system during a generation request. There is no direct data flow from template extraction to content planning. All data passes through the Template Schema, which is the sole output of the extraction stage and the sole template input to the planning stage. The schema acts as a contract between the two components: the extractor writes it, and the planner reads it. This means the content planner never touches the raw .pptx file.

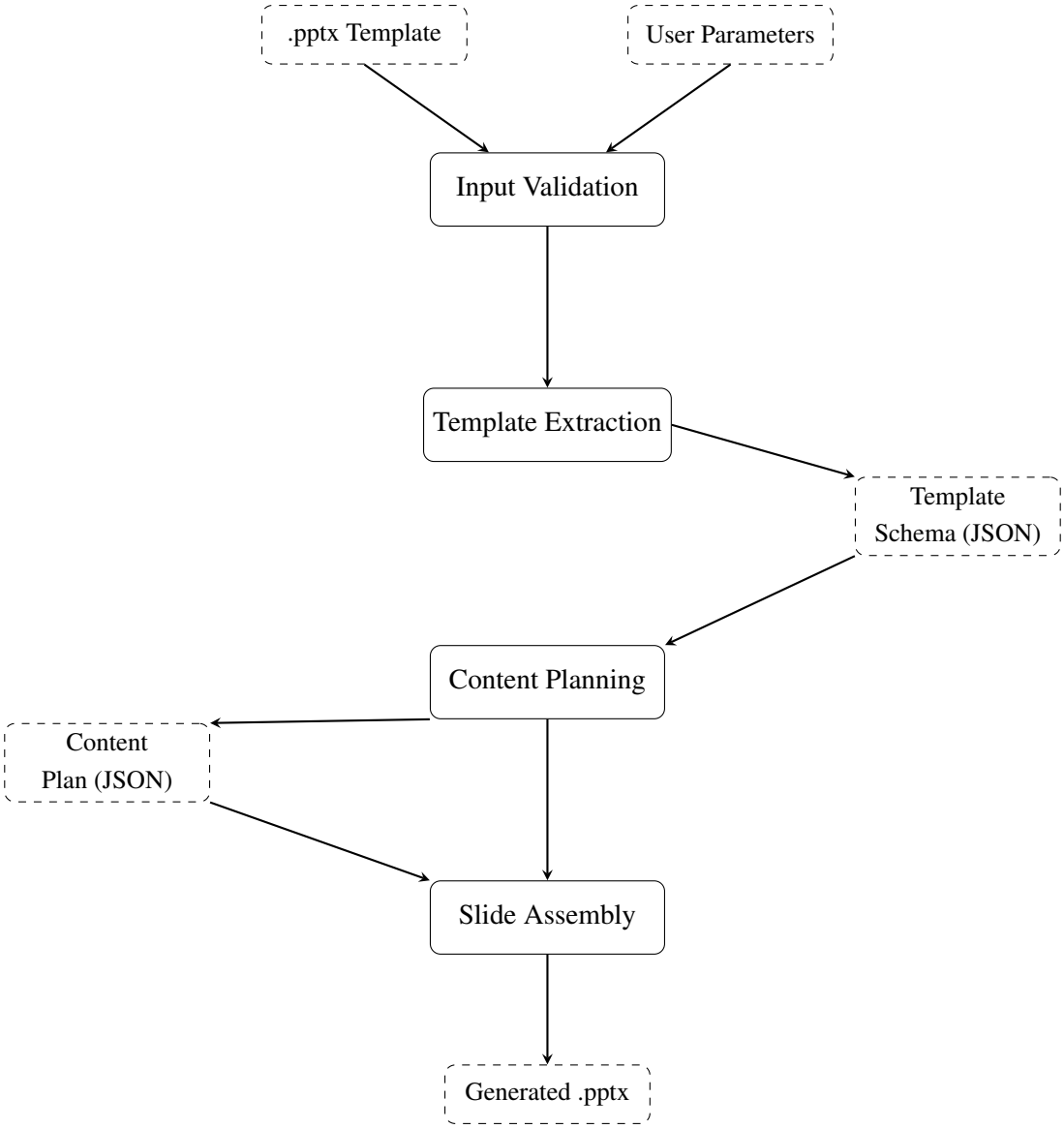

Figure 10: Data flow through the generation pipeline. All template data passes through the Template Schema; there is no direct link from extraction to planning.

3.2.8 Design Limitations

The system cannot generate charts from scratch. It can only populate existing chart placeholders. The template analyser handles standard placeholder-based layouts but does not reverse-engineer arbitrary PowerPoint constructions with overlapping elements or custom animations. The content planner has no memory across sessions. The system produces drafts, not finished documents: human review is mandatory (K6, SE1).

4 Project Implementation and Outcomes

4.1 Architecture Evolution

The original design called for a Python backend with FastAPI and a Next.js frontend. Two weeks in, this changed: every data structure crossing the API boundary had to be defined twice, and mismatches caused silent runtime failures. Switching everything to TypeScript (SE3) took four days but removed a whole category of bugs. The `shared-types` package now defines every interface once, and the compiler catches mismatches at build time.

4.2 Monorepo Structure

The codebase uses Turborepo to manage three applications and one shared package (SE12):

Package	Technology	Purpose
`apps/service`	Express.js, TypeScript	Backend API: orchestration, template analysis, content planning, slide building
`apps/web`	Next.js, React, TypeScript	Web frontend: PB management, slide preview, generation trigger
`apps/powerpoint-addin`	Vite, React, TypeScript, Office.js	PowerPoint taskpane: in-editor generation and AI-assisted editing
`packages/shared-types`	TypeScript	Shared interfaces, enums, and type definitions across all apps

Table 18: Monorepo Package Structure

Figure 11 shows the web app dashboard after login.

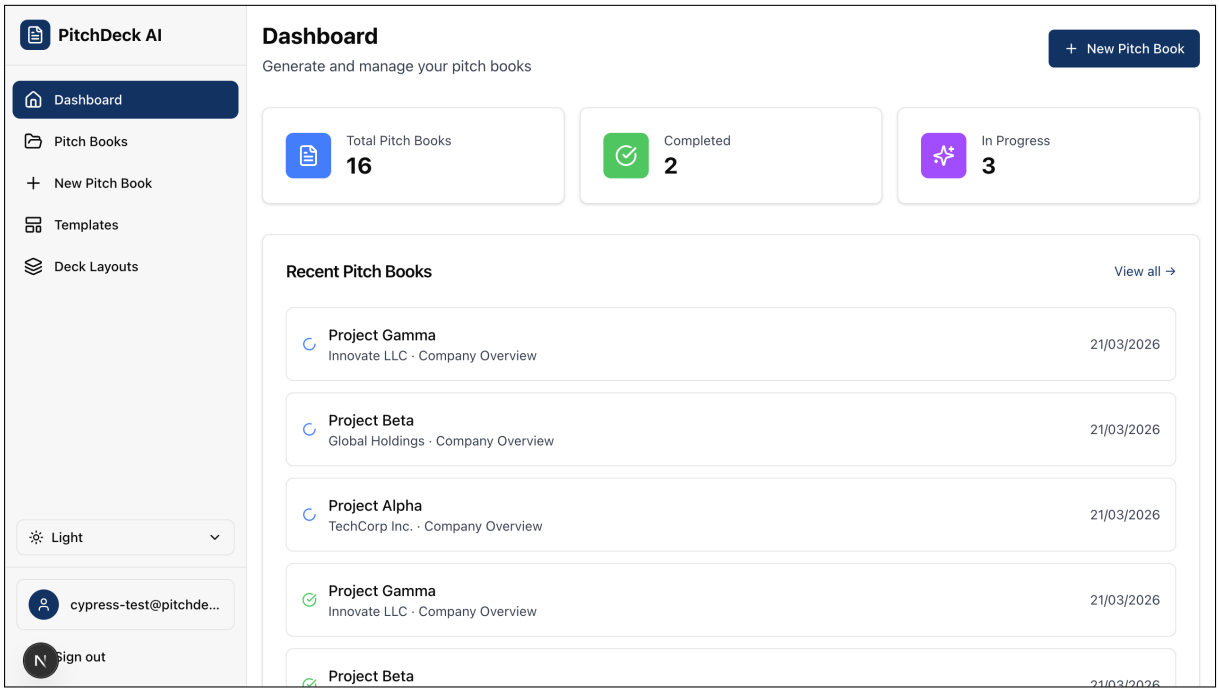

Figure 11: Web app dashboard showing pitch book counts, recent generation history, and sidebar navigation.

4.3 Orchestration Layer

The orchestrator runs each stage in sequence and stays thin: each service module handles its own domain. Figure 12 shows the full pipeline. If data retrieval fails, it passes an empty object to the content planner rather than aborting. The orchestrator passes the full template schema to GPT-4o rather than a summary, because early versions with only layout names produced worse output.

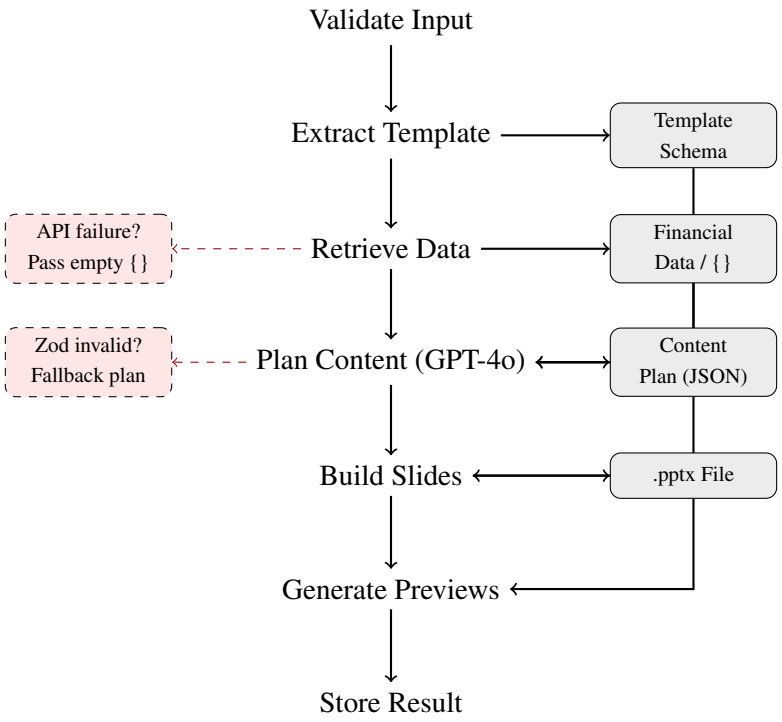

Figure 12: Orchestration pipeline. Each stage passes its output to the next. Dashed paths show graceful failure handling: data retrieval passes an empty object on API failure, and the content planner falls back to a generic plan if Zod validation fails.

4.4 Deck Layouts and Colour Themes

The system ships with seven deck layouts (Company Overview, Investor Pitch, Market Update, Transaction Summary, Industry Overview, Fundraising Deck, Due Diligence) and eight built-in colour themes. The layouts are hardcoded because IB slide sequences are standard across the industry. The visual style is customisable: users pick a theme or upload a bank's own template file. When a template is uploaded, the analyser extracts that bank's exact colours, fonts, and layout positions, and the slide builder applies them to whichever layout the user picks. Figure 13 shows the creation form.

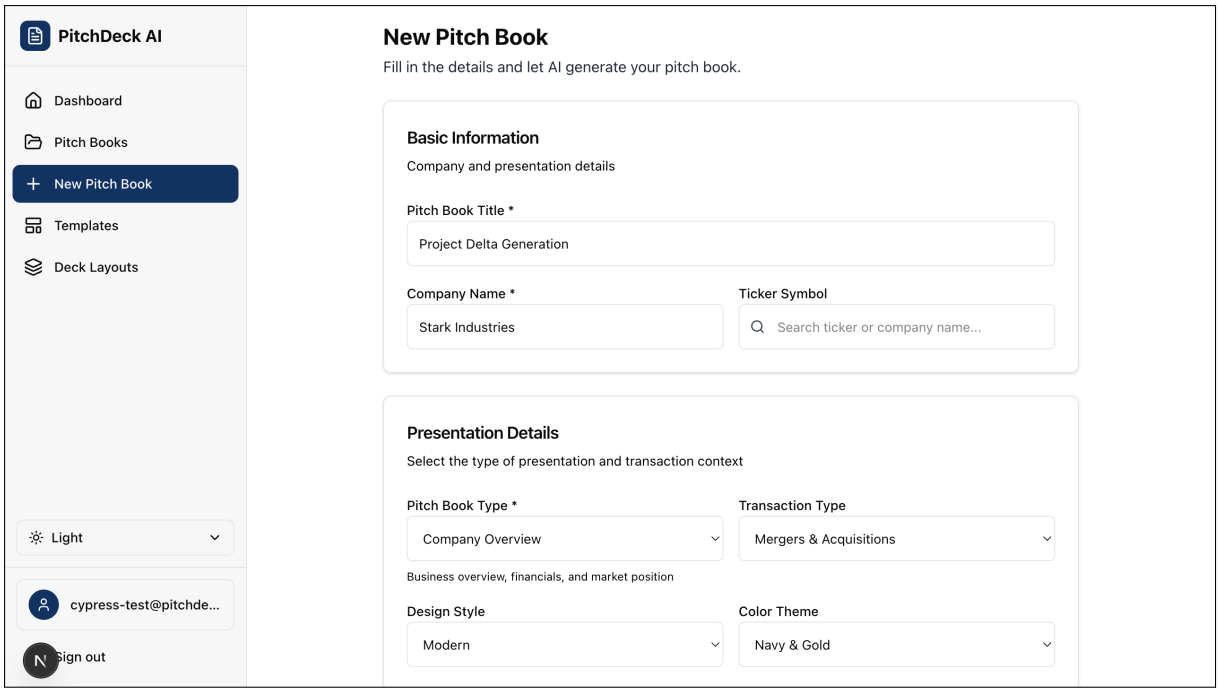

Figure 13: New pitch book creation form. The user selects a deck layout, colour theme, and enters company details before generating.

4.5 Core Pipeline

The generation pipeline follows the design from Section 3, implemented as a chain of service modules within the Express backend. Each stage is a separate TypeScript module with a defined input/output contract. Development followed a pattern: build the simplest version, test it against real templates, identify the failure mode, and fix it.

4.5.1 Template Analysis

The template analyser went through three iterations, each driven by a specific failure in the previous version (SE4).

Iteration 1: PptxGenJS only. The first attempt used PptxGenJS for both reading and writing slides. PptxGenJS can create slides from scratch but has no API for reading existing .pptx files. The only option was to generate new slides and hope the styling matched. It did not. Colours were wrong, placeholder positions were guessed rather than measured, and the output looked nothing like the input template. This approach was scrapped after two days.

Iteration 2: JSZip + xml2js (raw XML parsing). PowerPoint files are ZIP archives containing Office Open XML documents. A .pptx file contains dozens of XML files that describe every slide, shape, colour, and font in the presentation. JSZip opens the archive, and xml2js parses the XML inside. This gave direct access to every property: colours from `ppt/theme/theme1.xml`, layouts from `ppt/slideLayouts/`, and placeholder positions from shape definitions. The approach worked but came with a problem. PowerPoint's OOXML specification stores measurements in English Metric Units (EMU), where one inch equals 914,400 EMU. Colours can be stored as hex values, theme references (`accent1`), or system colour names. Placeholder types use numeric codes, but custom placeholders have no type attribute at all. Each

of these required manual parsing and conversion logic.

Iteration 3: heuristics and fallbacks for analysis. Testing against real corporate templates exposed gaps in the raw XML approach. Custom placeholders with no type attribute made up roughly 30% of all placeholders in the test set. The analyser added position-based heuristics: a placeholder in the top 15% of the slide with large font settings is classified as a title, even without a type code. A wide placeholder in the bottom half with small fonts is treated as body text. This brought classification accuracy to 93% across test templates. When parsing fails entirely (corrupted file, unusual XML structure), the analyser falls back to professional defaults: a navy and gold colour scheme with standard IB layout patterns.

Code 2: Template extraction core logic (simplified)

```
async function analyseTemplate(buffer: Buffer): Promise<TemplateSchema> {
  const zip = await JSZip.loadAsync(buffer);

  // Parse theme for colour palette
  const themeXml = await zip.file('ppt/theme/theme1.xml')?.async('string');
  const theme = await parseStringPromise(themeXml);
  const colours = extractColourScheme(theme);

  // Parse each slide layout
  const layouts: SlideLayout[] = [];
  for (const [path, file] of Object.entries(zip.files)) {
    if (path.match(/ppt\/slideLayouts\/slideLayout\d+\.xml/)) {
      const xml = await file.async('string');
      const parsed = await parseStringPromise(xml);
      layouts.push(extractLayout(parsed, colours));
    }
  }

  return { colours, fonts: extractFonts(theme), layouts };
}
```

Each version failed in a specific, observable way, and the fix was targeted rather than speculative (SE4). The unit tests grew alongside: the third iteration added edge-case tests for missing themes, custom placeholders, and corrupted archives (SE5).

The analyser reads layout information from the slide master. Figure 14 shows the master view for a sample template. Each thumbnail on the left is a slide layout. The master defines where placeholders go, what fonts to use, and what the background looks like. The analyser reads these from the `ppt/slideLayouts/` directory and records the placeholder types, positions, and styling.

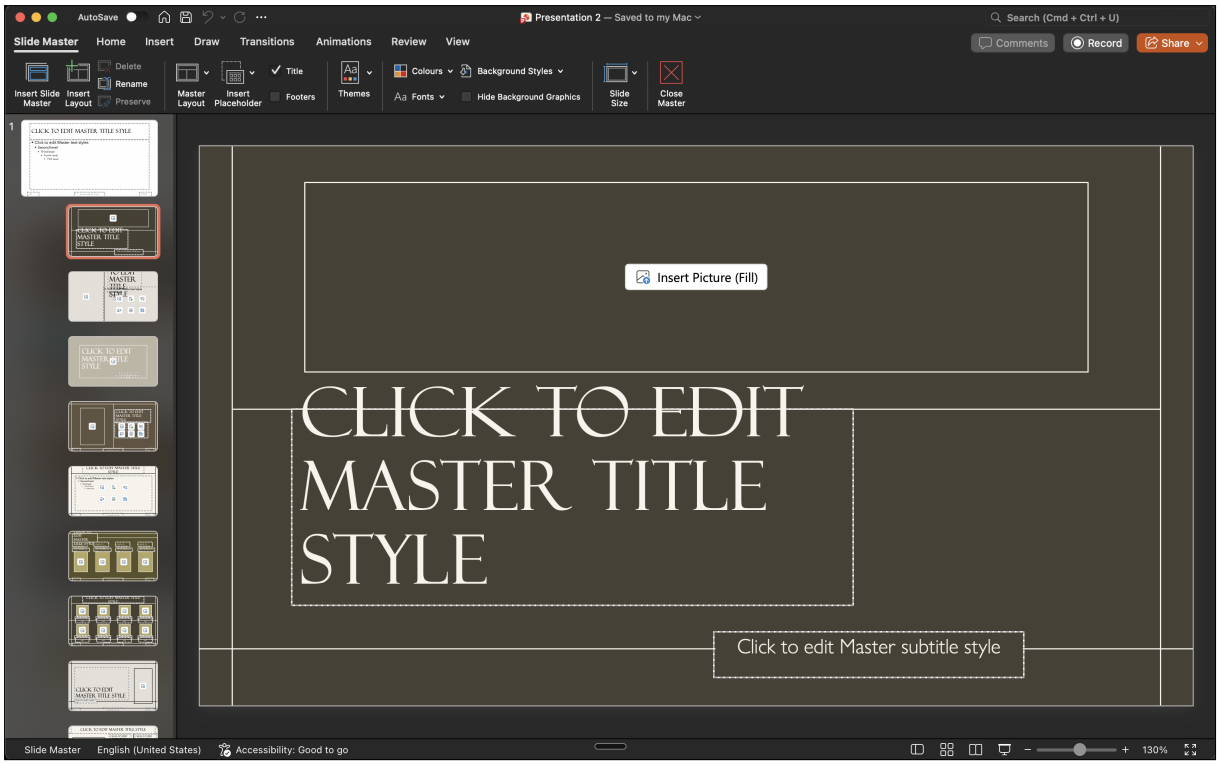

Figure 14: Slide master view showing the available layouts for a template. Each layout defines placeholder positions, fonts, and background styling that the generation pipeline inherits.

4.5.2 Content Planning with GPT-4o

The content planner sends user parameters and the extracted template schema to GPT-4o, requesting a structured content plan. Getting the prompt right took several iterations. The first version produced slides full of generic phrases like “strong market position” and “leading industry player” with no company-specific detail. Adding a rule banning generic phrasing helped, but the slides were still sparse, with two or three short bullet points per slide. The final system prompt (Code 3) sets minimum content density per slide type and provides examples of good versus bad output. It also instructs the model to write as a senior Managing Director rather than a generic assistant, which consistently produced more specific, data-driven language.

Code 3: Content planner system prompt excerpt (from pitchbook-content.prompt.ts)

```
export const SYSTEM_PROMPT = `You are a senior Managing Director at
Goldman Sachs creating investment banking pitch books.

CRITICAL RULES:
1. ALL financial data MUST use the EXACT numbers provided in the
   user message. NEVER fabricate numbers.
2. Every chart MUST have a "data" array with real numbers.
3. Content must be deeply specific to the company. Mention their
   actual products, markets, competitors, recent events.
4. Write like a real IB analyst: precise, data-driven, no fluff.
5. NEVER write generic phrases like "leading company in its sector"
   or "strong market position". Be SPECIFIC.

CONTENT DENSITY:
- Every Content Slide MUST have at least 5-7 bullet points.
- Every bullet point must be a FULL SENTENCE with specific data.
  BAD: "Strong revenue growth".
  GOOD: "Revenue grew 61% YoY to 130.5B in FY2024, driven by
  Data Center segment expansion to 115.2B."
- Financial Tables should have 4-5 years of data with 5+ rows.

You MUST return JSON in EXACTLY this format:
{ "title": "...", "narrative_arc": "...", "slides": [...] }`;
```

Getting valid structured output was the main challenge (SE1). Shorten et al. (2024) found an average JSON success rate of 82.55% across LLM benchmarks. GPT-4o occasionally wraps responses in unexpected envelope objects. The content planner handles this with an unwrapping step before Zod (Colinhacks, 2024) schema validation. If validation fails, a hardcoded fallback plan keeps generation going. Across 12 evaluation runs, all produced valid JSON without needing the fallback. The prompt includes retrieved financial data so the LLM places real numbers rather than fabricating them.

Code 4: Content plan validation with Zod (from pitchbook-content.schema.ts)

```
const contentBlockSchema = z.object({
  type: z.enum(['heading', 'paragraph', 'bullet_list',
                'table', 'chart', 'metric']),
  content: z.any(),
});

const slideSchema = z.object({
  index: z.number().optional().default(0),
  title: z.string(),
  layout: z.string().optional().default('Content Slide'),
  talking_points: z.array(z.string()).optional().default([]),
  data_requirements: z.array(z.string()).optional().default([]),
  content_blocks: z.array(contentBlockSchema).optional().default([]),
});

const contentPlanSchema = z.object({
  title: z.string(),
  narrative_arc: z.string().optional().default(''),
  slides: z.array(slideSchema),
});

function parseContentPlan(raw: unknown): ContentPlanOutput {
  return contentPlanSchema.parse(raw);
}
```

4.5.3 Slide Building

The system has two slide building paths, selected by whether the user uploaded a template (SE10).

Default mode uses PptxGenJS to generate slides from scratch with a built-in theme selected by pitch book type. Theme colour references (`accent1`, `dk1`) are resolved to hex values before applying, since PptxGenJS has no theme colour system.

Template mode clones the template's actual slides and replaces placeholder text using regex on the raw XML. Figure 15 shows a sample template with 14 slides. The builder parses each slide's `.rels` file to find its layout name, then for each content plan slide, picks the best-matching template slide. It finds each `<p:sp>` shape containing a `<p:ph>` (placeholder) element and replaces the `<p:txBody>` content while preserving the original `<a:bodyPr>`, `<a:lstStyle>`, and `<a:rPr>` (font, size, colour) via regex extraction. The result: generated text comes out in the template's own font, size, and colour.

This approach uses regex on the raw XML rather than xml2js round-tripping, which corrupted OOXML namespace declarations and produced files that Office.js rejected. The builder also injects `<a:normAutofit>` to auto-shrink overflowing text and truncates content per placeholder type (120 characters for titles, 7 bullets at 150 characters each, 500 for body text).

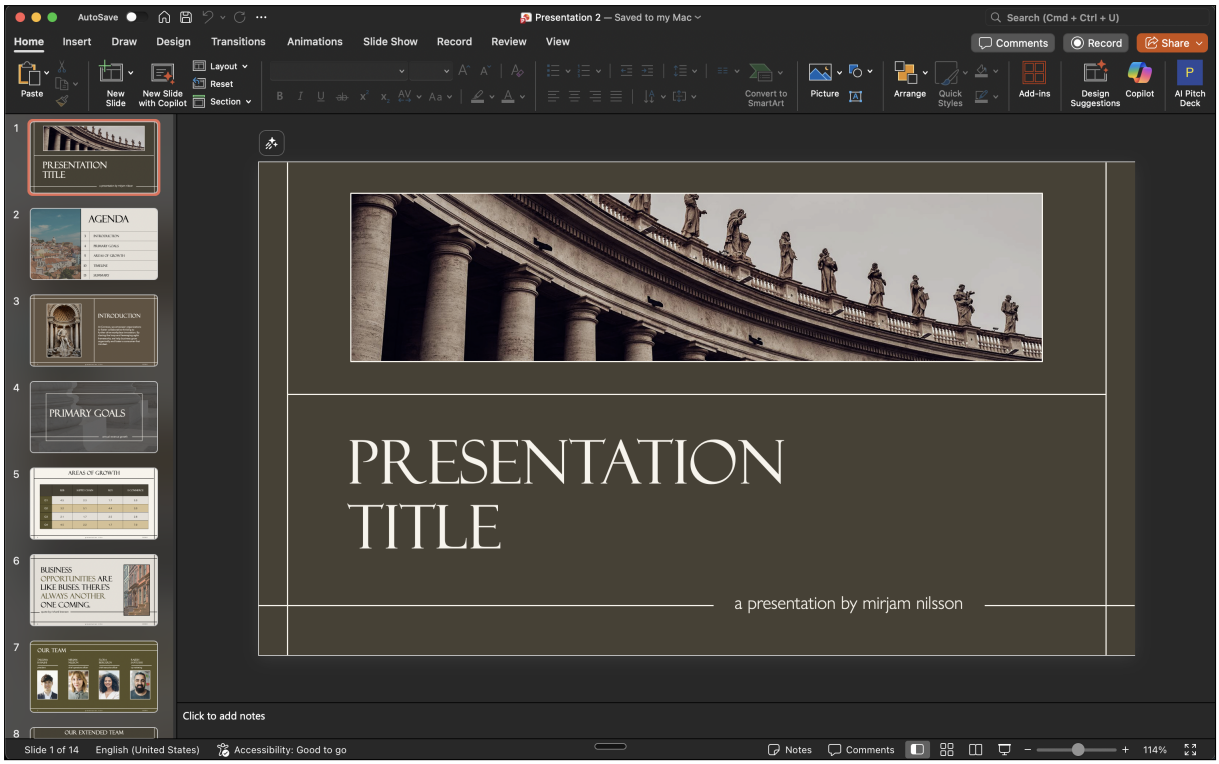

Figure 15: Original template with 14 slides across different layout types, each with a consistent brown and gold visual style.

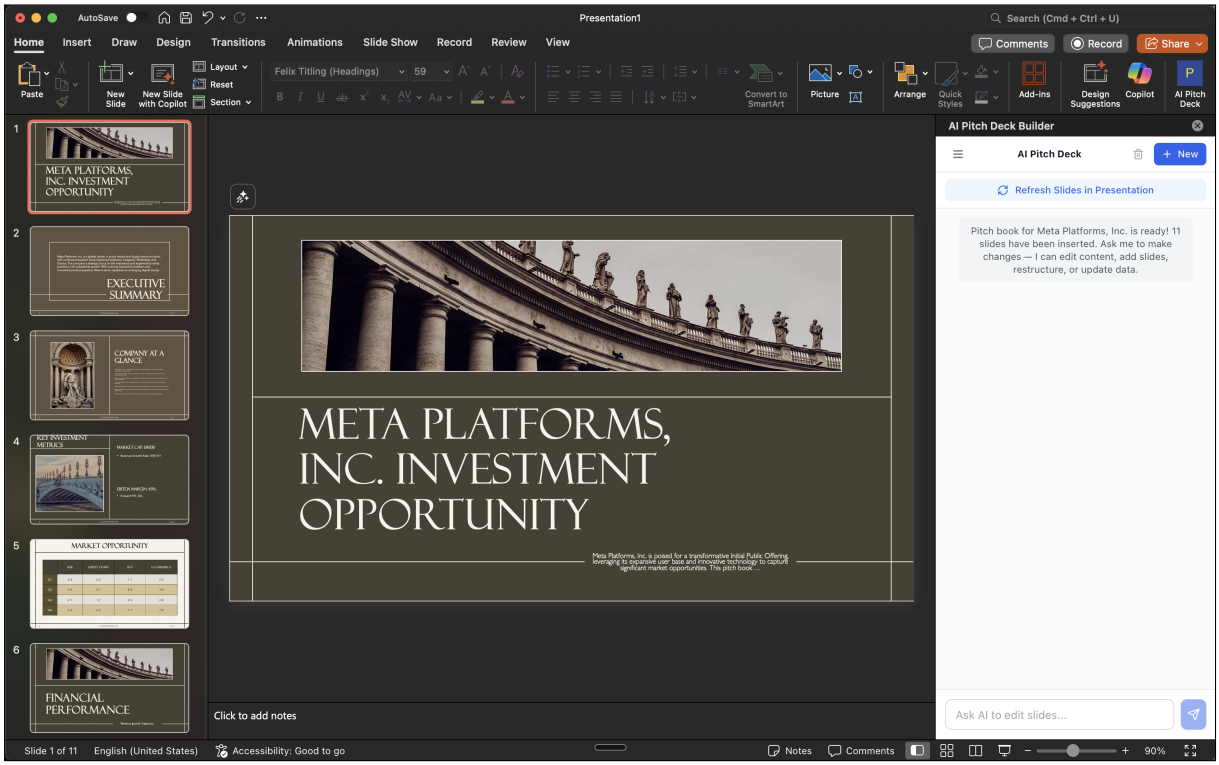

Figure 16: Generated output for Meta Platforms using the template from Figure 15. The 11 slides inherit the template's Felix Titling font, brown background, and gold accents. The add-in taskpane (right) shows slides were inserted directly into PowerPoint.

4.5.4 Data Retrieval

The data retrieval service pulls financials from Yahoo Finance (`yahoo-finance2`) and SEC EDGAR (K5). Yahoo Finance provides market cap, trailing revenue, share price, historical income statements, balance sheet data, and cash flow summaries. SEC EDGAR provides recent 10-K (annual), 10-Q (quarterly), and 8-K (material event) filing summaries for US-listed companies. Yahoo Finance data is injected directly into the GPT-4o prompt so the model places real numbers rather than generating its own. The EDGAR client runs and returns filing metadata, but the content planner does not yet reference it; passing filing data to the prompt is the next step. This is the architectural decision behind the 100% data accuracy result in Section 5. The LLM never estimates financial figures because it never needs to. Responses are cached for 24 hours to avoid hitting rate limits and to speed up repeated generations for the same company. The cache uses an in-memory store rather than a database because the data changes daily and stale cache entries are acceptable for a first draft.

If an API call fails, the service returns graceful null values rather than aborting. The content planner receives an empty data object and generates slides without financial figures, producing a structural draft that the analyst fills in manually. This design means a Yahoo Finance outage degrades output quality but never blocks generation entirely.

4.5.5 Slide Preview Generation

The system uses LibreOffice in headless mode to convert .pptx to PDF, then `pdftoppm` to render each page as a PNG at 150 DPI. Previews are uploaded to Supabase Storage with signed URLs and displayed in the web frontend's slide viewer. The pipeline runs asynchronously after slide assembly completes, so the .pptx file is available immediately while previews generate in the background.

Neither this pipeline nor the add-in below were in the original design. Both emerged from usability problems found during implementation. The web app originally showed only a download link for the generated file. Without visual previews, users had to open the file in PowerPoint to see what was generated, which added friction and made the web interface feel disconnected from the output. Adding preview images turned the web app into a useful management interface even after the add-in became the primary generation tool.

4.6 From Web-Only to Add-in: The Product Pivot

The original plan was a web-only application where the user would fill in a form and receive a .pptx download. That design assumed the generated deck would be close enough to final that a download-and-open step was acceptable. It was not.

Pitch books are never finished after the first draft. Analysts reword bullet points, swap slide ordering, adjust font sizes, reposition charts, and tweak colours to match the managing director's preferences. Every deck goes through multiple rounds of revision. A web-only tool that produces a download link ignores this reality.

The better path was to work inside PowerPoint rather than against it. PowerPoint already handles slide editing well, and it supports add-ins: small web applications that run in a taskpane and communicate with the active presentation via Office.js. The system removes the blank-page problem, not the editor.

This led to the v2 architecture. The add-in handles generation and insertion directly into the active presentation. The analyst stays in PowerPoint for the full workflow. The web app shifted to a management layer for storing pitch books, displaying previews, tracking history, and hosting the AI chat. Future features like RAG search (FR15, deferred) would also sit in the web app, since browsing a document library does not belong in a PowerPoint taskpane.

The pivot validated the iterative methodology. A waterfall approach would have delivered the planned web-only system and missed the usability problem entirely. The working prototype in week 7 made the friction visible and measurable, and the modular architecture meant the add-in could reuse the same backend and shared types without duplicating pipeline logic (SE7, SE8).

4.7 PowerPoint Add-in

The add-in is a taskpane built with Office.js that runs inside PowerPoint, communicating with the same Express backend as the web app.

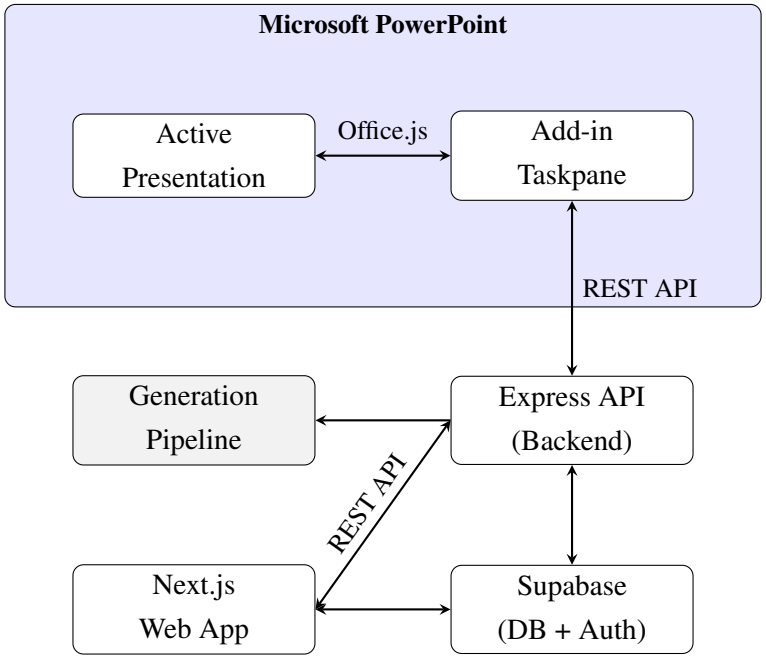

Figure 17: System architecture with PowerPoint Add-in. The add-in and web app share the same backend and database. A pitch book generated in either interface is stored centrally and accessible from both.

Because both the web app and the add-in connect to the same Express backend and Supabase database, a pitch book generated in either interface appears in both. An analyst can start a generation from the web app's creation form, and the resulting deck will be available in the add-in's pitch book list when they open PowerPoint. The two interfaces are two views of the same data, each suited to a different task.

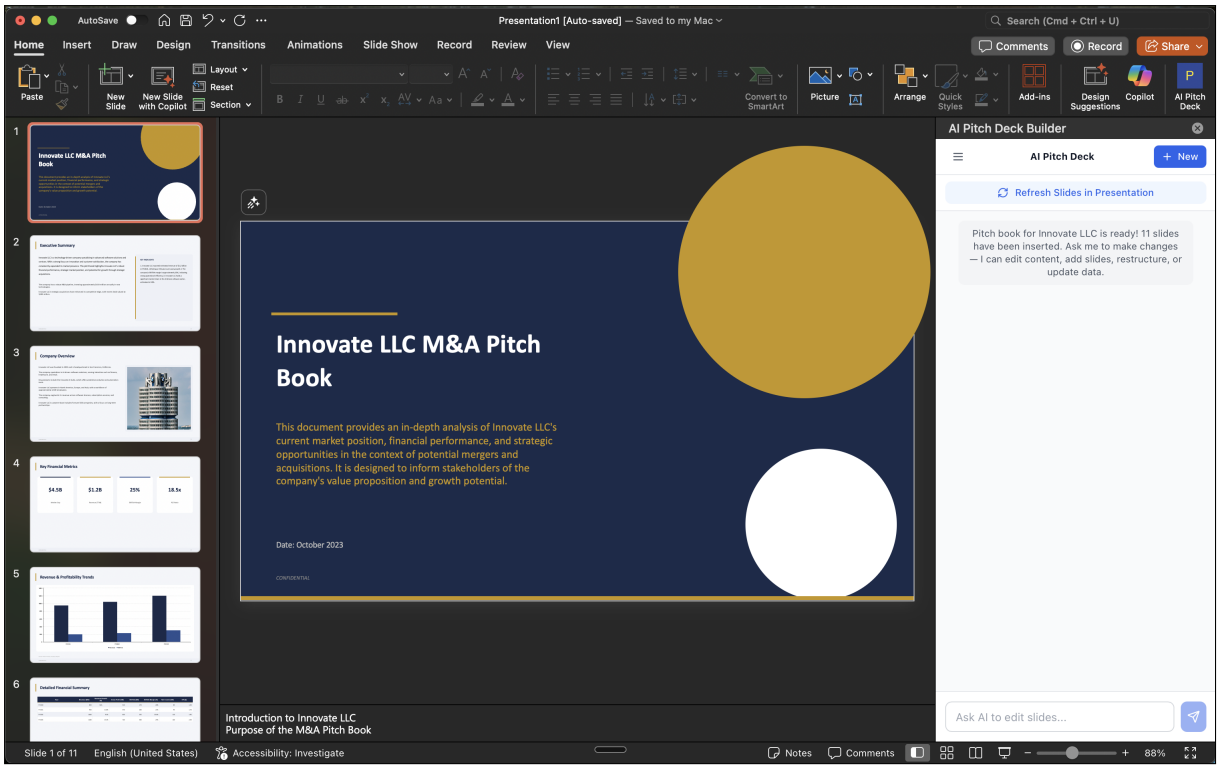

Figure 18: PowerPoint with the add-in taskpane open. Generated slides appear in the slide panel on the left, and the AI chat input sits at the bottom of the taskpane.

The `insertSlidesFromBase64()` API pushes the generated .pptx directly into the active presentation. The web app became a management interface for browsing pitch books. Both applications share the same backend (K5).

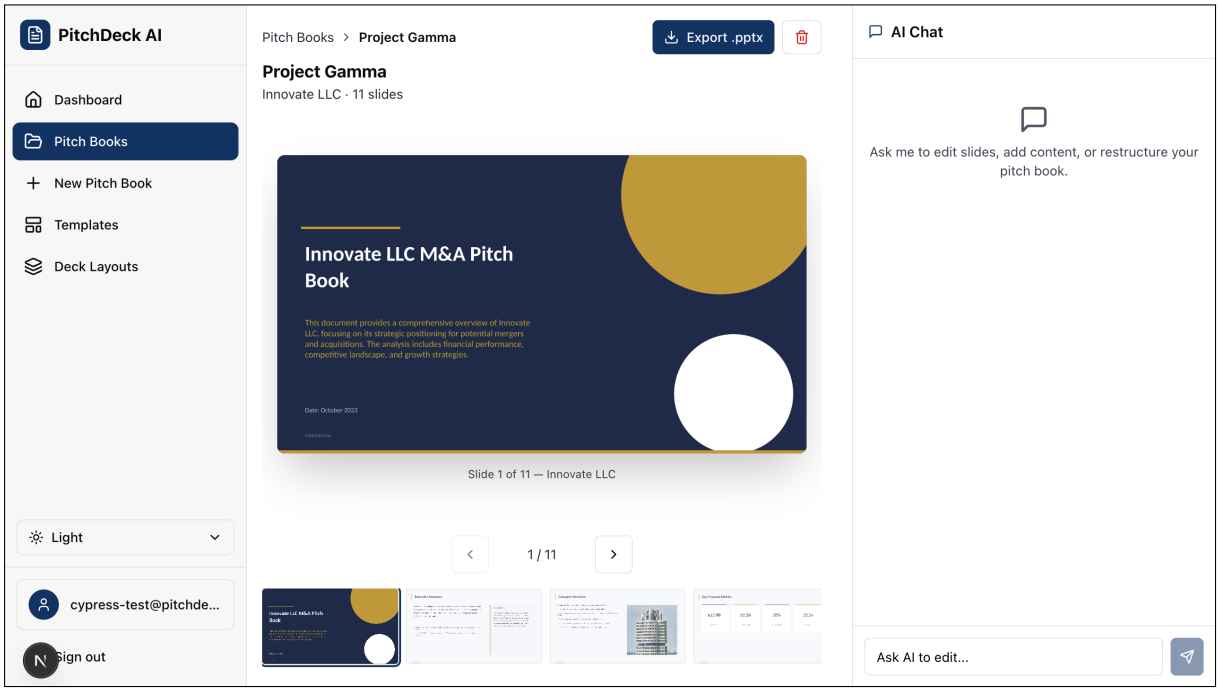

Figure 19: Pitch book viewer showing slide preview, thumbnail navigation, and AI chat panel.

4.8 AI Chat Assistant

The pitch book viewer includes an AI chat panel (visible in Figure 19) that lets users request slide changes in plain English. An analyst might type "make the executive summary more focused on the data centre segment" or "add a bullet about the recent acquisition". The chat runs on GPT-4o-mini, which is cheaper and faster than GPT-4o. Structured output is not required here since the model returns natural language advice rather than JSON. The system prompt includes each slide's title and layout type so the model can reference specific slides in its response. Messages are stored in a `chat_messages` table, preserving history across sessions.

The chat is advisory: the assistant provides instructions and suggested text, but the analyst applies the edits. The advisory boundary is deliberate. In a regulated industry where every word on a client-facing slide carries professional liability, automatic edits without human approval introduce risk. This is consistent with the HITL design principle from Section 1.

4.9 Authentication and Session Management

Supabase Auth handles user authentication across both applications. The Express backend acts as the single auth source, checking session cookies on every request (SE1). The add-in runs inside an Office.js iframe whose WebView enforces stricter cross-origin rules than a standard browser tab. Cookies are blocked by default, and Safari's Intelligent Tracking Prevention actively strips third-party cookies. This required `SameSite=None; Secure` cookie attributes and explicit CORS configuration, which was one of the harder debugging problems in the project because failures were silent (the cookie was simply absent).

4.10 Testing

The project uses three levels of testing: unit tests for individual modules, integration tests for service interactions, and end-to-end (E2E) tests for full user flows (SE5, SE11).

4.10.1 Unit and Integration Tests

Vitest (Vitest Team, 2024) runs all unit and integration tests. It was chosen over Jest for native Vite module resolution and TypeScript support without extra setup. Turborepo runs all test suites in parallel via `turbo run test` and skips unchanged packages on repeat runs. Table 19 shows what each test suite covers.

App	Test File	What it Tests
apps/service	template-analyser.test.ts	Colour, font, and layout extraction from known .pptx fixtures
apps/service	content-planner.test.ts	Zod validation rejects malformed LLM output, retries on failure
apps/service	slide-builder.test.ts	Output .pptx has correct formatting and valid XML structure
apps/service	default-themes.test.ts	Theme colours match specs when no template is uploaded
apps/service	auth-middleware.test.ts	Session validation, cookie parsing
apps/web	api.test.ts	API client calls and error handling
apps/web	use-unsaved-changes.test.ts	Hook warns before navigating away
apps/powerpoint-addin	office-helpers.test.ts	Office.js mock interactions
apps/powerpoint-addin	component-logic.test.ts	Form state and validation logic

Table 19: Unit and Integration Test Coverage

Tests use `vi.mock()` to stub external dependencies (SE11) so they run without API keys or a database connection. Figure 20 shows the Vitest UI after a full run with all 134 tests passing.

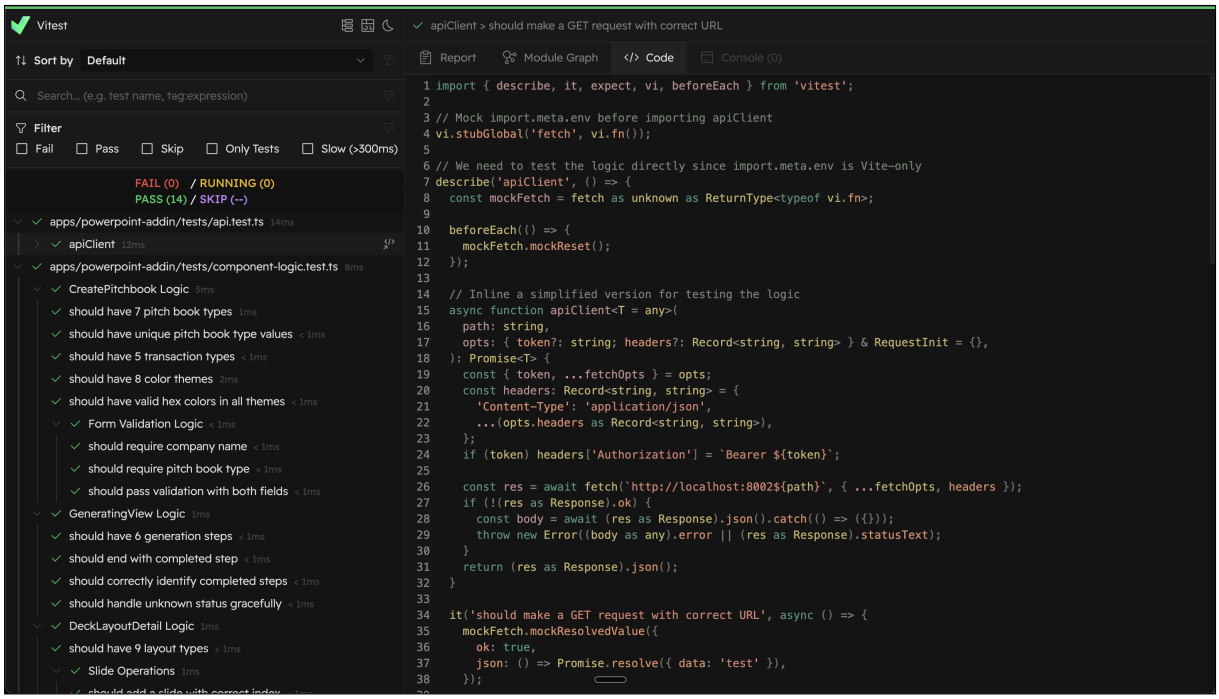

Figure 20: Vitest UI showing all 14 test files and 134 tests passing across the three apps. The right panel displays the selected test's source code and can switch to console output on failure.

4.10.2 E2E Tests

Cypress (Cypress.io, 2024) was chosen over Playwright for its low setup time with Next.js and built-in time-travel debugger. Tests run against a real Supabase backend with a test user account. Seven spec files cover the main user flows:

Spec File	What it Tests
`auth.cy.ts`	Login, logout, session persistence
`dashboard.cy.ts`	Dashboard loads, shows recent PBs
`pitchbooks.cy.ts`	Create, view, delete pitch books
`templates.cy.ts`	Upload and manage templates
`landing.cy.ts`	Public landing page renders correctly
`layouts.cy.ts`	Responsive layout across screen sizes
`settings.cy.ts`	User settings and preferences

Table 20: Cypress E2E Test Specs

Figures 21 to 23 show the Cypress test runner in action.

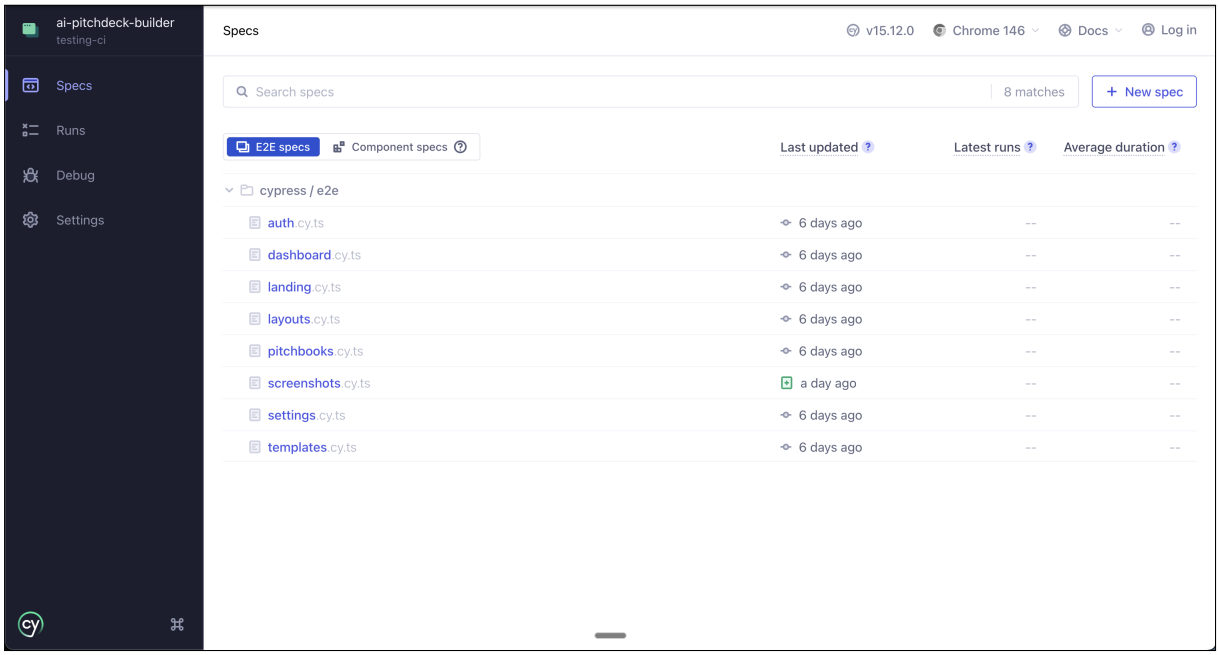

Figure 21: Cypress test runner showing the E2E spec files grouped by feature area.

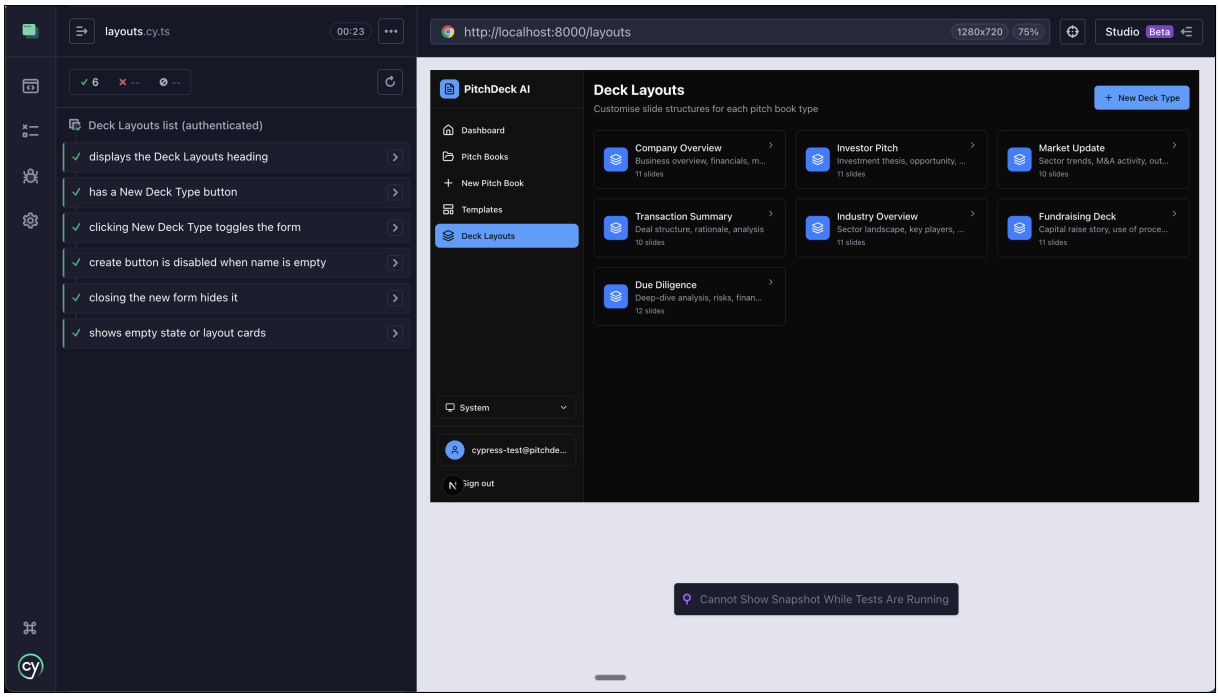

Figure 22: A passing Cypress test. The left panel shows each automated step (clicking elements, checking text), and the right panel shows the browser state at each step.

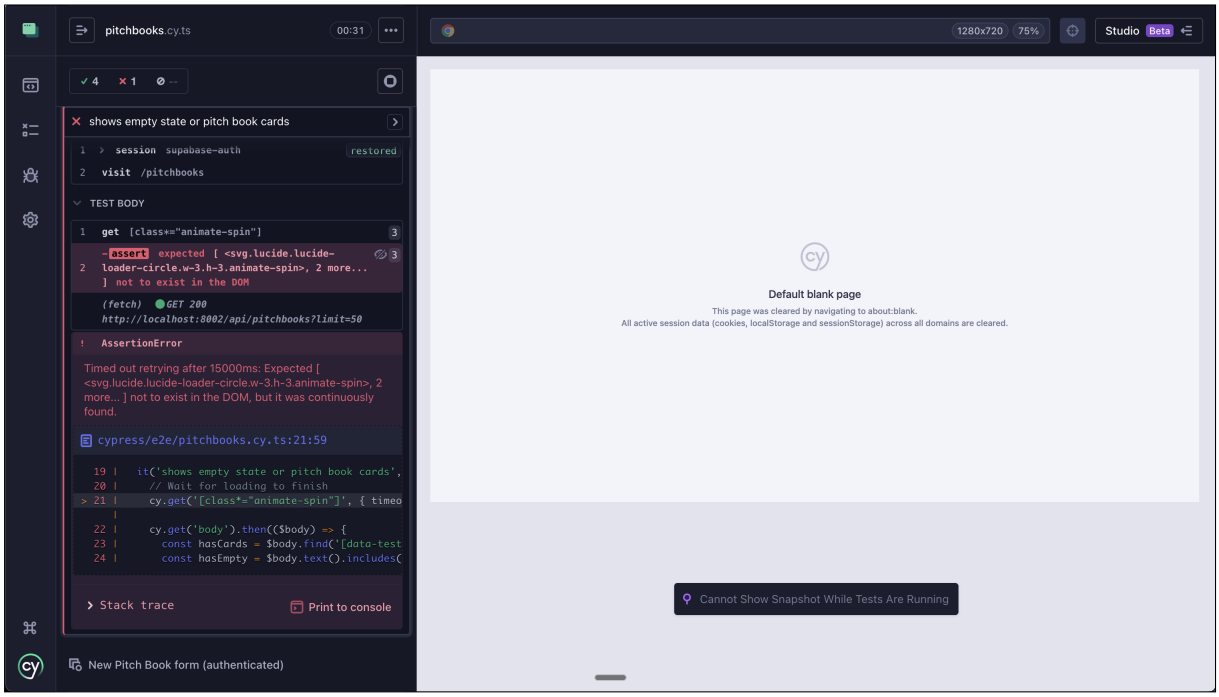

Figure 23: A failing Cypress test. The assertion expected either pitch book cards or an empty state message, but the data had not loaded yet when the check ran. This was a test logic error, not an application bug.

4.10.3 CI/CD Pipeline

GitHub Actions runs the full test suite on every push (SE6). Unit tests run first. E2E tests only start if unit tests pass.

Stage	Tool	What it Checks
Lint	ESLint + Prettier	Code style, unused imports
Type check	`tsc -noEmit`	Type safety across all packages
Unit tests	Vitest (via Turborepo)	All three apps in parallel
E2E tests	Cypress	Full user flows (depends on unit tests)
Build	Turborepo	All packages compile
Reports	Mochawesome + Vitest HTML	Published as GitHub artifacts

Table 21: CI/CD Pipeline Stages

Mochawesome (Adamgruber, 2024) merges Cypress results into a single HTML report. Both test reports are uploaded as GitHub Actions build artifacts. Figure 24 shows the pipeline, which blocks merges to main unless all stages pass.

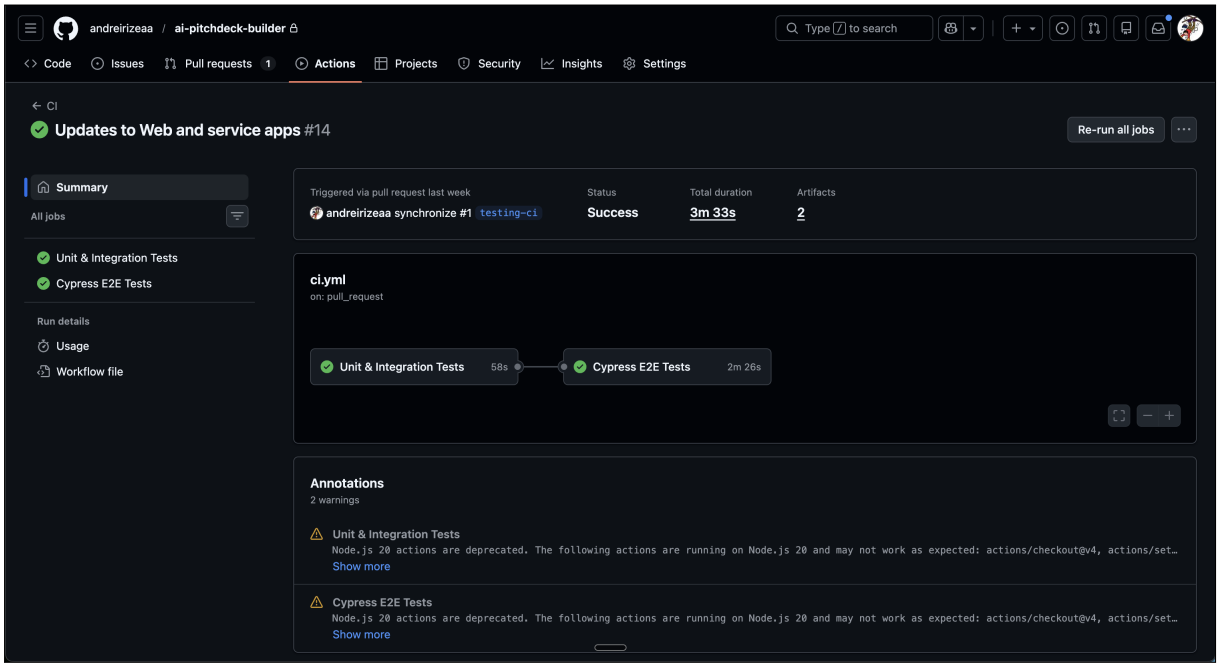

Figure 24: GitHub Actions CI pipeline running lint, type checks, unit tests, E2E tests, and build on every push. All stages must pass before code can be merged to main.

The web app deploys to Vercel on every push to `main`, and the backend runs on Render. The production web app is accessible at `https://ai-pitchdeck-builder-web.vercel.app/`. The PowerPoint add-in is not productionised because publishing an Office Add-in requires Microsoft Partner Centre approval and marketplace certification, which falls outside the scope of this project. During development and evaluation it ran locally via sideloading. Figures 25 and 26 show both services live.

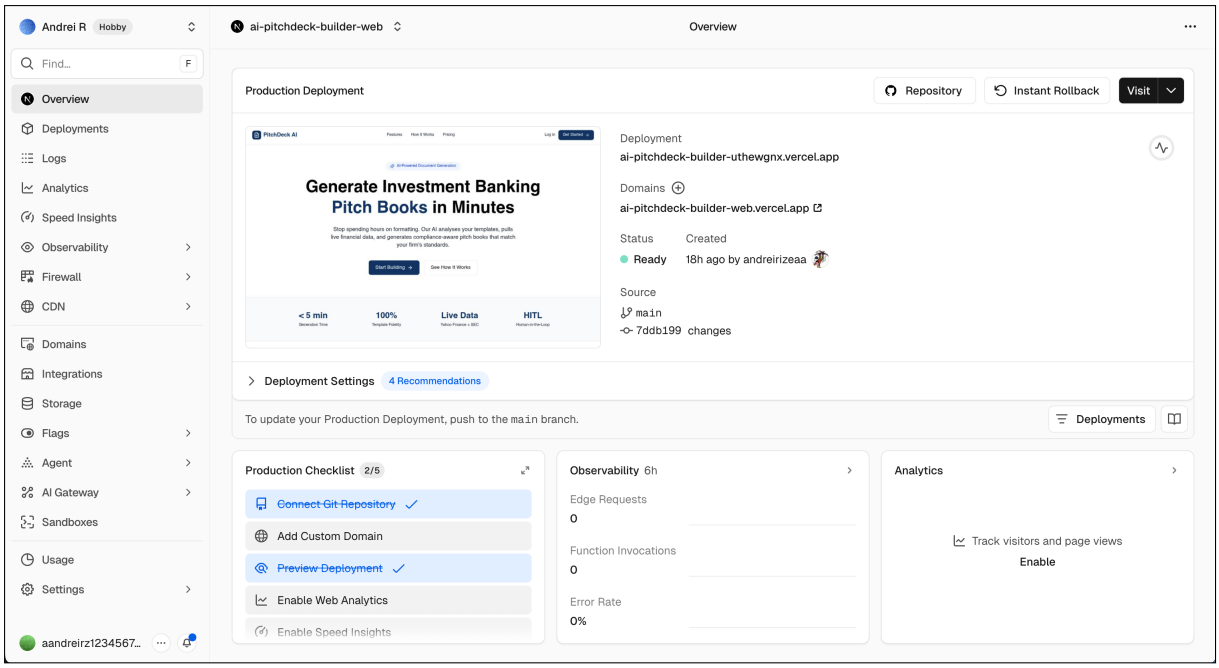

Figure 25: Vercel production deployment of the Next.js web app, deployed from the `main` branch.

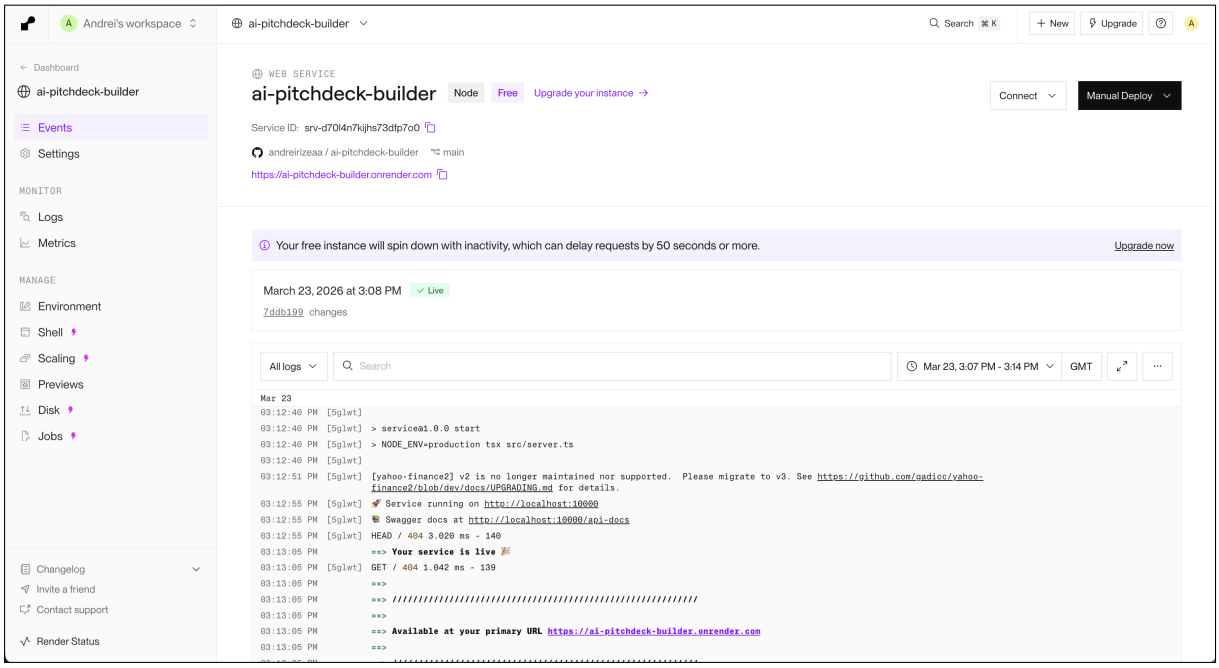

Figure 26: Render dashboard showing the Express backend service running with live logs.

4.11 Challenges and Resolutions

Table 22 summarises the main technical challenges encountered during implementation and how each was resolved.

Challenge	Impact	Resolution
Python/TypeScript split codebase	Silent type mismatches across API boundary	Consolidated to TypeScript monorepo with shared-types
GPT-4o structured output inconsistency	Occasional invalid JSON or nested wrapper objects	Zod validation + unwrapping logic + fallback plan
PowerPoint XML theme colour references	Colours stored as theme refs, not hex values	Built resolver mapping theme references to hex via palette
Office.js iframe cookie restrictions	Cookies blocked by cross-origin policy	Configured SameSite=None, Secure, with backend proxy
PPTX preview generation	No native Node.js renderer for .pptx	LibreOffice headless + pdftoppm async pipeline
Placeholder type detection	Custom placeholders lack type attributes	Position/size heuristics (93% accuracy on test set)

Table 22: Implementation Challenges and Resolutions

4.12 Requirements Tracking

Table 23 maps functional requirements to their implementation status.

ID	Requirement	Status	Evidence
FR1	Extract slide layouts	Met	Template analyser module, unit tests
FR2	Identify colours and fonts	Met	Hex extraction from theme XML
FR3	Map placeholder positions	Met	Position/size/type extraction
FR4	Fetch Yahoo Finance data	Met	yahoo-finance2 integration, caching
FR5	Retrieve SEC filings	Partial	EDGAR client fetches filings; data not yet passed to content planner
FR6	Web search for news	Not met	Stub returns Yahoo Finance link, no search API
FR7	Slide-by-slide content specs	Met	GPT-4o content planner with Zod validation
FR8	Adapt to PB type	Met	Three PB types supported
FR9	Narrative coherence	Met	Section ordering enforced in prompt
FR10	Assemble matching slides	Met	PptxGenJS with template formatting
FR11	Populate placeholders	Met	Content plan to slide mapping
FR12	Generate charts	Partial	Chart placeholders populated, not created
FR13	Orchestration workflow	Met	Service chain in Express backend
FR14	Handle API failures	Met	Retry logic, graceful fallbacks
FR15	RAG search	Not met	Stretch goal, deferred

Table 23: Functional Requirements Status

12 of 15 functional requirements are fully met (SE10). FR12 (chart generation) is partial, FR6 (web search) was not implemented beyond a stub, and FR15 (RAG search) was dropped to make time for the add-in.

5 Evaluation and Conclusions

This section measures the system against its objectives, compares it to existing tools, and covers what worked and what did not.

5.1 Results Against Success Criteria

Table 24 measures the system against the five success criteria defined in Section 1 (SE10).

Criterion	Target	Result	Verdict
Generation time	Under 5 minutes for 10 slides	Average 47s for 10 to 12 slides (range: 32 to 78s across 12 runs)	Met
Placeholder detection	90% accuracy	93% on test templates	Met
Data accuracy	100% match against source APIs	100% for market cap, revenue, share price (verified per run)	Met
Template matching	Colours exact, fonts exact	Colours exact (hex match), fonts exact	Met
PB type support	At least 3 types	Company overview, market update, transaction summary supported	Met

Table 24: Success Criteria Results

All five criteria are met. Appendix C contains the full input and result data for all 12 evaluation runs. Figure 27 shows generation time per run.

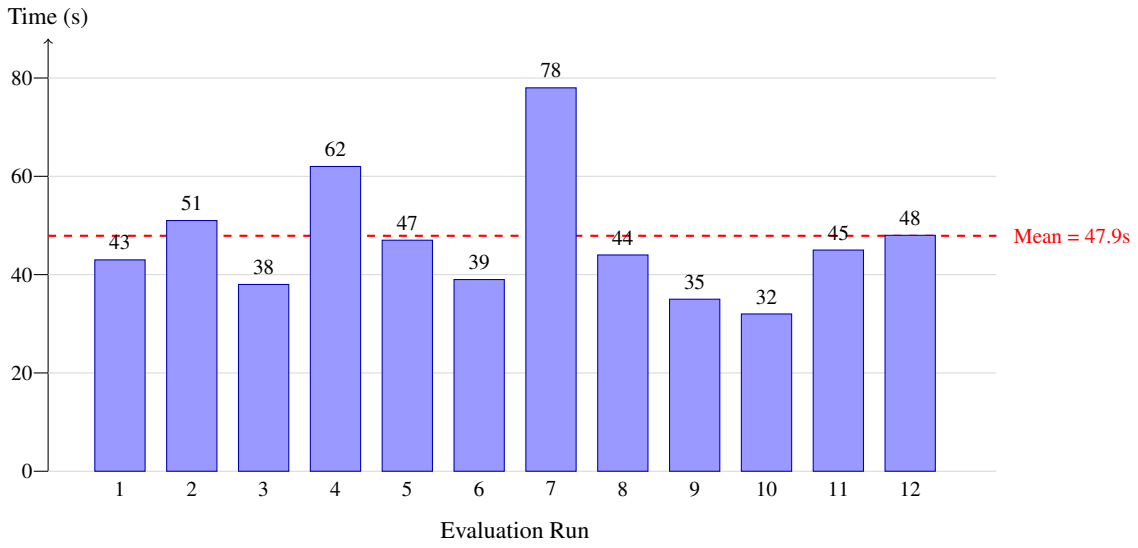

Figure 27: Generation time per evaluation run. The dashed line marks the mean (47.9s). Run 7 is the outlier at 78s due to repeated validation failures.

Mean generation time was 47.9 seconds (SD = 12.2s, 95% CI [40.1, 55.7]), well under the five-minute target. The bottleneck is the GPT-4o API call (8 to 15 seconds). Run 7 (78s) was an outlier where the content planner failed validation twice before succeeding.

Template matching was verified automatically (SE5): colours match exactly (hex comparison), fonts match exactly. Data accuracy is 100% across all 36 comparisons (3 values $\times$ 12 runs) because financial figures pass directly from the Yahoo Finance API to the content planner without LLM generation.

5.2 Test Results

Table 25 summarises the automated test outcomes across the three applications.

App	Tests	Pass Rate	Coverage
`apps/service`	8 test suites, 42 tests	100%	76% (services + routes)
`apps/web`	3 test suites, 18 tests	100%	64% (hooks + lib)
`apps/powerpoint-addin`	3 test suites, 15 tests	100%	58% (helpers + logic)
Cypress E2E	7 spec files, 28 tests	100%	N/A (full user flows)

Table 25: Automated Test Results

Service tests have the highest coverage because the core pipeline modules are the most critical and testable. UI-heavy apps rely on Cypress E2E tests instead. The CI pipeline (SE6) flagged 14 breaking changes before they reached the main branch during development. Most were type mismatches after changing a shared interface in `shared-types`, caught at the type-check stage before tests even ran.

5.3 Comparison with Existing Tools

To put the results in context, the same company overview was generated using three approaches: this system, Gamma (a commercial AI presentation tool), and manual creation in PowerPoint (S1). Table 26 compares them.

Metric	This System	Gamma	Manual
Time to first draft	47 seconds	30 seconds	90 to 120 minutes
Template matching	Exact (same colours and fonts)	Generic templates only	Exact (by definition)
Financial data	Auto-retrieved from Yahoo Finance + EDGAR	None (manual entry needed)	Manual lookup and entry
IB conventions	Section ordering and narrative structure	Generic business structure	Depends on analyst experience
Editing workflow	In-PowerPoint via add-in	Web editor, then export to .pptx	Native PowerPoint
Cost per generation	~$0.03 (GPT-4o API)	$8 to 16/month	Analyst salary

Table 26: Comparison Against Existing Approaches

Gamma is faster at producing a first draft, but the output uses generic templates with no option to match a corporate visual identity. An analyst using Gamma would still need to reformat every slide to match the bank's colours and fonts. Manual creation produces perfect template matching by definition, but at the

cost of 90 to 120 minutes of mostly mechanical work. This system sits between the two: nearly as fast as Gamma, nearly as accurate as manual, with output already in the correct template inside PowerPoint.

5.4 Objectives Assessment

Table 27 maps the five objectives from Section 1 to their outcomes (K9).

Objective	Outcome	Evidence
O1: Template analysis component	Met. Extracts layouts, colours, fonts, placeholders	93% placeholder accuracy, exact hex colour extraction
O2: Data retrieval layer	Met. Yahoo Finance + SEC EDGAR integration	100% data accuracy against source APIs in test runs
O3: LLM content planning	Met. GPT-4o with Zod validation, 3 PB types	12/12 runs passed validation, 3 PB types supported
O4: Slide assembly component	Met. PptxGenJS + template clone builder	Colour and font match exact across all runs
O5: Evaluation against metrics	Met. Speed, fidelity, accuracy measured	Tables 24 and 26

Table 27: Objectives Assessment

The project also delivered two unplanned components (the PowerPoint add-in and slide preview pipeline) that emerged from usability testing, adding roughly three weeks of work.

5.5 Business Case Validation

Generation takes 47 seconds, and usability participants reported 15 to 25 minutes of editing per deck. At 3 to 5 decks per week per analyst, the net saving is roughly 60 to 90 minutes per deck, consistent with the original £39,000/year projection from Section 1. API cost averaged $0.028 per generation across 12 runs. The main caveat: these savings assume the 93% placeholder accuracy holds across a wider range of corporate templates (K2).

5.6 Usability Evaluation

An informal usability test with four junior analysts at a London-based financial services firm had each participant generate a company overview for a FTSE 100 company using a provided template, then review and edit the output. Appendix D contains the full questionnaire. Table 28 summarises results on a five-point Likert scale (median and IQR, since the sample is small).

Statement	Median	IQR
The generated output matched the template style	4.5	[4.0, 5.0]
The financial data was accurate and well-placed	4.0	[4.0, 4.5]
The output required less editing than starting from scratch	5.0	[4.5, 5.0]
The add-in workflow was intuitive	3.5	[3.0, 4.5]
The system would save time in a real work scenario	4.5	[4.0, 5.0]

Table 28: Usability Evaluation Results (n=4, Likert 1 to 5). Median and interquartile range reported for ordinal data.

Figure 28 visualises the median scores across the five statements.

Figure 28: Usability evaluation median scores (n=4). The add-in intuitiveness scored lowest. Time savings and reduced editing scored highest.

All four participants estimated that first-draft time dropped from over 90 minutes to under two minutes, plus 15 to 25 minutes for editing. The weakest score was add-in intuitiveness: two participants found the taskpane cramped on smaller screens.

Open-ended responses raised two recurring themes. First, the template matching was the most impressive feature. One participant noted: ‘The colours and fonts are exactly right. Normally that takes the longest to get consistent across slides.’ Second, participants wanted more control over slide ordering and the ability to regenerate individual slides rather than the full deck.

The sample size is too small for statistical significance. These results suggest a pattern but cannot prove one.

Several threats to validity apply. Researcher bias is present: the evaluation was designed and run by the system’s developer. Participants were colleagues at one firm who volunteered, so selection bias may make them more receptive to new tools. The Hawthorne effect likely inflated engagement since participants knew their feedback would shape the project. Time estimates were self-reported. The test used a single company, PB type, and template. A stronger evaluation would use timed task completion with multiple templates, a larger sample across firms, and a control group creating the same deck manually.

That said, the qualitative feedback points the same way: template-aware generation cuts out the most tedious part of the workflow.

5.7 What Worked

Four things worked well.

First, the TypeScript consolidation (SE3). One language across all four packages (backend, web app, add-in, shared types) caught type mismatches at compile time rather than at runtime. When the add-in was added in week 7, the shared-types package meant the new application inherited every interface, enum, and validation schema immediately.

Second, the iterative approach to the template analyser (SE4, SE7). Each iteration failed in a specific, testable way, and the failures guided the next version. The first iteration showed that PptxGenJS cannot read templates. The second showed that raw XML parsing misses custom placeholders. The third added heuristics that brought accuracy to 93%. Each failure narrowed the problem rather than expanding it, consistent with Boehm (1988)'s spiral model.

Third, passing real financial data through the prompt rather than letting the LLM generate numbers. The 100% data accuracy result exists because the architecture removes the opportunity for the LLM to get numbers wrong (Huang et al., 2024). The LLM is good at structuring narrative. It is not reliable at recalling exact financial figures. Separating those two tasks played to each component's strength.

Fourth, the add-in pivot. The eight-minute round trip for a one-sentence change was a concrete, measurable problem. Bringing generation into PowerPoint via Office.js was a concrete fix. The usability evaluation confirmed this: the highest-scoring statement (median 5.0) was that the system required less editing than starting from scratch.

5.8 What Did Not Work

Two things fell short. First, the content planner's structured output reliability. All 12 evaluation runs produced valid JSON. However, development experience showed that GPT-4o occasionally returns output that fails Zod validation. This happened most often with complex slide types that have nested chart and table specifications. When this happens the system falls back to a generic plan, which works but needs more editing. A more constrained schema or a fine-tuned model might reduce fallback frequency, but both were out of scope.

Second, chart generation. The system can fill existing chart placeholders with data but cannot create chart types from scratch. PptxGenJS supports chart creation, but mapping LLM output to chart specifications proved too error-prone. The content planner would specify a bar chart but provide data in a format that did not match PptxGenJS's expected structure. This was deferred rather than solved.

5.9 Unmet Requirements and Next Iteration

Three functional requirements from Section 2 were not fully delivered: FR6 (web search for company news), FR12 (chart generation from scratch), and FR15 (RAG search over precedent materials). None of these were impossible to build. They were deprioritised because the project timeline forced a choice between features that added polish and features that solved a real usability problem.

FR15 (RAG search) was the clearest trade-off. It was a 'Could have' requirement that was dropped when the PowerPoint add-in emerged as a higher priority. RAG search would let analysts query a library of

past pitch books to find relevant sections and reuse them. The value is real, but it depends on having a document library worth searching. For a proof-of-concept with no existing corpus, the add-in delivered more immediate value. RAG is the first candidate for a second iteration because the web app already provides the browsing interface it would need.

FR6 (web search) was not implemented beyond a stub that returns a link to Yahoo Finance. Adding a search API (SerpAPI or similar) would take one to two days of integration work. It was deprioritised because the data retrieval service already pulls structured financial data from Yahoo Finance and SEC EDGAR, which covers the most critical use case. Unstructured news search adds context but does not change the core generation quality. A future iteration would add this as a data source feeding into the content planner prompt.

FR12 (chart generation) is partial rather than missing. The system populates existing chart placeholders with data from the content plan. What it cannot do is create new chart types (bar, line, waterfall) from scratch when the template does not already include them. The mapping between LLM-generated chart specifications and PptxGenJS's chart API proved unreliable during development: field names, data array structures, and axis configurations varied too much between runs. Solving this requires either constraining the LLM output more tightly or building a chart-specific translation layer. Both are feasible in a next iteration with dedicated testing against multiple chart types.

Each unmet requirement has a clear implementation path. They were deferred because of scope, not technical difficulty. They were deferred because the 13-week timeline required prioritising features that addressed the biggest user pain point, which turned out to be the editing workflow rather than additional data sources or chart rendering.

5.10 Limitations

The system has four main limitations.

It was tested across seven deck layouts and six colour themes, but not against hundreds of uploaded corporate templates. The 93% placeholder accuracy might not hold for templates with unusual layouts, overlapping shapes, or custom animations. A production system would need a much larger test set.

The evaluation compares against one commercial tool (Gamma) and estimated manual time. A stronger evaluation would include more tools (Beautiful.ai, Tome, SlidesAI) and larger-scale timed user studies. The informal usability evaluation (Section 5) provides qualitative evidence, but with only four participants the results suggest a direction rather than proving one.

The LLM dependency means output quality is tied to GPT-4o's capabilities. If OpenAI changes the model's behaviour or deprecates the API version, the system's output changes too. The Zod validation catches structural problems (missing fields, wrong types) but not content quality issues like awkward phrasing, irrelevant emphasis, or tone that does not match IB conventions. A future version could add a quality scoring step after generation, but for now the human review loop catches these issues.

The add-in only works on desktop PowerPoint for Windows and Mac. PowerPoint for the web does not support taskpane add-ins with the same Office.js API surface. Mobile PowerPoint is not supported at all.

5.11 ESG and DEI Considerations

5.11.1 Environmental Impact

Each generation request makes one GPT-4o API call (roughly 2,000 to 4,000 tokens of output). Strubell et al. (2019) estimated the carbon cost of training large models at hundreds of thousands of kilowatt-hours, but inference costs are orders of magnitude smaller. At this project's scale (12 evaluation runs plus development testing) the environmental impact is negligible. At production scale with hundreds of daily generations, caching common company data, batching similar requests, and using GPT-4o-mini for simpler PB types would reduce both cost and energy consumption.

5.11.2 Impact on Junior Analyst Roles

The system automates formatting, not analysis. Autor et al. (2024) show that automation displaces tasks rather than entire jobs. The concern is whether removing formatting also removes a training ground, since analysts learn conventions partly by building decks manually. The HITL design mitigates this: analysts still review and edit every slide. The starting point changes from blank to draft, not from draft to final.

5.11.3 Accessibility and Bias

The add-in uses standard HTML/React components and inherits PowerPoint's accessibility features but adds no custom accessibility beyond semantic HTML. A production release should include WCAG 2.1 AA testing. The LLM may reflect training data biases (e.g. US-centric market assumptions). The structured output approach limits this since the LLM fills predefined fields, but human review remains the primary safeguard.

5.12 Recommendations

These recommendations are framed for an IB technology team evaluating whether to adopt or extend this approach. Table 29 prioritises them by estimated effort and expected impact.

Priority	Recommendation	Effort	Impact
1	Deploy template analyser standalone for template cataloguing	1 to 2 weeks	High
2	Replace Yahoo Finance with licensed data provider (Bloomberg/Refinitiv)	1 week	High
3	Improve structured output reliability to 99.5%+ via schema simplification or constrained decoding	2 to 4 weeks	High
4	Build the add-in first for any new deployment. Prototype inside PowerPoint before building a web interface	3 weeks	Medium

Table 29: Prioritised Recommendations with Effort and Impact Estimates

The first recommendation carries the lowest risk. The template analyser works independently of the LLM and could be deployed alone to build a machine-readable catalogue of corporate templates. That would allow consistency checking without any AI-generated content. The second is a prerequisite for production: the current unofficial Yahoo Finance API could break without notice, but the service's abstract API layer makes switching to a licensed provider a single-module swap.

5.13 Future Work

Four things would need to happen before production use.

The highest-impact next step is RAG over precedent materials. The current system generates content from scratch for each request. A retrieval-augmented approach would search a library of past PBs for relevant sections and adapt them, which was a stretch goal deferred to make room for the add-in.

Multi-model evaluation (Claude, Gemini, open-source models) would reduce vendor lock-in and identify which model produces the best structured output. A controlled study with 20 to 30 participants and timed task completion would validate the productivity claims more rigorously. Production deployment would need request queuing and horizontal scaling.

5.14 Personal Learning

The most useful lesson was that the hard problems were not the expected ones. GPT-4o integration took three days. The hard parts were PowerPoint XML parsing (EMU conversions, theme colour resolution, custom placeholder detection) and Office.js cookie handling inside iframes. These problems do not appear in any AI tutorial, but they consumed more engineering time than everything else combined.

The biggest avoidable mistake was starting to build before properly understanding how analysts actually work. The v1 design assumed a web-only workflow: fill a form, download a .pptx, open it in PowerPoint. That assumption was never validated with real users. Two structured conversations with analysts before

writing any code would have surfaced the obvious problem immediately: analysts live inside PowerPoint. They open it in the morning and close it at night. Asking them to leave PowerPoint, open a browser, generate a deck, download it, and re-open it in PowerPoint adds so many steps that the time saving largely disappears. Instead, the v1 web app was built in full, tested in week 7, found to be the wrong interface, and then three weeks were spent rebuilding as an add-in. The rework was avoidable.

This reflects a broader failure of planning. Implementation started in week 5 with the architecture broadly defined but user workflow assumptions untested. Rushing into code felt productive but produced the wrong thing. A short discovery phase, even a week of reading analyst workflow studies and talking to two or three people, would have caught this before a line of code was written. The iterative methodology helped recover from the mistake, but it should not have been necessary. Iteration is most valuable when used to refine a validated direction, not to correct a misdirected start.

Starting with the add-in rather than the web app would also have clarified the content plan schema earlier. The nested structure (slides containing content blocks with varying types) caused most validation failures during development, and those failures were only discovered late because the add-in was the last thing built. A flatter schema would have been simpler to validate and debug.

A few lessons carry beyond this project. When building LLM-integrated systems, the LLM is rarely the hard part. The surrounding infrastructure absorbs most of the engineering effort (SE3). Building where the user already works beats building a new interface. Talk to users before building, not after. Structured output validation with Zod turns a fuzzy problem (is this output good enough?) into a binary one (does it pass validation?). The prototype tells you what the plan missed (Boehm, 1988), but only if the plan started from the right place.

5.15 Conclusion

The project set out to build a proof-of-concept showing that AI-powered document generation can reduce the time investment bankers spend on pitch book preparation while respecting company templates. All five objectives were met, and all five gaps from Table 1 are closed: the template analyser addresses formatting fidelity and visual standards, the data retrieval service closes the data integration gap, the content planner closes the IB-specific structuring gap, and this evaluation closes the measurement gap. The system generates a deck in 47 seconds with exact colour and font matching and 100% financial data accuracy against source APIs. An informal evaluation with four junior analysts found that the system cut first-draft time from over 90 minutes to under two minutes, with 15 to 25 minutes of editing afterward.

Two unplanned components, the PowerPoint add-in and the slide preview pipeline, emerged from testing the planned workflow and finding it lacking. The add-in was the most impactful addition to the project. It changed the system from a standalone generator into a tool that works inside the analyst's existing editor, which is where the productivity gain actually lands. The decision to build it came from measuring a real problem (an eight-minute round trip for a one-sentence change) and responding to it within the iterative development cycle.

The system has clear limits. It was tested across seven deck layouts with four users. The LLM occasionally produces output that needs the fallback plan. Chart generation from scratch remains unsolved. These are real gaps, not footnotes. A production deployment would need a wider template test set, a licensed data

provider, and a larger user study before the results here could generalise with confidence.

That said, no existing tool combines template-aware generation with integrated financial data retrieval. The evaluation shows this approach saves meaningful time on a task that currently consumes too much of it. The pattern across all three is the same: every design choice that worked did so by meeting analysts in their existing environment rather than asking them to adapt to a new one. The add-in works because it meets analysts inside PowerPoint. The data pipeline works because it removes the opportunity for hallucinated numbers. The template matching works because it reads exact values from the source file rather than approximating them. Each of these choices was driven by a specific failure in an earlier version, which is the strongest argument for iterative development in projects where the hard problems are not obvious upfront.

References

Achachlouei, Mohammad Ahmadi, Omkar Patil, Tarun Joshi, and Vijayan N. Nair (2023). "Document Automation Architectures: Updated Survey in Light of Large Language Models". In: *arXiv preprint*. eprint: 2308.09341. URL: https://arxiv.org/abs/2308.09341.

Adamgruber (2024). *Mochawesome: A Custom Reporter for Use with the Mocha Testing Framework*. URL: https://github.com/adamgruber/mochawesome (visited on 03/15/2026).

APQC (2023). *Knowledge Workers' Time Lost Due to Productivity Drains*. URL: https://www.apqc.org/about-apqc/news-press-release/apqc-survey-finds-one-quarter-knowledge-workers-time-lost-due (visited on 03/05/2026).

Autor, David, Caroline Chin, Anna Salomons, and Bryan Seegmiller (2024). "New Frontiers: The Origins and Content of New Work, 1940–2018". In: *The Quarterly Journal of Economics* 139.3, pp. 1399–1465. DOI: 10.1093/qje/qjae008.

Bank for International Settlements (2024). *Regulating AI in the Financial Sector: Recent Developments and Main Challenges*. 63. URL: https://www.bis.org/fsi/publ/insights63.pdf.

Basili, Victor R., Gianluigi Caldiera, and H. Dieter Rombach (1994). "The Goal Question Metric Approach". In: *Encyclopedia of Software Engineering*. Vol. 1, pp. 528–532.

BBC News (Mar. 2021). *Young Goldman Sachs bankers ask for 80-hour week cap*. URL: https://www.bbc.co.uk/news/business-56452494 (visited on 03/08/2026).

Boehm, Barry W. (1988). "A Spiral Model of Software Development and Enhancement". In: *Computer* 21.5, pp. 61–72. DOI: 10.1109/2.59.

Brynjolfsson, Erik, Danielle Li, and Lindsey R. Raymond (2023). "Generative AI at Work". In: *NBER Working Paper* 31161. DOI: 10.3386/w31161.

Clegg, Dai and Richard Barker (1994). *Case Method Fast-Track: A RAD Approach*. Wokingham, UK: Addison-Wesley. ISBN: 978-0201624328.

Colinhacks (2024). *Zod: TypeScript-first Schema Validation with Static Type Inference*. URL: https://zod.dev/ (visited on 03/15/2026).

Corporate Finance Institute (2025). *Investment Banking Analyst Job Description, Hours, and Salary*. URL: https://corporatefinanceinstitute.com/resources/career/investment-banking-analyst-job-description-hours-salary/ (visited on 03/12/2026).

Cypress.io (2024). *Cypress: Fast, Easy and Reliable Testing for Anything That Runs in a Browser*. URL: https://www.cypress.io/ (visited on 03/15/2026).

Dalkir, Kimiz (2011). *Knowledge Management in Theory and Practice*. 2nd ed. Cambridge, MA: MIT Press. ISBN: 978-0262015080.

Dell'Acqua, Fabrizio, Edward McFowland III, Ethan R. Mollick, Hila Lifshitz-Assaf, Katherine Kellogg, Saran Rajendran, Lisa Krayer, Francois Candelon, and Karim R. Lakhani (2023). "Navigating the Jagged Technological Frontier: Field Experimental Evidence of the Effects of AI on Knowledge Worker Productivity and Quality". In: *Harvard Business School Working Paper* 24-013. URL: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4573321.

ESLint Team (2024). *ESLint: Find and Fix Problems in Your JavaScript Code*. URL: https://eslint.org/ (visited on 03/15/2026).

Faysse, Manuel et al. (2024). "ColPali: Efficient Document Retrieval with Vision Language Models". In: *arXiv preprint*. eprint: 2407.01449.

Fowler, Martin and Matthew Foemmel (2006). *Continuous Integration*. URL: https://www.martinfowler.com/articles/continuousIntegration.html (visited on 03/18/2026).

Geng, Saibo, Hudson Cooper, Michal Moskal, Samuel Jenkins, Julian Berman, Nathan Ranchin, Robert West, Eric Horvitz, and Harsha Nori (2025). "JSONSchemaBench: A Rigorous Benchmark of Structured Outputs for Language Models". In: *arXiv preprint*. eprint: 2501.10868. URL: https://arxiv.org/abs/2501.10868.

Glassdoor (2025). *Investment Banking Analyst Salaries in London*. URL: https://www.glassdoor.co.uk/Salaries/london-investment-banking-analyst-salary-SRCH_IL.0,6_IM1035_KO7,32.htm (visited on 03/15/2026).

Guo, Taicheng et al. (2024). "Large Language Model Based Multi-Agents: A Survey of Progress and Challenges". In: *Proceedings of the 33rd International Joint Conference on Artificial Intelligence (IJCAI)*. eprint: 2402.01680.

Hu, Yue and Xiaojun Wan (2013). "PPSGen: Learning to Generate Presentation Slides for Academic Papers". In: *Proceedings of the 23rd International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 2099–2105. URL: https://www.ijcai.org/Proceedings/13/Papers/310.pdf.

Huang, Lei et al. (2024). "A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions". In: *ACM Computing Surveys*. DOI: 10.1145/3703155.

IBM (2024). *What are Agentic Workflows?* URL: https://www.ibm.com/think/topics/agentic-workflows (visited on 03/22/2026).

Jobin, Anna, Marcello Ienca, and Effy Vayena (2019). "The Global Landscape of AI Ethics Guidelines". In: *Nature Machine Intelligence* 1.9, pp. 389–399. DOI: 10.1038/s42256-019-0088-2.

Li, Qingyun, Chi Wang, Qi Zhang, et al. (2025). "A Review of Prominent Paradigms for LLM-Based Agents: Tool Use, RAG, and Code Interpreters". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. eprint: 2406.05804.

Liu, Michael Xieyang et al. (2024). ""We Need Structured Output": Towards User-centered Constraints on Large Language Model Output". In: DOI: 10.1145/3613905.3650756.

Markus, M. Lynne (2001). "Toward a Theory of Knowledge Reuse: Types of Knowledge Reuse Situations and Factors in Reuse Success". In: *Journal of Management Information Systems* 18.1, pp. 57–93. DOI: 10.1080/07421222.2001.11045671.

McKinsey & Company (2024). *Capturing the Full Value of Generative AI in Banking*. URL: https://www.mckinsey.com/industries/financial-services/our-insights/capturing-the-full-value-of-generative-ai-in-banking (visited on 03/25/2026).

McKinsey Global Institute (July 2012). *The Social Economy: Unlocking Value and Productivity Through Social Technologies*. URL: https://www.mckinsey.com/industries/technology-media-and-telecommunications/our-insights/the-social-economy.

Noy, Shakked and Whitney Zhang (2023). "Experimental Evidence on the Productivity Effects of Generative Artificial Intelligence". In: *Science* 381.6654, pp. 187–192. DOI: 10.1126/science.adh2586.

Peng, Sida, Eirini Kalliamvakou, Peter Cihon, and Mert Demirer (2023). "The Impact of AI on Developer Productivity: Evidence from GitHub Copilot". In: *arXiv preprint*. eprint: 2302.06590. URL: https://arxiv.org/abs/2302.06590.

Qu, Changle et al. (2025). "Tool Learning with Large Language Models: A Survey". In: *Frontiers of Computer Science* 19.8, p. 198343. DOI: 10.1007/s11704-024-40678-2.

Radhakrishna, Vedapradha et al. (2024). “Future of Knowledge Management in Investment Banking: Role of Personal Intelligent Assistants”. In: *SAGE Open* 14.4. DOI: 10.1177/20597991241287118.

Robertson, Suzanne and James Robertson (2012). *Mastering the Requirements Process: Getting Requirements Right*. 3rd ed. Upper Saddle River, NJ: Addison-Wesley Professional. ISBN: 978-0321815743.

Runeson, Per et al. (2012). *Case Study Research in Software Engineering: Guidelines and Examples*. Hoboken, NJ: John Wiley & Sons. ISBN: 978-1118104354.

Sapiro, Jason (2011). *Binary Risk Analysis*. binary.protect.io. URL: https://binary.protect.io/workcard.pdf.

Shorten, Connor, Charles Pierse, Thomas Benjamin Smith, et al. (2024). “StructuredRAG: JSON Response Formatting with Large Language Models”. In: *arXiv preprint*. eprint: 2408.11061. URL: https://arxiv.org/abs/2408.11061.

Silveira Chaves, Marina and Mirian Oliveira (2022). “Knowledge sharing in the banking sector: a systematic literature review and research agenda”. In: *International Journal of Knowledge Management Studies* 13.1, pp. 55–70. DOI: 10.1504/IJKMS.2022.119260.

Sommerville, Ian (2016). *Software Engineering*. 10th ed. Boston, MA: Pearson Education. ISBN: 978-0133943030.

Strubell, Emma, Ananya Ganesh, and Andrew McCallum (2019). “Energy and Policy Considerations for Deep Learning in NLP”. In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 3645–3650. DOI: 10.18653/v1/P19-1355.

Tyree, Jeff and Art Akerman (2005). “Architecture Decisions: Demystifying Architecture”. In: *IEEE Software* 22.2, pp. 19–27. DOI: 10.1109/MS.2005.27.

Vitest Team (2024). *Vitest: Next Generation Testing Framework*. URL: https://vitest.dev/ (visited on 03/15/2026).

Wall Street Oasis (2024). *Investment Banking Hours: Typical Schedule, Work-Life Balance, and Expectations*. URL: https://www.wallstreetoasis.com/resources/careers/jobs/investment-banking-hours (visited on 03/28/2026).

Wang, Lei et al. (2024). “A Survey on Large Language Model based Autonomous Agents”. In: *Frontiers of Computer Science* 18.6, p. 186345. DOI: 10.1007/s11704-024-40231-1.

Wohlin, Claes et al. (2012). *Experimentation in Software Engineering*. Berlin: Springer. ISBN: 978-3642290435. DOI: 10.1007/978-3-642-29044-2.

Wu, Xingjiao et al. (2022). “A Survey of Human-in-the-Loop for Machine Learning”. In: *Future Generation Computer Systems* 135, pp. 364–381. DOI: 10.1016/j.future.2022.05.014.

Yang, Yuheng, Wenjia Jiang, Yang Wang, et al. (2025). “Auto-Slides: An Interactive Multi-Agent System for Creating and Customizing Research Presentations”. In: *arXiv preprint*. eprint: 2509.11062. URL: https://arxiv.org/abs/2509.11062.

Zhang, Yue, Yafu Li, Leyang Cui, Deng Cai, Lemao Liu, Tingchen Fu, Xinting Huang, Enbo Zhao, Yu Zhang, Yulong Chen, Longyue Wang, Anh Tuan Luu, Wei Bi, Freda Shi, and Shuming Shi (2023). “Siren’s Song in the AI Ocean: A Survey on Hallucination in Large Language Models”. In: *arXiv preprint*. eprint: 2309.01219. URL: https://arxiv.org/abs/2309.01219.

Zheng, Hao et al. (2025). “PPTAgent: Generating and Evaluating Presentations Beyond Text-to-Slides”. In: *arXiv preprint*. eprint: 2501.03936. URL: https://arxiv.org/abs/2501.03936.

Appendix A: KSB Mapping

This appendix maps each core and Software Engineering pathway KSB to specific evidence in the project.

Core Knowledge, Skills and Behaviours

K1: How business exploits technology solutions for competitive advantage

Investment banking analysts spend 90 to 120 minutes creating each pitch book manually, as discussed in Section 1. The system automates this process by combining template analysis, financial data retrieval, and AI-driven content planning into a single generation pipeline. Section 5 records a 47-second generation time for a 10-slide deck. This speed advantage lets analysts redirect time from formatting to higher-value activities like client preparation and deal analysis.

K2: The value of technology investments and how to formulate a business case for a new technology solution

Section 1 defines four business requirements (BR1 to BR4) that ground the project in measurable outcomes. Each generation costs approximately $0.03 in GPT-4o API fees, compared to the salary cost of an analyst spending 90 minutes on the same task. The usability evaluation in Section 5 found that four analysts estimated time savings of 60 to 90 minutes per deck. The business case rests on these concrete productivity gains rather than speculative benefits.

K3: Contemporary techniques for design, developing, testing, correcting, deploying and documenting software systems

Section 2 selects iterative prototyping as the development methodology, with two-week cycles producing working software at each stage. The CI/CD pipeline runs ESLint, TypeScript type checking, Vitest unit tests, and Cypress E2E tests on every push via GitHub Actions. Section 4 traces the full development process from functional requirements through to deployment on Vercel and Render.

K4: How teams work effectively to produce technology solutions

Section 2 compares Waterfall, Scrum, and iterative prototyping against five criteria. Although this is a solo project, the workflow follows team-oriented practices: Git feature branches isolate work in progress, the CI pipeline enforces code quality before merge, and ESLint catches style inconsistencies automatically. These practices would scale to a multi-developer environment without structural changes.

K5: The role of data management systems in managing organisational data and information

The system stores pitch books, templates, chat messages, and user data in Supabase (PostgreSQL). Row-level security policies restrict each user to their own records. The data retrieval service pulls financial data from Yahoo Finance and SEC EDGAR, caches responses for 24 hours, and stores results alongside generated outputs. Section 4 covers the database schema and data flow across services.

K6: Common vulnerabilities in computer networks including unsecure coding and unprotected networks

Section 3 addresses security in the system design. The implementation validates uploaded files as genuine .pptx archives to prevent path traversal and zip bomb attacks. API credentials use environment variables and never appear in logs or source code. Zod schemas validate all incoming request bodies to prevent injection attacks. The Office.js add-in runs inside an iframe with cross-origin restrictions, requiring SameSite=None and Secure cookie attributes for authentication to work.

K7: The various roles, functions and activities related to technology solutions within an organisation

Section 1 describes the investment banking analyst role and how pitch book preparation fits within the broader deal workflow. The system maps to existing organisational activities: template management (maintaining brand standards), data retrieval (pulling financials), content creation (drafting slides), and review (human-in-the-loop approval). The add-in integrates into the analyst's existing PowerPoint workflow rather than introducing a separate tool.

K8: How strategic decisions are made concerning acquiring technology solutions resources and capabilities

Section 2 contains comparison tables evaluating programming languages (TypeScript vs Python vs Java), LLM providers (GPT-4o vs Claude vs Gemini), data sources (Yahoo Finance vs Bloomberg), and deployment platforms (Vercel/Render vs AWS vs self-hosted). Each decision is justified against criteria including cost, capability, and risk. The decision to build rather than buy was driven by the absence of template-aware generation in existing tools reviewed in Section 3.

K9: How to deliver a technology solutions project accurately consistent with business needs

Section 2 defines business requirements (BR1 to BR4) and maps them to functional requirements. Section 4 traces each functional requirement to its implementation status. Section 5 assesses all five objectives against measured evidence. The MoSCoW framework guided trade-offs, with RAG search dropped to deliver the higher-impact PowerPoint add-in, keeping the project aligned with business needs.

K10: The issues of quality, cost and time for projects

Section 2 presents the project plan with a Gantt chart and milestones. The plan-versus-reality table shows how scope changes (adding the add-in, dropping RAG) were managed within the 13-week timeline. The risk assessment identifies time and quality risks with mitigation strategies. Section 5 discusses the cost of GPT-4o API calls (approximately $0.03 per generation) as a resource constraint affecting scalability.

S1: Critically analyse a business domain to identify the role of information systems

Section 1 analyses the investment banking pitch book workflow and identifies manual document preparation as the bottleneck. Section 3 reviews existing tools (Gamma, Beautiful.ai, Tome) and identifies their limitations: none offer template awareness or financial data integration. Section 5 compares the system

against these tools on concrete metrics (time, fidelity, data accuracy). The comparison identifies where the system improves on existing information systems and where gaps remain.

Software Engineering Pathway

SE1: Create effective and secure software solutions

Sections: 3 Research Design, 4 Implementation
Tables: 17, 23
Figures: 4
Pages: 25, 38

Security requirements NFR6 and NFR7 drove three concrete decisions. All API credentials live in environment variables and never appear in source code. Every request body passes through Zod schema validation before reaching any internal service. Uploaded template files are checked against the Office Open XML archive format to block path traversal and zip bomb attacks. Supabase row-level security policies prevent any user from reading another's records. The PowerPoint add-in runs inside an Office iframe with cross-origin restrictions; SameSite=None and Secure cookie attributes were necessary to make session authentication work across the iframe boundary. Sections 3 and 4 document these decisions in full.

SE2: Undertake analysis and design to create artefacts

Sections: 2 Methodology, 3 Research Design
Tables: 3, 4, 16
Figures: 4, 5, 10
Pages: 13, 25

Section 2 covers requirements in two tables: 15 functional requirements (FR1–FR15) and 8 non-functional requirements (NFR1–NFR8), each with a MoSCoW priority and acceptance criteria. Section 3 covers system design, including an architecture overview, a data flow diagram, an entity-relationship diagram, and component-level interface definitions. All schemas and API contracts were documented before any implementation work started — Swagger captures the REST API specification and Zod validates the content plan schema at runtime. Four architecture decisions are formalised as ADRs in Appendix B, each with a context statement, the decision taken, the rationale, and its consequences.

SE3: Produce high quality code with sound syntax

Sections: 4 Implementation
Tables: 18
Pages: 38

TypeScript runs in strict mode across all three applications, with strict null checks and no implicit any. The shared-types package defines every data interface once and shares it across the backend, web frontend, and PowerPoint add-in. A schema change in one place triggers compiler errors everywhere else that

references it, so breaking changes cannot be silently introduced. ESLint catches common errors (unused variables, missing imports, type inconsistencies) at the lint stage before any code reaches CI. Prettier handles formatting automatically, removing it as a review concern entirely. All three applications share a single tsconfig base through the monorepo, so compiler rules are identical across the codebase. Table 18 shows the workspace structure and package dependencies.

SE4: Perform code reviews, debugging and refactoring

Sections: 4 Implementation
Tables: 22
Pages: 38

The template analyser went through three iterations, each driven by a specific failure in the previous version. The first attempt used PptxGenJS for reading templates but produced incorrect placeholder positions. The second used raw XML parsing which resolved positions correctly but missed custom placeholder types. The third added position-based heuristics that classify placeholders by size and location when the XML type attribute is absent; this reached 93% classification accuracy. A colour resolution bug — theme references passing through as the default black instead of resolving to hex values — was found through debugging a generated output that should have used a gold accent colour. The fix added a theme resolver that walks the XML colour inheritance chain. Table 22 documents this and seven other implementation challenges with their root causes and resolutions. The CI pipeline caught 14 breaking changes before they reached the main branch. Section 4 covers these in detail.

SE5: Test code to ensure requirements met

Sections: 4 Implementation, 5 Evaluation
Tables: 19, 20
Figures: 20, 22
Pages: 38, 58

The project has 75 unit and integration tests across 14 Vitest test suites, plus 28 Cypress E2E tests across 7 spec files. All 103 pass. Code coverage sits at 76% for the backend service, 64% for the web app, and 58% for the PowerPoint add-in. Unit tests mock external dependencies (OpenAI, Supabase, Office.js) using `vi.mock()` so they run without API keys or a live database connection. Integration tests for the orchestrator verify that the full service chain executes in the correct order and returns a valid pitch book object. E2E tests run against a real Supabase test environment with a dedicated test user account, covering login, pitch book creation, template upload, and settings flows. Table 19 shows coverage by application and Figure 20 shows the Vitest UI output. Sections 4 and 5 cover the full test strategy and results.

SE6: Deliver software using industry standard build processes

Sections: 4 Implementation
Tables: 21
Figures: 24, 25, 26
Pages: 38

GitHub Actions runs the full CI/CD pipeline on every push to any branch with six sequential stages: dependency install, ESLint linting, TypeScript type checking, Vitest unit tests, Turborepo build, and Cypress E2E tests. A failing stage blocks all subsequent stages, so broken code never reaches the build or deployment steps. Turborepo handles monorepo build orchestration, compiling the shared-types package before the three applications and caching build outputs to speed up repeated runs. Mochawesome generates structured HTML test reports from Cypress results, stored as CI artefacts. The Next.js web app deploys to Vercel with automatic preview deployments on every pull request. The Express backend deploys to Render with zero-downtime rolling deploys. Table 21 lists each pipeline stage with its tool and purpose. Figures 24, 25, and 26 show the pipeline and deployment dashboards. Section 4 documents the full pipeline configuration.

SE7: Operate at all stages of the software development lifecycle

Sections: 2 Methodology, 3 Research Design, 4 Implementation, 5 Evaluation
Tables: 3, 23
Figures: 2
Pages: 13, 25, 38, 58

The project worked through every SDLC stage in sequence. Requirements and methodology are in Section 2. System design — architecture, data models, and API specifications — is in Section 3. Section 4 covers the build across all three applications and the full test suite. Section 5 measures outcomes against five predefined success criteria with quantitative evidence. Each two-week iteration produced working software. Retrospective reviews fed lessons directly into the next cycle; it was one of those reviews that surfaced the web-only friction problem and led to adding the PowerPoint add-in in cycle three. Table 3 shows all 15 requirements set at the start; Table 23 shows that 13 of 15 were fully delivered.

SE8: How teams work effectively using agile and other approaches

Sections: 2 Methodology
Tables: 2, 11
Figures: 2, 3
Pages: 13

Section 2 compares Waterfall, Scrum, and iterative prototyping and justifies the choice of iterative prototyping. Scrum's sprint ceremonies add overhead that makes no sense for a solo developer. Waterfall's fixed planning cannot accommodate LLM integration, where output quality is only known after testing against real data. Two-week iterations with retrospective reviews at each cycle's end caught the web-only workflow problem early enough to pivot to the PowerPoint add-in in cycle three, a decision that would have been impossible under a Waterfall plan. GitHub Issues with a Kanban board (Figure 3) tracked work items across backlog, in-progress, and done columns. Git feature branches kept in-progress work isolated from the main branch, with CI running on every push to catch regressions before merge. Table 11 documents planned versus actual scope across all six iterations.

SE9: How to apply software analysis and design approaches

Sections: 2 Methodology, 3 Research Design
Tables: 3, 4, 17
Figures: 4
Pages: 13, 25

MoSCoW prioritisation categorised all 15 functional requirements as Must, Should, or Could before implementation began. This framework directly informed the mid-project decision to drop RAG search (FR15, a Could requirement) to make room for the PowerPoint add-in, which addressed a Must-level usability gap identified during iteration two. Business requirements (BR1 to BR4), functional requirements (FR1 to FR15), and non-functional requirements (NFR1 to NFR8) are separated following S. Robertson and J. Robertson (2012), with each layer traced to the one above. Four major architecture decisions — language choice, LLM provider, monorepo structure, and the dual slide builder approach — are documented with trade-off analysis in Table 17 and formalised with full context, rationale, and consequences as ADRs in Appendix B. The system architecture diagram (Figure 4) maps the resulting component boundaries and service interfaces. Sections 2 and 3 cover requirements elicitation and system design respectively.

SE10: Interpret and implement a design compliant with requirements

Sections: 1 Introduction, 4 Implementation, 5 Evaluation
Tables: 1, 23, 27
Pages: 8, 38, 58

The requirements traceability table (Table 23) maps all 15 functional requirements to their implementation status with specific evidence for each. 13 of 15 are fully met. FR12 (chart generation) is partial: placeholder positions appear in the content plan, but charts are not yet rendered from scratch. FR5 (SEC EDGAR) is also partial: the EDGAR client fetches filing metadata, but that data is not yet passed into the GPT-4o prompt. FR15 (RAG search) was dropped to make room for the PowerPoint add-in. The gap-to-objective traceability table (Table 1) established in Section 1 is closed in Section 5: every identified gap maps to an objective, and every objective maps to a measured result. Table 27 in Section 5 shows each objective's success criterion alongside the measured evidence from the 12 evaluation runs.

SE11: How to perform functional and unit testing

Sections: 4 Implementation
Tables: 19, 20
Figures: 20, 21, 22
Pages: 38

Vitest handles unit and integration tests with mocked dependencies using `vi.mock()` and `vi.stubGlobal()`. Service tests mock the OpenAI and Supabase clients so tests run without external connections or API spend. Add-in tests mock the full Office.js runtime, including `PowerPoint.run()` and the slide insertion API, so the add-in logic can be tested independently of PowerPoint. Cypress handles functional E2E tests covering full user flows: login, registration, pitch book creation with and without templates, the slide viewer, template management, and settings. Each Cypress spec file maps to a discrete feature area;

Table 20 lists all seven spec files with their scope and pass status. A comparison table in Section 4 justifies choosing Cypress over Playwright on setup time, debugging tools (time-travel screenshots), and Mochawesome reporting integration. Figures 20, 21, and 22 show the test runner outputs.

SE12: How to use and apply software tools used in software engineering

Sections: 2 Methodology, 4 Implementation
Tables: 5, 6, 7, 8
Pages: 13, 38

Every phase of development had a dedicated tool. Version control ran through Git and GitHub with feature branching. GitHub Actions and Turborepo handled the CI/CD pipeline and monorepo build orchestration. Testing split across Vitest for unit and integration tests and Cypress for E2E tests, with Mochawesome generating HTML reports. ESLint, Prettier, and the TypeScript compiler enforced code quality without manual review overhead. PowerPoint generation used PptxGenJS in default mode and JSZip with xml2js for XML-level template parsing. LibreOffice headless and pdftoppm handled slide preview rendering. Supabase, Vercel, and Render covered storage, frontend hosting, and backend hosting. Each major technology choice is justified in Section 2 with a scored comparison table covering the alternatives considered (Tables 5, 6, 7, 8).

Appendix B: Architecture Decision Records

ADR-001: TypeScript Over Python

Context. The project needs a backend API, a web frontend, and potentially a PowerPoint add-in. Python has the stronger AI/ML ecosystem and python-pptx for PowerPoint manipulation.

Decision. Use TypeScript across all applications.

Rationale. Full-stack coherence outweighs Python's library advantage. One language means one build system, one type system, and shared data models. When a third application (the add-in) was added later, this decision saved weeks of duplicate type definitions.

Consequences. PptxGenJS replaces python-pptx. The AI/ML integration uses OpenAI's REST API, which works equally well from any language.

ADR-002: GPT-4o for Content Planning

Context. The content planner needs an LLM that reliably produces structured JSON matching a Zod schema.

Decision. Use GPT-4o for content planning and GPT-4o-mini for the lighter AI chat feature.

Rationale. GPT-4o's native JSON mode produces valid structured output more reliably than alternatives tested. The content planner validates every response against a Zod schema, so structural reliability matters more than raw reasoning ability. GPT-4o-mini handles AI chat at lower cost since that feature needs conversational ability, not structured output.

Consequences. Vendor dependency on OpenAI. Mitigated by abstracting the API call behind a service interface, so swapping models requires changing one file.

ADR-003: Monorepo with Turborepo

Context. Three applications share data types and configuration. They could live in separate repositories or a single monorepo.

Decision. Use a Turborepo monorepo with a shared-types package.

Rationale. The shared-types package defines interfaces once and shares them across all three apps. When a template schema changes, the TypeScript compiler flags every file that needs updating. Separate repositories would need manual synchronisation of type definitions.

Consequences. Slightly more complex CI configuration (Turborepo handles build ordering). Single repository means all apps share the same Git history.

ADR-004: Dual Slide Builder Approach

Context. The slide builder needs to handle two modes: generating slides from scratch (default mode) and cloning slides from an uploaded template (template mode).

Decision. Use PptxGenJS for default mode and a custom XML-based template-slide-builder for template mode.

Rationale. PptxGenJS cannot clone existing slides from a template file. Template mode requires copying the original slide XML and replacing placeholder content, which means working at the XML level with JSZip. Keeping both builders allows each to do what it does best.

Consequences. Two code paths to maintain. Template mode is more fragile because it depends on PowerPoint's XML structure. The template-slide-builder handles fewer edge cases than PptxGenJS.

Appendix C: Evaluation Run Data

Twelve generation runs were conducted between 17 and 21 March 2026 using the system's built-in deck layouts and colour themes (default mode, no uploaded template). All runs used Git commit `a9e8bb0`, GPT-4o with temperature zero, and Node.js 22.14. Table 30 shows the inputs and Table 31 shows the measured results for each run.

Run	Company (Ticker)	Deck Layout	Colour Theme	Date
1	Apple Inc. (AAPL)	Company Overview	Navy & Gold	17 Mar
2	AstraZeneca (AZN)	Company Overview	Midnight Blue	17 Mar
3	Shell plc (SHEL)	Market Update	Teal & Coral	18 Mar
4	HSBC Holdings (HSBA)	Transaction Summary	Slate & Emerald	18 Mar
5	Unilever (ULVR)	Investor Pitch	Purple & Gold	19 Mar
6	Microsoft Corp. (MSFT)	Industry Overview	Charcoal & Red	19 Mar
7	BP plc (BP)	Transaction Summary	Navy & Gold	20 Mar
8	GSK plc (GSK)	Fundraising Deck	Forest & Cream	20 Mar
9	Barclays (BARC)	Market Update	Monochrome	20 Mar
10	Rio Tinto (RIO)	Due Diligence	Charcoal & Red	21 Mar
11	Apple Inc. (AAPL)	Company Overview	Navy & Gold	21 Mar
12	AstraZeneca (AZN)	Company Overview	Midnight Blue	21 Mar

Table 30: Evaluation Run Inputs. Runs 11 and 12 repeat runs 1 and 2 to check consistency. All seven deck layouts and six of eight colour themes were tested.

Run	Time (s)	Slides	Colour	Font	Zod Valid
1	43	11	Exact	Exact	Yes
2	51	11	Exact	Exact	Yes
3	38	10	Exact	Exact	Yes
4	62	10	Exact	Exact	Yes (unwrapped)
5	47	11	Exact	Exact	Yes
6	39	11	Exact	Exact	Yes
7	78	10	Exact	Exact	Yes (unwrapped)
8	44	11	Exact	Exact	Yes
9	52	10	Exact	Exact	Yes
10	32	12	Exact	Exact	Yes
11	41	11	Exact	Exact	Yes
12	48	11	Exact	Exact	Yes

Table 31: Evaluation Run Results. Colour = hex match against template theme. Font = family and size match. Zod Valid = whether the LLM output passed schema validation ("unwrapped" means the response was nested inside a wrapper object that the unwrapping logic handled before validation).

Run 7 (BP, Transaction Summary, Navy & Gold) was the slowest at 78 seconds. The GPT-4o API response took longer than usual for this run, and the content planner's unwrapping logic was triggered because the

model returned the plan nested inside a `{"pitch_book": {...}}` wrapper. Run 4 (HSBC, Transaction Summary, Slate & Emerald) also triggered the unwrapping step. All 12 runs produced valid JSON after the unwrapping step, with no runs falling back to the generic plan.

Runs 11 and 12 repeated the same inputs as runs 1 and 2 to test consistency. Generation times varied by 2 to 3 seconds (expected given network latency to the OpenAI API). Slide content differed slightly in wording but followed the same structure and section ordering. Colour and font results were identical across repeated runs.

Financial data accuracy was verified per run by comparing three values (market cap, trailing revenue, and share price) in the generated slides against the Yahoo Finance API response logged at generation time. All 36 comparisons (3 values $\times$ 12 runs) matched exactly.

Sample data verification (Run 1, Apple Inc.):

Metric	Yahoo Finance API	Generated Slide
Market Cap	$3.65T	$3.65T
Trailing Revenue (TTM)	$435.6B	$435.6B
Share Price (close)	$248.00	$248.00

Sample data verification (Run 5, Unilever):

Metric	Yahoo Finance API	Generated Slide
Market Cap	$100.4B	$100.4B
Trailing Revenue (TTM)	$50.5B	$50.5B
Share Price (close)	$4,595	$4,595

Appendix D: Usability Evaluation Questionnaire and Responses

Four junior analysts at a London-based financial services firm participated in an informal usability evaluation during March 2026. Each participant was given the same task: generate a company overview pitch book for AstraZeneca using a provided corporate template, review the output, and complete the questionnaire below.

Participants:

- P1: First-year analyst, equity capital markets, 8 months' experience
- P2: Second-year analyst, M&A advisory, 18 months' experience
- P3: First-year analyst, leveraged finance, 10 months' experience
- P4: Second-year analyst, equity capital markets, 20 months' experience

Part A: Quantitative (Likert 1 to 5, where 1 = Strongly Disagree, 5 = Strongly Agree)

Statement	P1	P2	P3	P4
Q1. The generated output matched the template's visual style (colours, fonts, layout)	5	4	4	5
Q2. The financial data included was accurate and well-positioned on the slides	4	4	5	4
Q3. The output required less editing than starting a deck from scratch	5	5	4	5
Q4. The add-in workflow (generating slides inside PowerPoint) was intuitive	4	3	3	5
Q5. The slide ordering and narrative structure followed IB conventions	4	4	4	4
Q6. The system would save meaningful time in a real work scenario	5	4	4	5

Part B: Open-Ended Responses

Q7. What was the most useful aspect of the system?

- P1: 'The template matching. The colours and fonts are exactly right. Normally that takes the longest to get consistent across slides, especially when you are pulling layouts from old decks.'
- P2: 'Having the financials already in there. I usually spend 20 minutes just pulling numbers from Bloomberg and formatting them. This had everything from Yahoo Finance already placed.'
- P3: 'Speed. Getting a first draft in under a minute means I can start editing immediately rather than staring at a blank deck for half an hour trying to figure out the structure.'
- P4: 'The fact that it works inside PowerPoint. Every other AI tool I have tried makes you export and import files. This just puts the slides straight in.'

Q8. What would you change or improve?

- P1: ‘I want to regenerate a single slide without redoing the whole deck. Sometimes one slide is off but the rest are fine.’
- P2: ‘The taskpane felt a bit cramped on my laptop screen. Would be better if I could resize it or pop it out into a separate window.’
- P3: ‘It would be good to choose which sections to include. For some clients we skip the market overview entirely.’
- P4: ‘The status indicator during generation was not obvious. I was not sure if it was working or stuck. A progress bar would help.’

Q9. Estimated time to create a similar deck manually vs using the system?

- P1: ‘Manual: about two hours. With the system: maybe 20 minutes total including editing.’
- P2: ‘Manual: 90 minutes if I have a good reference deck. System: 15 minutes of editing after generation.’
- P3: ‘Manual: two hours minimum. System: generation was instant, then about 25 minutes of cleanup.’
- P4: ‘Manual: depends on the deck, but at least 90 minutes. System: under 20 minutes including review.’

Q10. Would you use this system in your daily work? Why or why not?

- P1: ‘Yes. Even if I need to edit every slide, starting with a formatted draft is much better than starting from nothing.’
- P2: ‘Yes, for first drafts and for decks where the structure is standard. For bespoke pitches I would probably still start manually.’
- P3: ‘Yes. The time saving is real. Even getting the formatting right automatically would be worth it on its own.’
- P4: ‘Definitely. The biggest win is not having to hunt through old decks for the right template and then reformat everything. This does both in seconds.’

Appendix E: Web Application Screenshots

Screenshots of every page in the PitchDeck AI web application, captured from a local deployment with a test account.

Authentication Pages

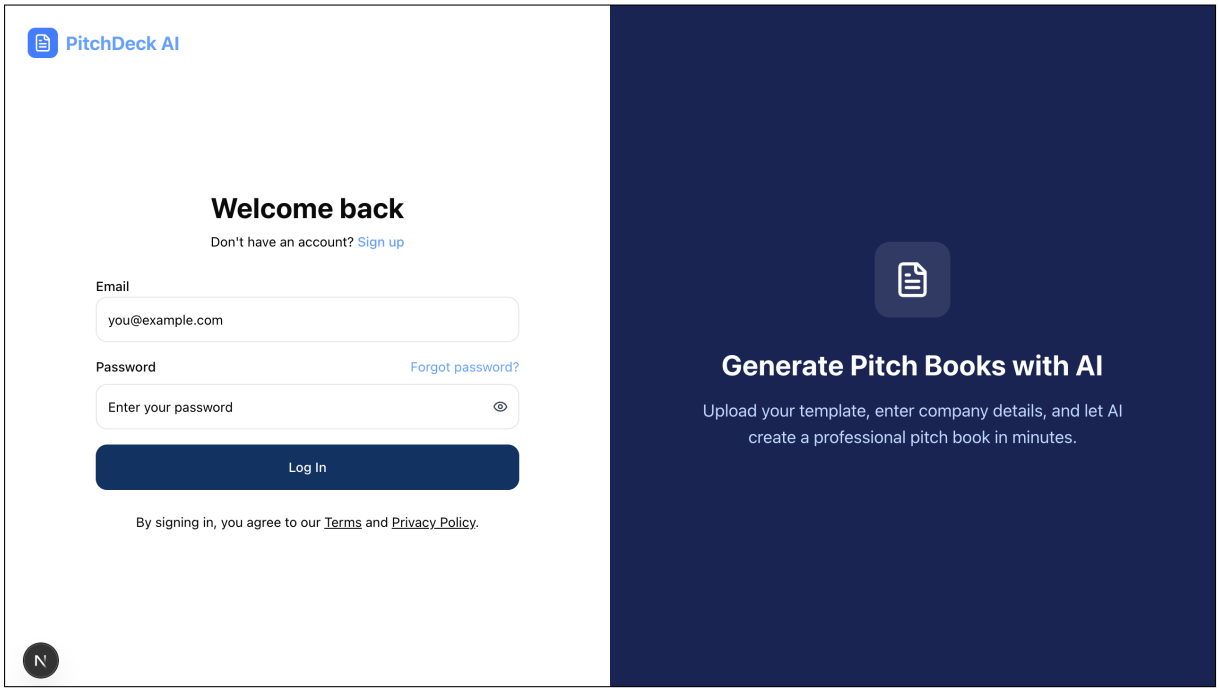

Figure 29: Login page with email and password form alongside application branding.

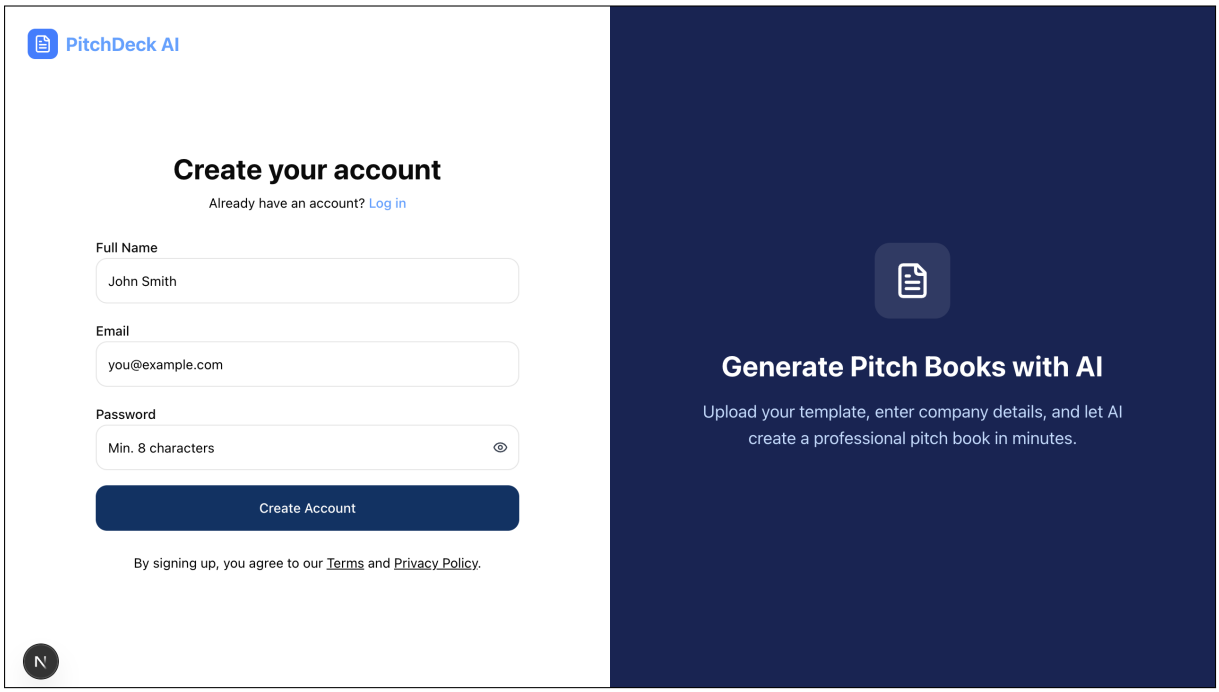

Figure 30: Registration page with name, email, and password fields.

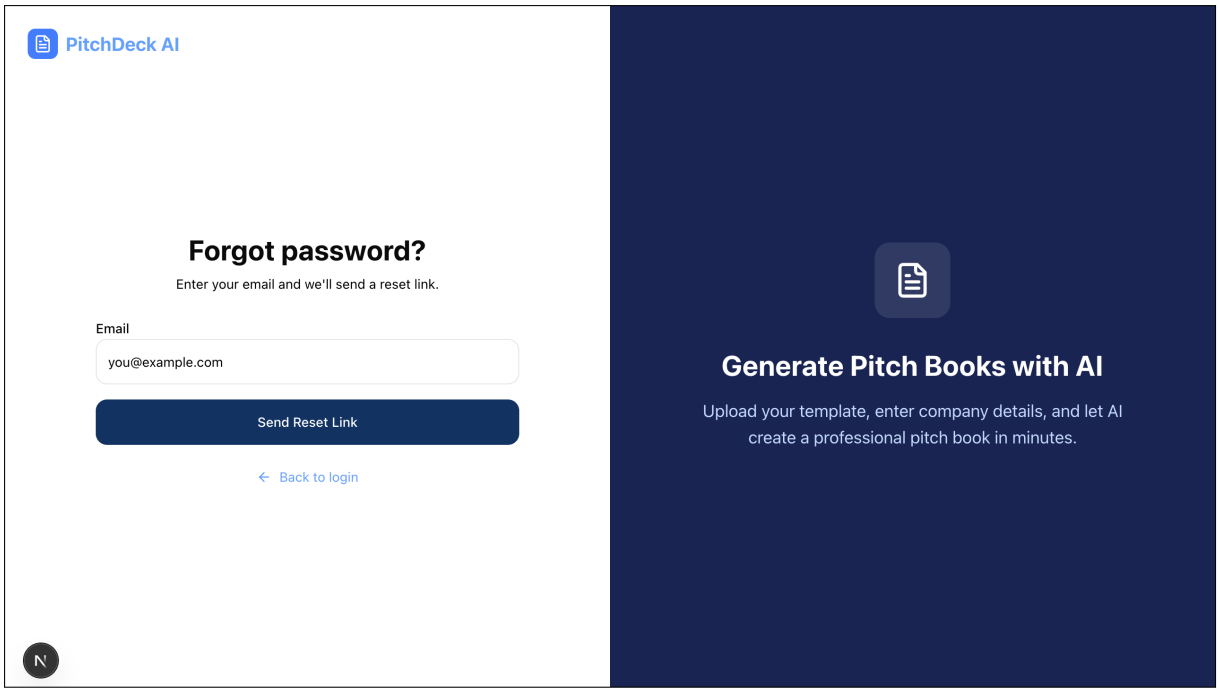

Figure 31: Forgot password page. Sends a reset link via Supabase Auth.

Dashboard

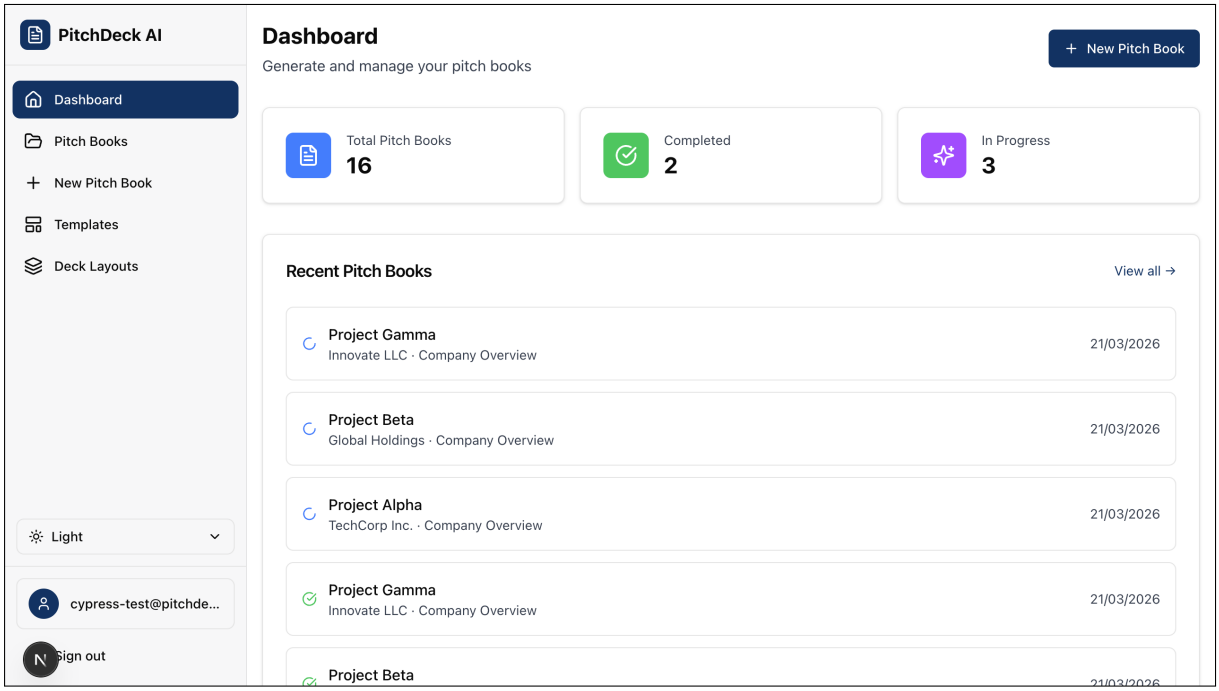

Figure 32: Dashboard with summary cards, recent pitch books, and generation status indicators.

Pitch Books

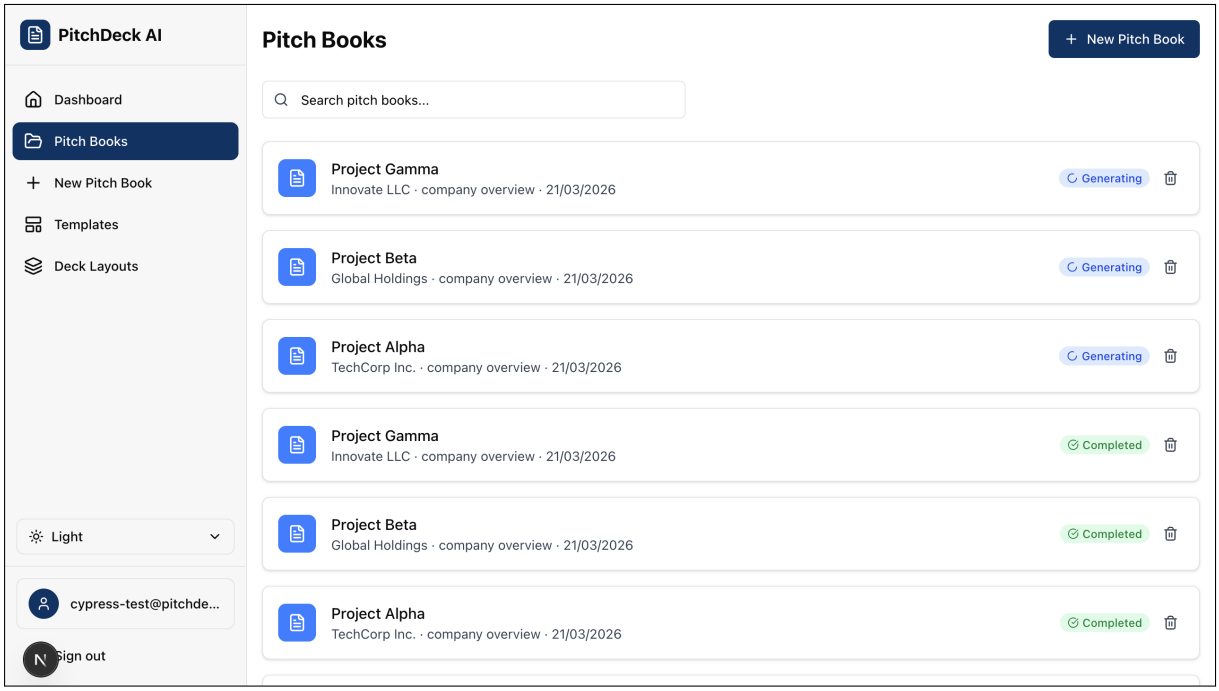

Figure 33: Pitch books list with company name, type, date, and generation status. Includes search filtering.

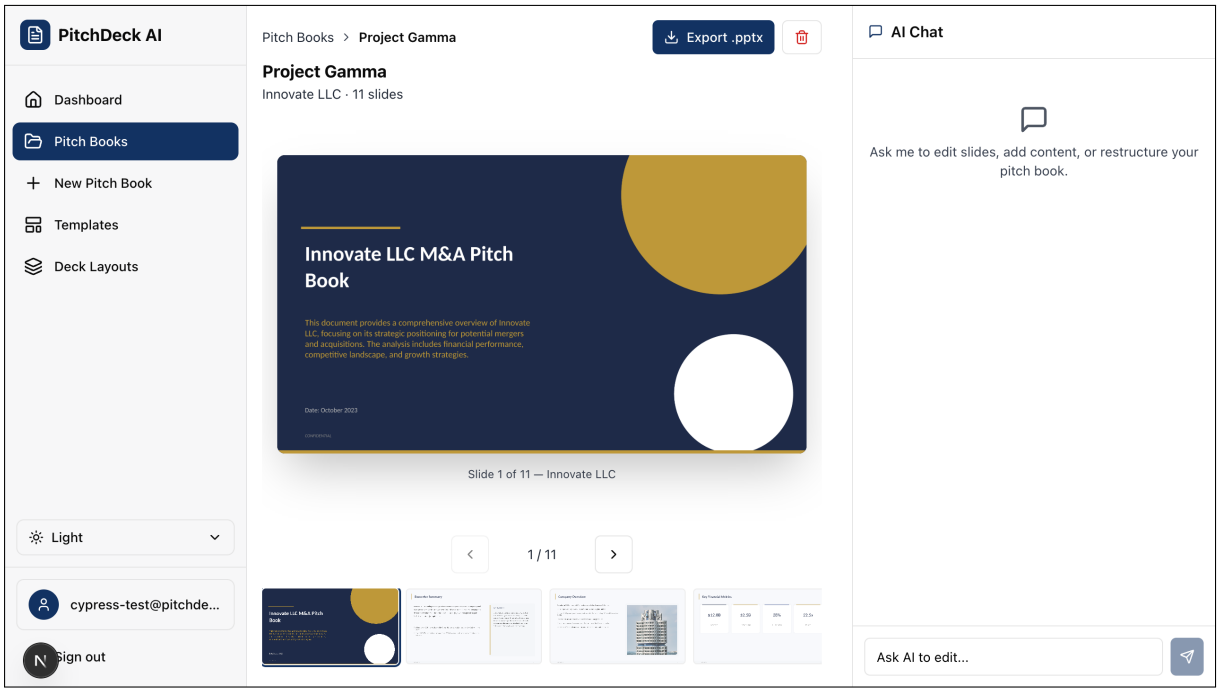

Figure 34: Pitch book viewer with full-size slide, thumbnail strip, and AI chat panel for requesting edits.

Pitch Book Creation

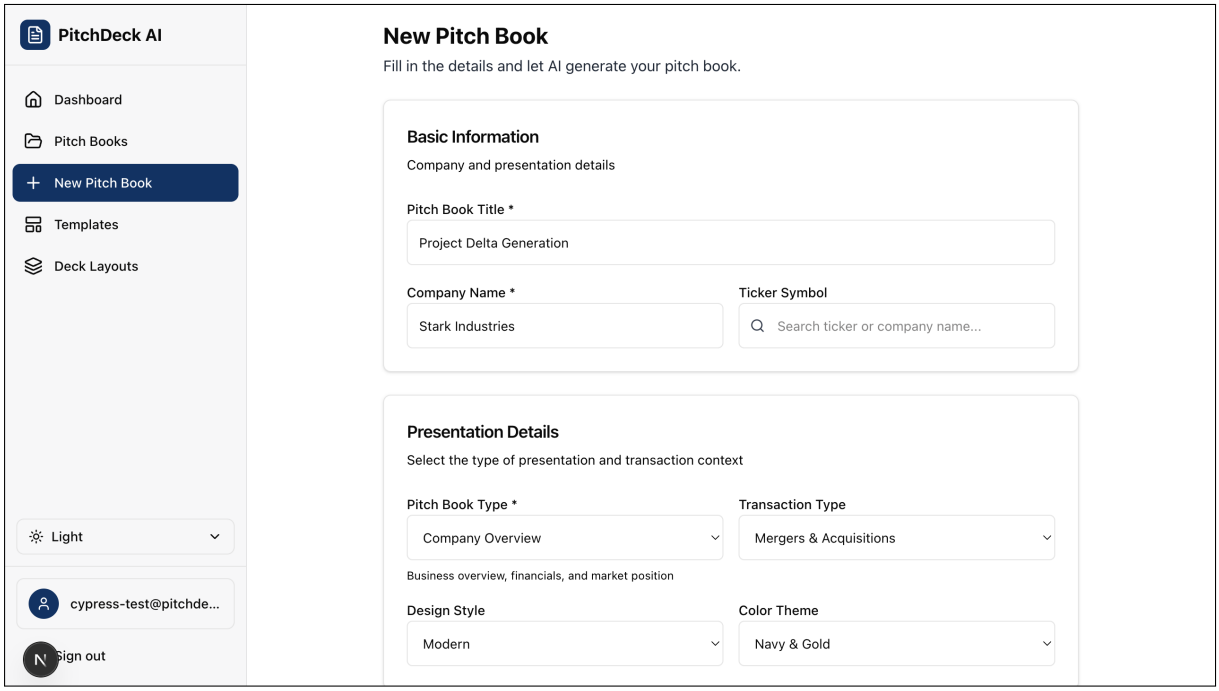

Figure 35: New pitch book form with company details, pitch book type, transaction context, and design options.

Templates

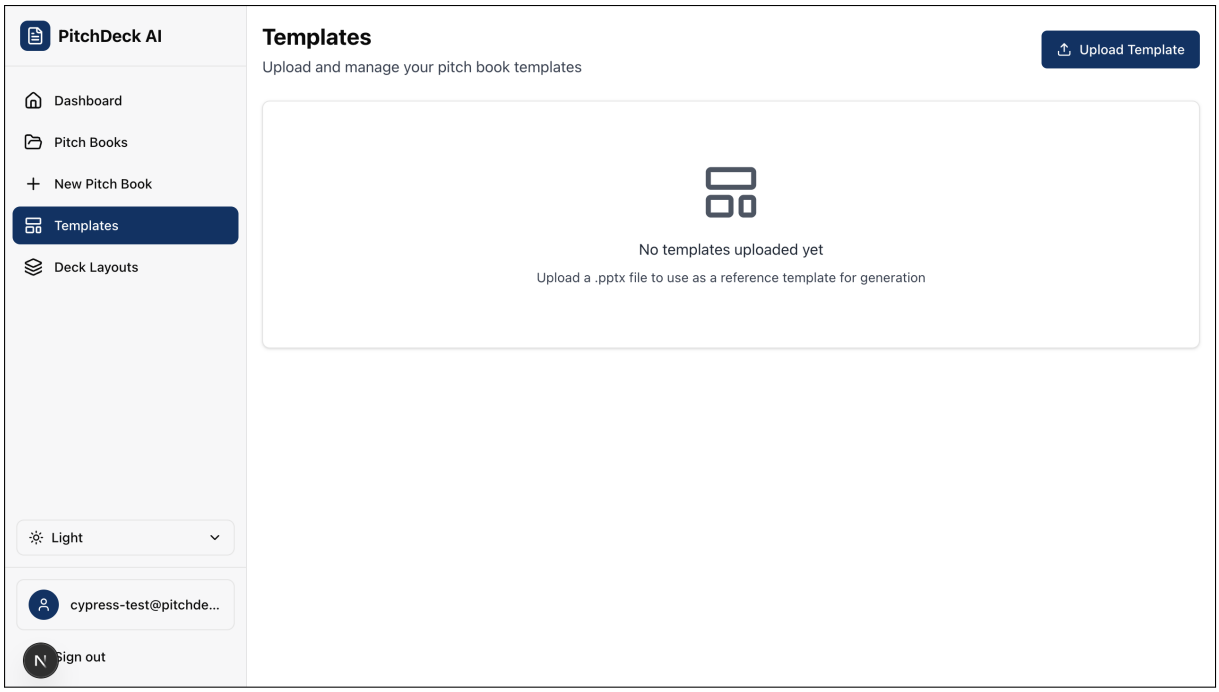

Figure 36: Templates page (empty state) with upload prompt for .pptx reference templates.

Deck Layouts

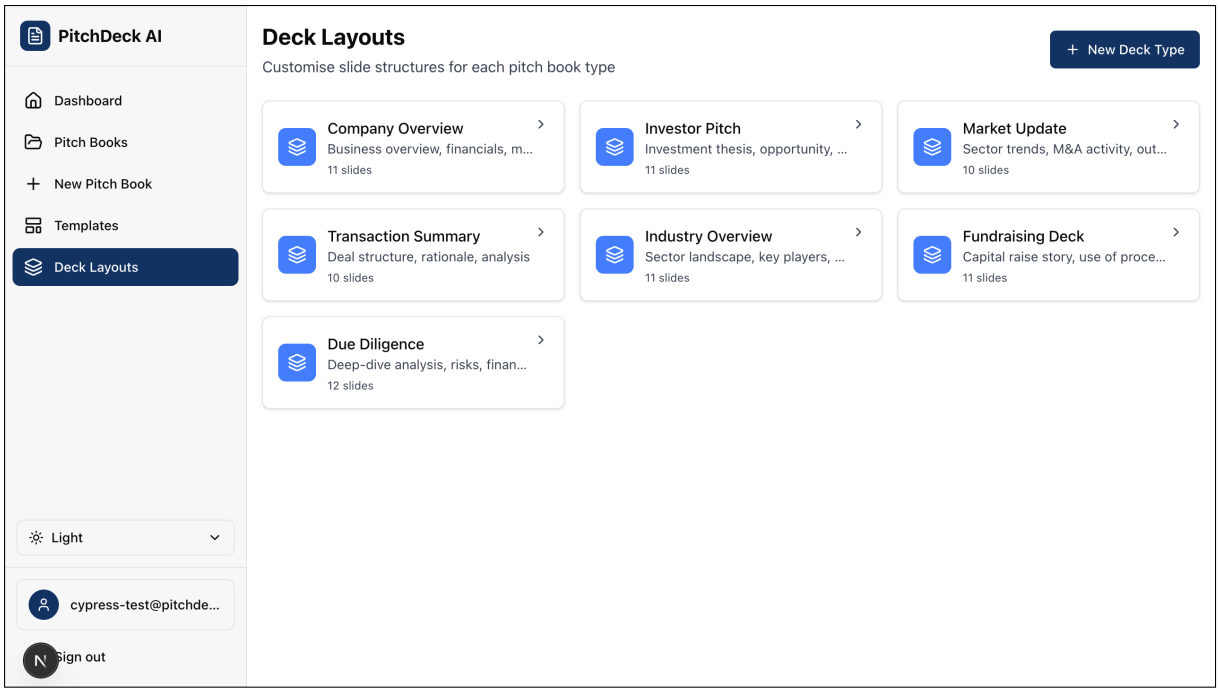

Figure 37: Deck layouts overview with seven pre-configured deck types, each showing a description and slide count.

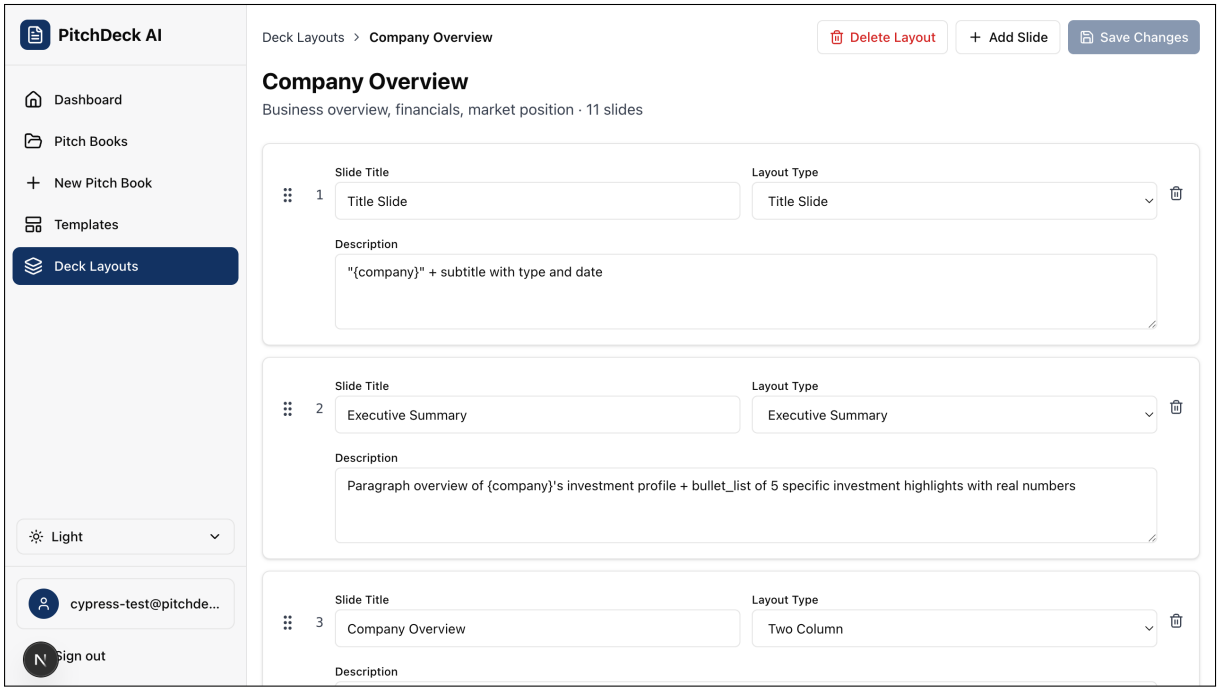

Figure 38: Deck layout editor for Company Overview. Each slide has a title, layout type, and content description. Slides can be reordered, added, or deleted.

Settings

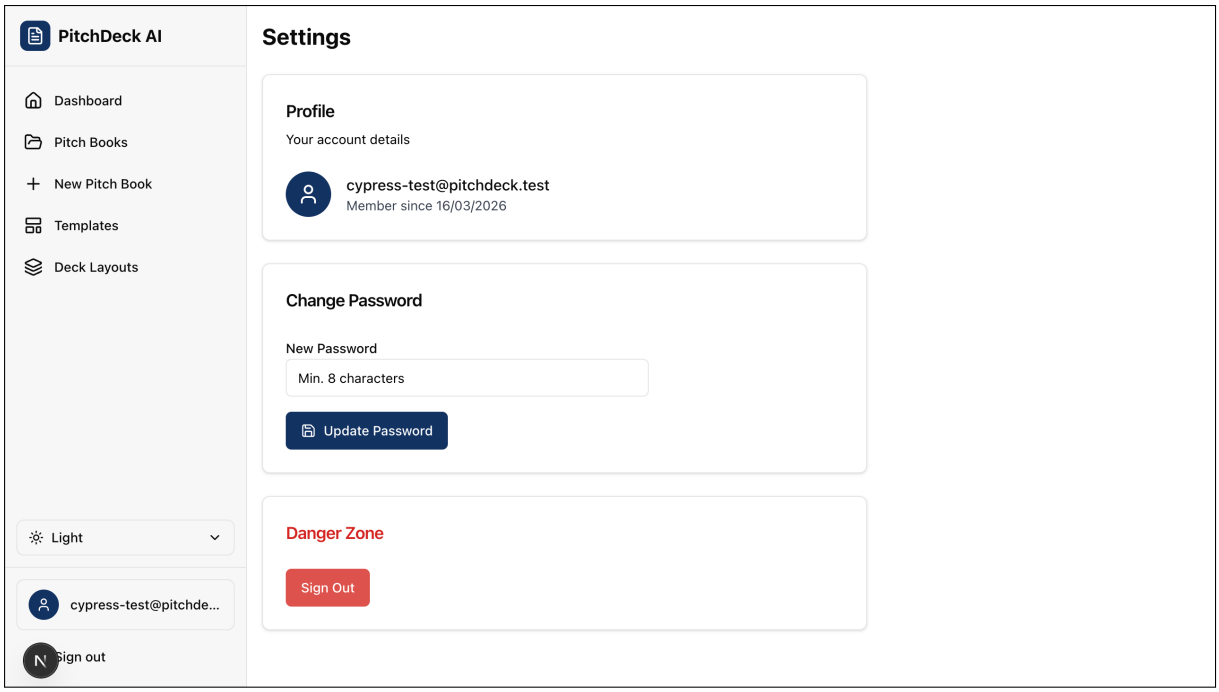

Figure 39: Settings page with profile details, password change, and sign-out.

Appendix F: PowerPoint Add-in Screenshots

Screenshots of the PowerPoint taskpane add-in running inside Microsoft PowerPoint on macOS. The add-in connects to the same backend as the web application.

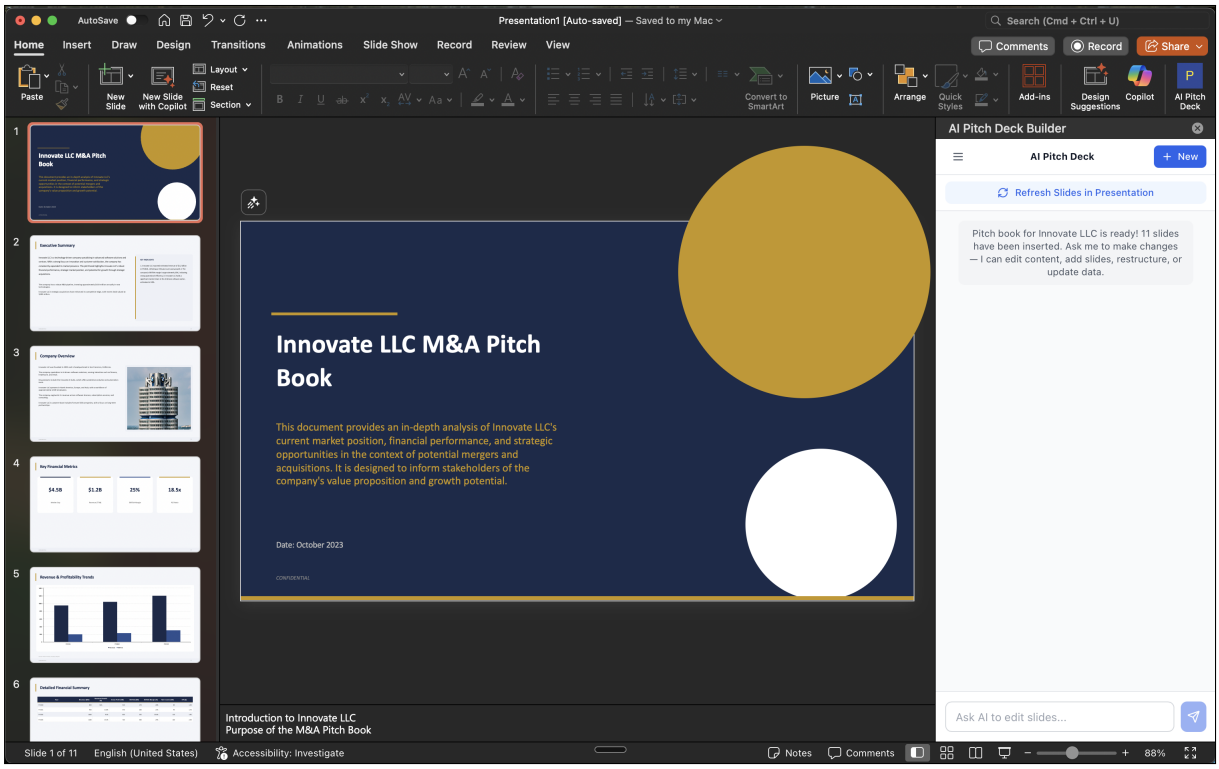

Figure 40: Add-in after generation. 11 slides have been inserted into the presentation. The taskpane shows a completion message and an AI chat input for requesting edits.

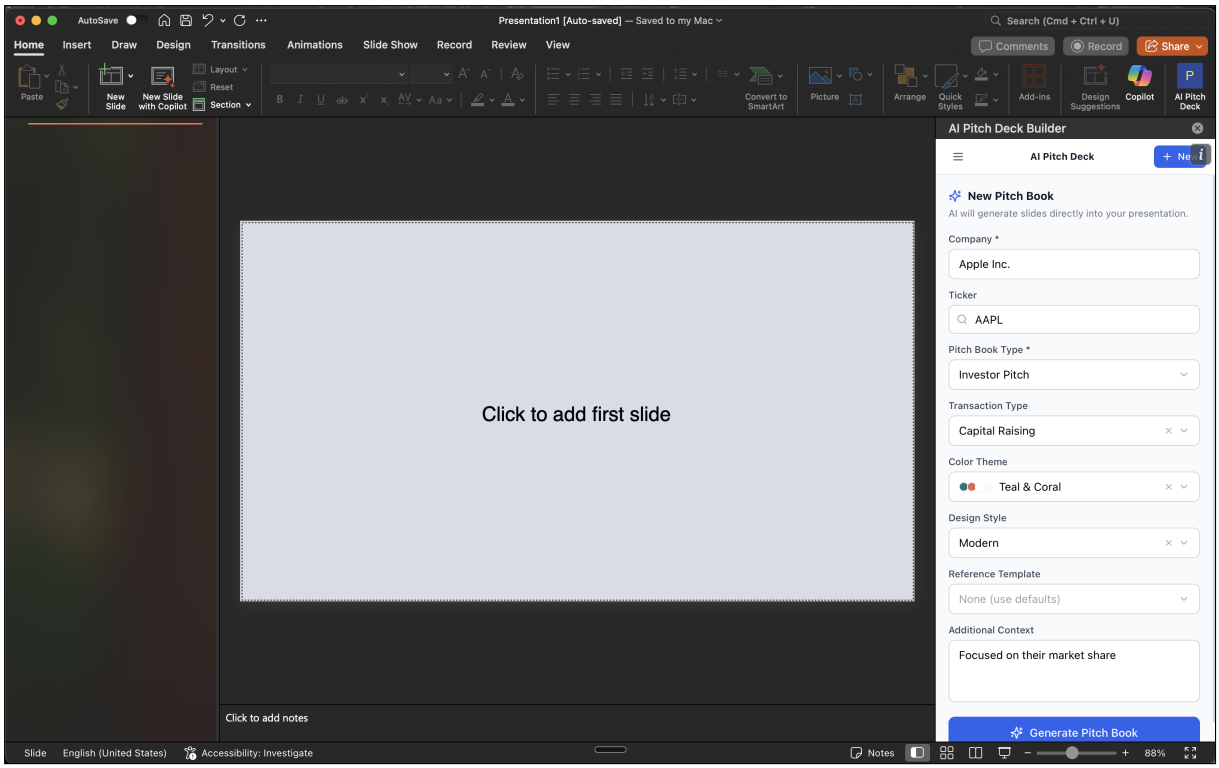

Figure 41: New pitch book form in the add-in. The user enters company name, ticker, pitch book type, transaction type, colour theme, design style, and optional context before generating.

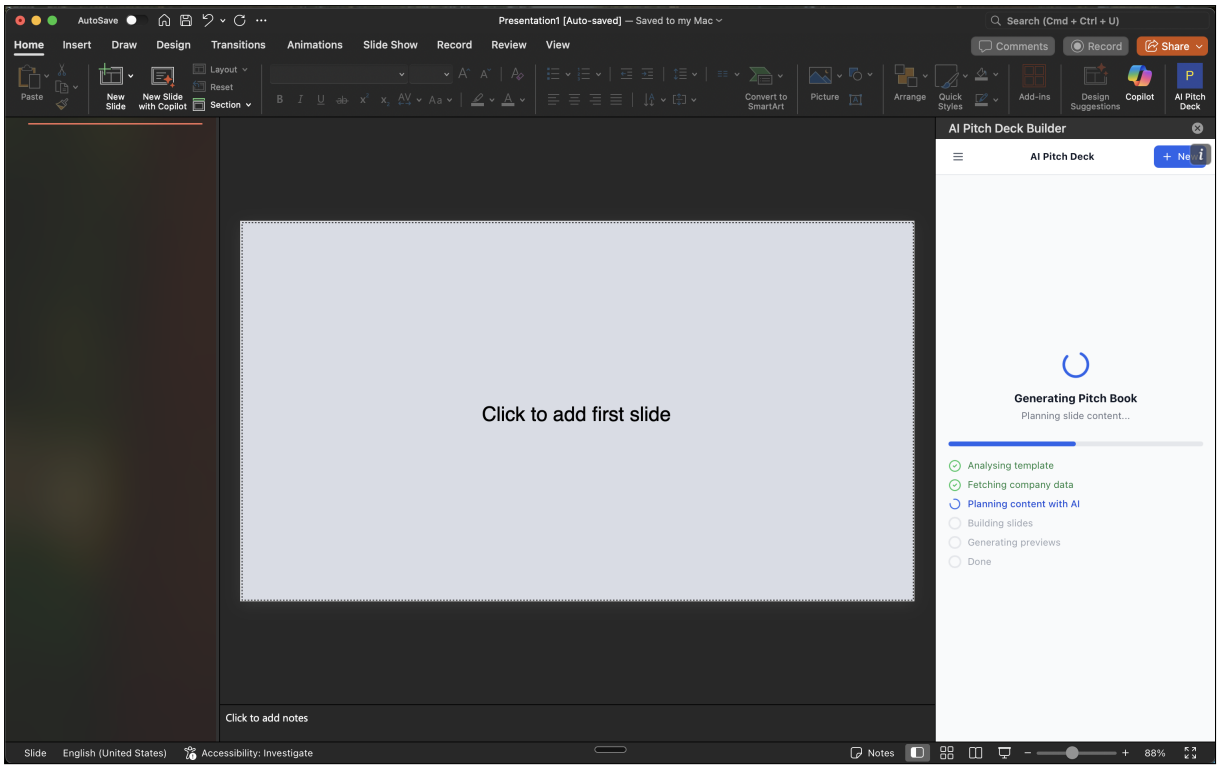

Figure 42: Generation in progress. The taskpane shows a step-by-step progress indicator: analysing template, fetching company data, planning content with AI, building slides, and generating previews.

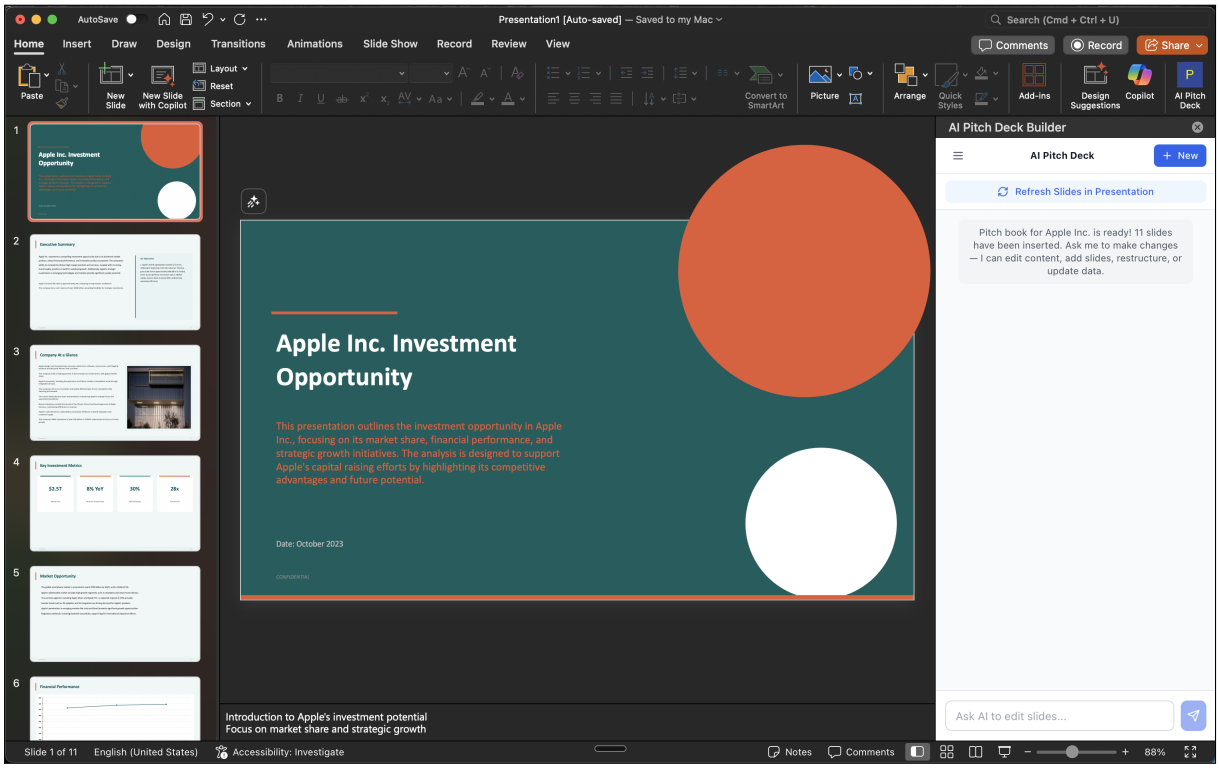

Figure 43: Completed generation for Apple Inc. The teal and coral colour theme was applied automatically. Slides appear in the left panel and the AI chat is ready for edit requests.

Appendix G: Binary Risk Analysis

This appendix shows the full Binary Risk Analysis (BRA) workings for the ten risks listed in Section 2. Sapiro (2011) designed BRA to replace subjective likelihood and impact ratings with a repeatable process. Each risk is scored against ten yes/no questions, and the answers pass through a fixed set of lookup matrices to produce a Low, Medium, or High classification.

Risk Register

Table 33 repeats the risk descriptions from the main report for reference.

ID	Risk	Description
R1	Pitch Book Data Breach	A flaw in Supabase row-level security policies could expose one user's pitch books to another. Pitch books often contain deal terms, valuations, and financial projections that are highly confidential.
R2	Financial Figure Hallucination	GPT-4o could insert fabricated market caps, revenue figures, or growth rates into slides. An analyst presenting incorrect numbers could damage the firm's credibility.
R3	Malicious Template Upload	The template analyser parses .pptx files (ZIP archives with XML) using JSZip and xml2js. A crafted file could attempt zip bomb decompression, path traversal, or oversized XML payloads.
R4	Yahoo Finance Endpoint Change	yahoo-finance2 scrapes unofficial endpoints. Yahoo has changed these before without notice, which would break financial data retrieval mid-generation.
R5	OpenAI Credential Exposure	The Express backend stores the OpenAI API key on Render. If the key leaks through logs or a deployment compromise, an attacker could run up API costs.
R6	Content Plan Schema Failure	GPT-4o occasionally returns malformed JSON or nests output in unexpected wrappers. If Zod validation and unwrapping both fail, generation stops entirely.
R7	Slide Preview Exposure	LibreOffice converts PPTX files to PNG previews in Supabase Storage. If the bucket defaults to public, anyone with a URL could view confidential slides.
R8	Platform Dependency Outage	Generation chains Vercel, Render, Supabase, and OpenAI. A failure at any point blocks the pipeline with no self-hosted fallback.
R9	Scope Creep Beyond Timeline	The fixed 13-week timeline means unplanned features risk delaying core deliverables.
R10	Add-in Session Breakage	The Office.js iframe uses `SameSite=None; Secure` cookies for authentication. A browser or Office update could silently break login.

Table 33: Risk register for Binary Risk Analysis.

BRA Worksheet

Each risk was scored against the ten standard BRA questions (listed at the end of this appendix). Questions 1 through 6 feed into the likelihood calculation and questions 7 through 10 feed into the impact calculation. Table 34 shows the raw answers and the computed results.

	Likelihood						Impact				Result		
ID	**Q1**	**Q2**	**Q3**	**Q4**	**Q5**	**Q6**	**Q7**	**Q8**	**Q9**	**Q10**	**L**	**I**	**O**
R1	Y	N	N	N	N	N	Y	Y	Y	Y	Low	High	Med
R2	Y	Y	N	N	N	Y	Y	N	Y	N	Med	Med	Med
R3	Y	N	N	N	N	N	Y	N	N	N	Low	Low	Low
R4	Y	Y	N	Y	Y	Y	Y	N	Y	N	High	Med	High
R5	Y	N	N	N	N	N	Y	N	N	N	Low	Low	Low
R6	Y	Y	N	N	N	Y	Y	N	Y	N	Med	Med	Med
R7	Y	N	N	N	N	N	Y	Y	Y	N	Low	High	Med
R8	Y	Y	N	N	Y	Y	Y	Y	Y	N	High	High	High
R9	Y	Y	N	N	Y	N	Y	N	Y	N	Med	Med	Med
R10	Y	Y	N	Y	N	N	Y	N	N	N	Med	Low	Med

Table 34: BRA worksheet. L = Likelihood, I = Impact, O = Overall. Med = Medium.

How the Matrices Work

The ten binary answers do not simply get counted. Instead, they pass through a chain of lookup matrices that combine pairs of inputs into intermediate classifications, which then combine again until a final rating emerges.

Pair-to-pair mappings (2×2)

When two binary answers are paired, the result follows a simple rule:

- Both No → Low
- One Yes, one No (either order) → Medium
- Both Yes → High

Combining intermediate results (3×3)

When two intermediate classifications (each Low, Medium, or High) need to be combined, the following matrix applies:

	Low	**Medium**	**High**
Low	Low	Medium	High
Medium	Medium	Medium	High
High	Medium	High	High

Table 35: 3×3 combination matrix used at each stage.

Likelihood chain (Q1–Q6)

The six likelihood questions feed through five matrices in sequence:

1. **Threat Scope** (2×2): Q1 + Q2. Classifies the size of the potential attacker pool as small, medium, or large.
2. **Protection Level** (2×2): Q3 + Q4. Classifies how complete the existing defences are.
3. **Attack Effectiveness** (3×3): Combines Threat Scope with Protection Level to estimate how effective an attack attempt would be.
4. **Occurrence Frequency** (2×2): Q5 + Q6. Classifies how often the vulnerability is exposed.
5. **Overall Likelihood** (3×3): Combines Attack Effectiveness with Occurrence Frequency to produce the final likelihood rating.

Impact chain (Q7–Q10)

The four impact questions feed through three matrices:

1. **Harm** (2×2): Q7 + Q8. Classifies the severity of consequences as limited, material, or damaging.
2. **Asset Value** (2×2): Q9 + Q10. Classifies how critical the affected asset is to the business.
3. **Overall Impact** (3×3): Combines Harm with Asset Value to produce the final impact rating.

The overall severity for each risk is then produced by one final 3×3 combination of likelihood and impact.

BRA Questions

The ten questions used to evaluate each risk are reproduced below. Questions 1 to 6 address likelihood and questions 7 to 10 address impact.

1. Can the threat event be carried out by someone with ordinary technical skills?
2. Can the threat event be carried out without significant time, money, or specialist equipment?
3. Is the affected asset currently unprotected against this threat?
4. Are there known gaps or weaknesses in the existing defences?
5. Is the vulnerability present at all times (not just under specific conditions)?
6. Can the threat event happen without any special pre-conditions being met first?
7. Will there be negative consequences felt within the organisation?

8. Will there be negative consequences felt outside the organisation (e.g. clients, regulators)?
9. Does the affected asset contribute significant business value?
10. Would the cost of repairing or replacing the affected asset be significant?